

=== ПРАВИЛА ОБРАЩЕНИЯ С ТЕКСТОМ И КОДОМ ===

<project_rules>

1. правила выдачи кода - если выдача превышает 3500 символов - выдачу необходимо разделять на части.

2. не переименовывай переменные и функции.

3. если нужны другие вызовы функций - используй перегрузку.

</project_rules>

=== ПОЛНЫЙ ТЕКСТ ДОКУМЕНТА: FileSystemNode.cs ===

```
using System.Collections.ObjectModel;
using System.ComponentModel;
using System.Runtime.CompilerServices;

namespace ContextSlicer.Filesystem;
public class FileSystemNode : INotifyPropertyChanged
{
    private bool? _isChecked = false;
    private bool _isExpanded = false;
    public bool IsExpanded
    {
        get => _isExpanded;
        set
        {
            if (_isExpanded != value)
            {
                _isExpanded = value;
                OnPropertyChanged(nameof(IsExpanded));
            }
        }
    }

    public string Name { get; set; } = string.Empty;
    public string RelativePath { get; set; } = string.Empty;
    public string FullPath { get; set; } = string.Empty;
    public bool IsFile { get; set; }
    public bool IsDirectory { get; set; }
    public FileSystemNode? Parent { get; set; }
    public ObservableCollection<FileSystemNode> Children { get; set; } = new();

    // === НОВЫЕ СВОЙСТВА ДЛЯ СИНТАКСИЧЕСКОГО ДЕРЕВА ===
    public bool IsSyntaxNode { get; set; } = false; // Флаг, что это элемент кода, а не
    файл/папка
    public EntryType SyntaxType { get; set; } // Тип элемента (Class, Method, Section...)
    public string SyntaxSpanInfo { get; set; } = string.Empty; // Координаты куска кода
    (Start,Length)
    public string EntryPath { get; set; } = string.Empty; // Уникальный синтаксический путь
    ("Class.Method")

    // Свойство, возвращающее ключ локализации для префикса (например, "Str_Type_Class")
    // Внутри FileSystemNode.cs обновите свойство TypeLocalKey:
    public string TypeLocalKey
    {
        get
        {
            if (!IsSyntaxNode) return string.Empty;

            // НОВОЕ: Мгновенный перехват литературных типов художественного текста
            if (SyntaxType == EntryType.Tab) return "Str_Type_Tab";
            if (SyntaxType == EntryType.Heading) return "Str_Type_Heading";

            // Существующая логика интеллектуального поиска SQL префиксов
            string ext = System.IO.Path.GetExtension(FullPath);
            if (ext.Equals(".sql", StringComparison.OrdinalIgnoreCase))
            {
                if (SyntaxType == EntryType.Function) return "Str_Type_SqlFunction";
                if (Name.Contains("[ПРЕДСТАВЛЕНИЕ]") || Name.Contains("[VIEW]")) return
                "Str_Type_SqlView";
                return "Str_Type_SqlTable";
            }

            // Базовый фолбэк для стандартных типов исходного кода (.cs, .js и т.д.)
            return $"Str_Type_{SyntaxType}";
        }
    }
}
```

```

}

public bool? IsChecked
{
    get => _isChecked;
    set => SetChecked(value, true, true);
}
// Добавьте это в FileSystemNode.cs для прямой передачи текста префикса в UI
// =====>
// ИСПРАВЛЕНО: УДАЛЕНИЕ ТЕХНИЧЕСКИХ ПРЕФИКСОВ ДЛЯ ХУДОЖЕСТВЕННОЙ ПРОЗЫ-->
// =====>
public string TypePrefixText
{
    get
    {
        if (!IsSyntaxNode) return string.Empty;

        // Если это глава рассказа или вкладка — префикс НЕ нужен, выводим чистый текст!
        if (SyntaxType == EntryType.Heading || SyntaxType == EntryType.Tab)
        {
            return string.Empty;
        }

        // Ваша существующая логика для исходного кода и SQL (оставляем без изменений)
        if (System.Windows.Application.Current.Resources.Contains(TypeLocalKey))
        {
            return System.Windows.Application.Current.Resources[TypeLocalKey] as string ??
            string.Empty;
        }
        return $"[{SyntaxType.ToString().ToUpper()}]";
    }
}

public void SetChecked(bool? value, bool updateChildren, bool updateParent)
{
    if (value == _isChecked) return;
    _isChecked = value;
    OnPropertyChanged(nameof(IsChecked));

    // 1. КАСКАД ВНИЗ: Если выбрали узел (папку, файл или класс), проставляем статус всем
    // детям внутри
    if (updateChildren && _isChecked.HasValue)
    {
        foreach (var child in Children)
        {
            child.SetChecked(_isChecked, true, false);
        }
    }

    // 2. КАСКАД ВВЕРХ: Обновляем состояние родительского контейнера (Indeterminate, если
    // выбрано не все)
    if (updateParent && Parent != null)
    {
        Parent.VerifyCheckState();
    }
}

public void VerifyCheckState()
{
    if (Children.Count == 0) return;

    bool hasChecked = false;
    bool hasUnchecked = false;
    bool hasIndeterminate = false;

```

```

bool hasLoadingStub = false; // НОВЫЙ ФЛАГ: Проверка на незавершенную загрузку

for (int i = 0; i < Children.Count; i++)
{
    var child = Children[i];
    if (child.Name == "LoadingStub...")
    {
        hasLoadingStub = true;
        continue; // Пропускаем заглушку при расчете математики галочек
    }

    if (child.IsChecked == true) hasChecked = true;
    else if (child.IsChecked == false) hasUnchecked = true;
    else hasIndeterminate = true;
}

bool? newState;

// ЖЕСТКОЕ ПРАВИЛО: Если под файлом висит полоса загрузки, структура НЕ полная.
// Файл не имеет права стать True (выбранным целиком), даже если все текущие видимые
// дети равны True!
if (hasLoadingStub)
{
    // Если хоть что-то выбрано – это строго квадратик Indeterminate (null). Если всё пусто
    // – False.
    newState = hasChecked || hasIndeterminate ? null : false;
}
else
{
    // Стандартная логика для полностью распарсенных файлов или обычных папок диска
    if (hasIndeterminate)
    {
        newState = null;
    }
    else if (hasChecked && hasUnchecked)
    {
        newState = null;
    }
    else if (hasChecked)
    {
        newState = true; // Файл выбран целиком, только если загрузка завершена и все дети True
    }
    else
    {
        newState = false;
    }
}

if (_isChecked != newState)
{
    _isChecked = newState;
    OnPropertyChanged(nameof(IsChecked));
}

// Передаем расчет по цепочке вверх к родительским папкам диска
Parent?.VerifyCheckState();
}

public event PropertyChangedEventHandler? PropertyChanged;
protected void OnPropertyChanged([CallerMemberName] string? name = null)
=> PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
}

```

=== ПОЛНЫЙ ТЕКСТ ДОКУМЕНТА: FileSystemService.cs ===

```
using System;
using System.Collections.Generic;
using System.Collections.ObjectModel;
using System.IO;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace ContextSlicer.Filesystem;

public class FileSystemService
{
    private readonly HashSet<string> _excludedDirs = new() { "bin", "obj", ".vs", ".git",
        ".idea", "node_modules", "publish" };

    // ИСПРАВЛЕНО: Метод теперь возвращает ObservableCollection из FileSystemNode, убирая
    конфликт типов
    public async Task<ObservableCollection<FileSystemNode>> ScanDirectoryAsync(string path)
    {
        return await Task.Run(() =>
        {
            var root = new DirectoryInfo(path);
            return new ObservableCollection<FileSystemNode> { ScanRecursive(root, null) };
        });
    }

    // ИСПРАВЛЕНО: Рекурсивный обход теперь строит дерево FileSystemNode и корректно
    связывает Parent
    private FileSystemNode ScanRecursive(DirectoryInfo dir, FileSystemNode? parent)
    {
        var node = new FileSystemNode
        {
            Name = dir.Name,
            FullPath = dir.FullName,
            IsDirectory = true, // В вашей модели папка определяется как IsFile = false, но для
            ясности используем поля FileSystemNode
            IsFile = false,
            Parent = parent
        };

        try
        {
            foreach (var subDir in dir.GetDirectories().Where(d =>
                !_excludedDirs.Contains(d.Name.ToLower())))
            {
                node.Children.Add(ScanRecursive(subDir, node));
            }

            foreach (var file in dir.GetFiles())
            {
                node.Children.Add(new FileSystemNode
                {
                    Name = file.Name,
                    FullPath = file.FullName,
                    IsFile = true,
                    Parent = node
                });
            }
        }
        catch (UnauthorizedAccessException) { /* Логгирование или пропуск */ }

        return node;
    }
}
```

=== ПОЛНЫЙ ТЕКСТ ДОКУМЕНТА: GlobalFilterService.cs ===

```
using System;
using System.Collections.Generic;
using System.Collections.ObjectModel;
using System.IO;
using System.Text.Json;

namespace ContextSlicer.Filesystem;

public class GlobalFilters
{
    public ObservableCollection<string> ExcludedFolders { get; set; } = new();
    public ObservableCollection<string> ExcludedExtensions { get; set; } = new();
}

public static class GlobalFilterService
{
    private static readonly string FilePath;
    public static GlobalFilters Current { get; private set; } = new();

    static GlobalFilterService()
    {
        string appData = Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData);
        string appDir = Path.Combine(appData, "ContextSlicer");
        Directory.CreateDirectory(appDir);
        FilePath = Path.Combine(appDir, "global_filters.json");

        LoadFilters();
    }

    public static void LoadFilters()
    {
        try
        {
            if (File.Exists(FilePath))
            {
                string json = File.ReadAllText(FilePath);
                var loaded = JsonSerializer.Deserialize<GlobalFilters>(json);
                if (loaded != null)
                {
                    Current = loaded;
                    return;
                }
            }
        }
        catch { }

        // Если файла нет или он поврежден — накатываем базовый дефолтный список
        Current = new GlobalFilters
        {
            ExcludedFolders = new ObservableCollection<string> { "bin", "obj", ".vs", "publish",
                ".git", ".idea", "node_modules" },
            ExcludedExtensions = new ObservableCollection<string> { ".png", ".jpg", ".jpeg", ".gif",
                ".ico", ".bmp", ".webp", ".mp3", ".wav", ".zip", ".rar", ".dll", ".exe", ".pdb", ".suo",
                ".user", ".bak", ".tmp", ".log" }
        };
        SaveFilters();
    }

    public static void SaveFilters()
    {
        try
        {
            if (File.Exists(FilePath))
            {
                string json = JsonSerializer.Serialize(Current);
                File.WriteAllText(FilePath, json);
            }
            else
            {
                File.Create(FilePath).Dispose();
                string json = JsonSerializer.Serialize(Current);
                File.WriteAllText(FilePath, json);
            }
        }
        catch { }
    }
}
```

```

string json = JsonSerializer.Serialize(Current, new JsonSerializerOptions {
WriteIndented = true });
File.WriteAllText(FilePath, json);
}
catch { }
}

// Теперь проверки папок и расширений работают на основе глобальных ObservableCollection
динамически
public static bool IsFolderExcluded(string folderName)
{
foreach (var f in Current.ExcludedFolders)
{
if (folderName.Equals(f, StringComparison.OrdinalIgnoreCase)) return true;
}
return false;
}

public static bool IsExtensionExcluded(string filePath)
{
string ext = Path.GetExtension(filePath);
foreach (var e in Current.ExcludedExtensions)
{
if (ext.Equals(e, StringComparison.OrdinalIgnoreCase)) return true;
}
return false;
}

}

```

=== ПОЛНЫЙ ТЕКСТ ДОКУМЕНТА: models.cs ===

```
using CommunityToolkit.Mvvm.ComponentModel;
using System;
using System.Collections.Generic;
using System.Collections.ObjectModel;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace ContextSlicer.Filesystem;

public class FileNode : ObservableObject
{
    // Добавляем = string.Empty; чтобы убрать предупреждения компилятора
    public string Name { get; set; } = string.Empty;
    public string FullPath { get; set; } = string.Empty;

    public bool IsDirectory { get; set; }

    private bool? _isChecked = false;
    public bool? IsChecked
    {
        get => _isChecked;
        set => SetProperty(ref _isChecked, value);
    }

    public ObservableCollection<FileNode> Children { get; set; } = new();
}
```

=== ПОЛНЫЙ ТЕКСТ ДОКУМЕНТА: NullToBoolConverter.cs ===

```
using System;
using System.Globalization;
using System.Windows.Data;

namespace ContextSlicer.Filesystem;

public class NullToBoolConverter : IValueConverter
{
    // Если объект НЕ null и НЕ пустая строка – возвращаем true (кнопка активна)
    public object Convert(object value, Type targetType, object parameter, CultureInfo
culture)
    {
        if (value is string str)
        {
            return !string.IsNullOrEmpty(str);
        }
        return value != null;
    }

    public object ConvertBack(object value, Type targetType, object parameter, CultureInfo
culture)
    {
        throw new NotImplementedException();
    }
}
```

=== ПОЛНЫЙ ТЕКСТ ДОКУМЕНТА: ProjectUpdateManager.cs ===

```
using System;
using System.IO;
using System.Net;
using System.Net.NetworkInformation;
using System.Threading;
using System.Threading.Tasks;
using System.Windows;

namespace ContextSlicer.Filesystem;

public enum UpdateType
{
    ReadDirectory,
    GoogleApiLoad,
    ConvertDocx
}

public class ProjectUpdateManager
{
    // Таймаут для сетевых задач (15 секунд, согласно лимитам HttpClient)
    private const int NetworkTimeoutSeconds = 15;

    /// <summary>
    /// Функция-заглушка для конвертации DOCX в Google Doc
    /// </summary>
    public async Task<bool> ConvertDocxToGoogleDocPlaceholderAsync(string filePath,
        Cancellation token)
    {
        // Имитация асинхронного процесса парсинга/конвертации
        await Task.Delay(2000, token);

        // На данном этапе возвращаем true, при подмене на рабочий код вызовется реальный парсер
        return true;
    }

    /// <summary>
    /// Единый обработчик обновления проекта
    /// </summary>
    public async Task ExecuteUpdateAsync(UpdateType type, string targetPath,
        Cancellation token)
    {
        // 1. Включаем индикацию загрузки в UI, блокируем кнопку "Отмена" для предотвращения
        гонки состояний
        SetUiLoadingState(true);
        UpdateStatusMessage(type);

        try
        {
            switch (type)
            {
                case UpdateType.ReadDirectory:
                    // Сценарий 1: Чтение локального диска или флешки
                    // При извлечении флешки упадет в IOException / UnauthorizedAccessException
                    // Метод ScanDirectoryAsync уже возвращает Task
                    var fileSystem = new FileSystemService();
                    await fileSystem.ScanDirectoryAsync(targetPath);
                    break;

                case UpdateType.GoogleApiLoad:
                    // Сценарий 2: Работа с Google API (проверяем физическую сеть)
                    if (!NetworkInterface.GetIsNetworkAvailable())
                    {
                        throw new WebException("Network is unavailable");
                    }
            }
        }
    }
}
```

```

}

using (var cts = CancellationTokenSource.CreateLinkedTokenSource(token))
{
    cts.CancelAfter(TimeSpan.FromSeconds(NetworkTimeoutSeconds));
    // Вызов метода скачивания из GoogleDownloader
    // await googleDownloader.DownloadToCacheAsync(...);
}
break;

case UpdateType.ConvertDocx:
    // Сценарий 3: Идентичен первому по типу обработки IO-ошибок
    bool convertResult = await ConvertDocxToGoogleDocPlaceholderAsync(targetPath, token);
    if (!convertResult)
    {
        throw new FormatException("Bad docx structure");
    }
    break;
}

// Успешный исход операции
CloseOverlay();
}
catch (Exception ex) when (ex is IOException || ex is UnauthorizedAccessException || ex
is FormatException)
{
    // Обработка сбоев диска/носителей (Сценарии 1 и 3)
    string errorKey = (type == UpdateType.ConvertDocx) ? "Str_Err_Update_Convert" :
    "Str_Err_Update_ReadDir";
    ShowOverlayError(errorKey, isNetworkError: false);
}
catch (Exception ex) when (ex is System.Net.WebException || ex is TaskCanceledException)
{
    // Обработка сбоев сети и таймаута (Сценарий 2)
    ShowOverlayError("Str_Err_Update_Network", isNetworkError: true);
}
finally
{
    // Разблокируем UI, возвращаем кнопку "Отмена" в активное состояние
    SetUiLoadingState(false);
}
}

private void UpdateStatusMessage(UpdateType type)
{
    string resourceKey = type switch
    {
        UpdateType.ReadDirectory => "Str_Update_ReadDir",
        UpdateType.GoogleApiLoad => "Str_Update_GoogleApi",
        UpdateType.ConvertDocx => "Str_Update_ConvertDocx",
        _ => "Str_Calculating"
    };
};

string message = Application.Current.Resources[resourceKey] as string ?? string.Empty;
// Передача строки message во View/ViewModel оверлея
}

private void ShowOverlayError(string resourceKey, bool isNetworkError)
{
    string errorMessage = Application.Current.Resources[resourceKey] as string ?? "Unknown
Error";
    // 1. Отобразить errorMessage в текстовом поле оверлея

    // 2. Если ошибка сетевая – перевести главную кнопку оверлея в режим "Повторить"
    if (isNetworkError)
    {

```

```
string retryText = Application.Current.Resources["Str_Update_BtnRetry"] as string ??  
"Retry";  
// Меняем контент/команду кнопки на повторную отправку  
}  
}  
  
private void SetUiLoadingState(bool isLoading) { /* Логика управления элементами */ }  
private void CloseOverlay() { /* Логика закрытия */ }  
}
```

=== ПОЛНЫЙ ТЕКСТ ДОКУМЕНТА: SyntaxParser.cs ===

```
using Microsoft.CodeAnalysis;
using Microsoft.CodeAnalysis.CSharp;
using Microsoft.CodeAnalysis.CSharp.Syntax;
using Newtonsoft.Json.Linq;
using System;
using System.Collections.Generic;
using System.IO;
using System.Text.RegularExpressions;
using System.Threading.Tasks;

namespace ContextSlicer.Filesystem;

// Типы элементов синтаксической структуры, которые мы поддерживаем
public enum EntryType
{
    Namespace,
    Class,
    Struct,
    Interface,
    Enum,
    Function,
    Section,
    Property,

    // ДОБАВЛЕНО: Новые типы для структуры художественных текстов
    Tab, // Для вкладок Google Docs
    Heading // Для заголовков глав рассказа (Heading 1-4)
}

// Модель элемента структуры для хранения на диске и отображения в UI
public class SyntaxEntry
{
    public string FilePath { get; set; } = string.Empty;
    public string EntryPath { get; set; } = string.Empty; // Иерархический путь, например
    "Вкладка 1/Глава 1"
    public string DisplayName { get; set; } = string.Empty;
    public EntryType Type { get; set; } = EntryType.Section;
    public string SpanInfo { get; set; } = string.Empty; // Формат: "startIndex,length"

    // НОВЫЕ СВОЙСТВА: Для быстрого и безопасного извлечения текста без парсинга строк
    [Newtonsoft.Json.JsonIgnore]
    public int StartIndex
    {
        get
        {
            if (string.IsNullOrEmpty(SpanInfo)) return 0;
            var parts = SpanInfo.Split(',');
            return parts.Length > 0 && int.TryParse(parts[0], out int res) ? res : 0;
        }
    }

    [Newtonsoft.Json.JsonIgnore]
    public int Length
    {
        get
        {
            if (string.IsNullOrEmpty(SpanInfo)) return 0;
            var parts = SpanInfo.Split(',');
            return parts.Length > 1 && int.TryParse(parts[1], out int res) ? res : 0;
        }
    }
}
```

```

// Общий интерфейс для всех языковых парсеров
public interface ISyntaxParser
{
    Task<List<SyntaxEntry>> ParseFileAsync(string absolutePath, string relativePath);
}

// ФАБРИКА ПАРСЕРОВ: определяет, какой парсер вызвать на основе расширения файла
public static class SyntaxParserFactory
{
    private static readonly HashSet<string> CodeExtensions =
        new(StringComparer.OrdinalIgnoreCase)
        {
            ".cs", ".php", ".js", ".sql", ".html"
        };

    private static readonly HashSet<string> TextExtensions =
        new(StringComparer.OrdinalIgnoreCase)
        {
            ".gdoc", ".docx", ".txt"
        };

    public static bool IsSupported(string ext)
    {
        if (string.IsNullOrEmpty(ext)) return false;
        return CodeExtensions.Contains(ext) || TextExtensions.Contains(ext);
    }

    public static ISyntaxParser? GetParser(string ext)
    {
        if (string.IsNullOrEmpty(ext)) return null;

        if (ext.Equals(".gdoc", StringComparison.OrdinalIgnoreCase))
        {
            return new GoogleDocSyntaxParser();
        }

        if (ext.Equals(".docx", StringComparison.OrdinalIgnoreCase) ||
            ext.Equals(".txt", StringComparison.OrdinalIgnoreCase))
        {
            return new TextDocumentParser();
        }

        // Ваша существующая логика выбора парсеров для кода исходников:
        return ext.ToLower() switch
        {
            ".cs" => new CSharpSyntaxParser(), // Пример вызова существующего парсера
            _ => null
        };
    }

    // 1. РЕАЛИЗАЦИЯ ДЛЯ C# (Roslyn API)
    public class CSharpSyntaxParser : ISyntaxParser
    {
        public async Task<List<SyntaxEntry>> ParseFileAsync(string absolutePath, string
            relativePath)
        {
            var result = new List<SyntaxEntry>();
            try
            {
                string code = await File.ReadAllTextAsync(absolutePath);
                SyntaxTree tree = CSharpSyntaxTree.ParseText(code);
                CompilationUnitSyntax root = tree.GetCompilationUnitRoot();
            }
            catch { }
        }
    }
}

```

```

if (root != null)
{
    // Блок импортов
    if (root.Usings.Count > 0)
    {
        int start = root.Usings[0].Span.Start;
        int length = root.Usings[root.Usings.Count - 1].Span.End - start;

        result.Add(new SyntaxEntry
        {
            FilePath = relativePath,
            EntryPath = "Global.Usings",
            DisplayName = "[Импорты (using)]",
            Type = EntryType.Section,
            SpanInfo = $"{start},{length}"
        });
    }

    // Запускаем плоский сбор элементов
    ParseNodeFlat(root, relativePath, string.Empty, result);
}
}
catch { /* Файл занят */ }
return result;
}

private void ParseNodeFlat(SyntaxNode node, string relPath, string parentPath,
List<SyntaxEntry> resultList)
{
    foreach (var child in node.ChildNodes())
    {
        SyntaxEntry? entry = null;
        string currentName = string.Empty;
        bool isContainer = false;

        if (child is NamespaceDeclarationSyntax ns)
        {
            currentName = ns.Name.ToString();
            entry = new SyntaxEntry { Type = EntryType.Namespace, DisplayName = currentName };
            isContainer = true;
        }
        else if (child is FileScopedNamespaceDeclarationSyntax fsns)
        {
            currentName = fsns.Name.ToString();
            entry = new SyntaxEntry { Type = EntryType.Namespace, DisplayName = currentName };
            isContainer = true;
        }
        else if (child is ClassDeclarationSyntax cls)
        {
            currentName = cls.Identifier.Text;
            entry = new SyntaxEntry { Type = EntryType.Class, DisplayName = currentName };
            isContainer = true;
        }
        else if (child is StructDeclarationSyntax str)
        {
            currentName = str.Identifier.Text;
            entry = new SyntaxEntry { Type = EntryType.Struct, DisplayName = currentName };
            isContainer = true;
        }
        else if (child is InterfaceDeclarationSyntax itemFace)
        {
            currentName = itemFace.Identifier.Text;
            entry = new SyntaxEntry { Type = EntryType.Interface, DisplayName = currentName };
            isContainer = true;
        }
        else if (child is EnumDeclarationSyntax en)
        {

```

```

currentName = en.Identifier.Text;
entry = new SyntaxEntry { Type = EntryType.Enum, DisplayName = currentName };
isContainer = true;
}
else if (child is MethodDeclarationSyntax method)
{
currentName = method.Identifier.Text;
entry = new SyntaxEntry { Type = EntryType.Function, DisplayName = $"{currentName}()" };
}
else if (child is GlobalStatementSyntax gs && gs.Statement is
LocalFunctionStatementSyntax lfs)
{
currentName = lfs.Identifier.Text;
entry = new SyntaxEntry { Type = EntryType.Function, DisplayName = $"{currentName}()
[Top-Level]" };
}

if (entry != null)
{
string fullEntryPath = string.IsNullOrEmpty(parentPath) ? currentName :
$"{parentPath}.{currentName}";
entry.FilePath = relPath;
entry.EntryPath = fullEntryPath;
entry.SpanInfo = $"{child.Span.Start},{child.Span.Length}";
resultList.Add(entry);

// Если это контейнер (класс, namespace), продолжаем сканировать ЕГО детей, прокидывая
обновленный путь родителя
if (isContainer)
{
ParseNodeFlat(child, relPath, fullEntryPath, resultList);
}
}
else if (child is not UsingDirectiveSyntax)
{
// Если узел промежуточный, просто идем глубь
ParseNodeFlat(child, relPath, parentPath, resultList);
}
}
}

// 2. УМНАЯ РЕАЛИЗАЦИЯ ДЛЯ JAVASCRIPT / TYPESCRIPT (С автоматическим вычислением тела
функций)
public class JavaScriptSyntaxParser : ISyntaxParser
{
public async Task<List<SyntaxEntry>> ParseFileAsync(string absolutePath, string
relativePath)
{
var result = new List<SyntaxEntry>();
try
{
string content = await File.ReadAllTextAsync(absolutePath);
// Ищем классические функции или стрелочные константы
var funcRegex = new Regex(@"(?:function\s+([a-zA-Z0-9_]+)|(?:const|let|var)\s+([a-zA-Z0-9_]+)\s*=\s*(?:\([^)]*\)|[a-zA-Z0-9_]+\s*>)", RegexOptions.Compiled);
var matches = funcRegex.Matches(content);

foreach (Match m in matches)
{
// Извлекаем имя из первой или второй группы захвата
string name = !string.IsNullOrEmpty(m.Groups[1].Value) ? m.Groups[1].Value :
m.Groups[2].Value;

```

```

if (string.IsNullOrEmpty(name)) continue;

// Вычисляем реальную длину тела функции по балансу фигурных скобок
int totalLength = FindJsBlockLength(content, m.Index);

result.Add(new SyntaxEntry
{
    FilePath = relativePath,
    EntryPath = name,
    DisplayName = $"{name}()",
    Type = EntryType.Function,
    SpanInfo = $"{m.Index},{totalLength}" // ТЕПЕРЬ ПЕРЕДАЕТСЯ ПОЛНЫЙ КОД JS ФУНКЦИИ!
});
}
}
catch { }
return result;
}

// Алгоритм баланса фигурных скобок для JS/TS функций
private int FindJsBlockLength(string content, int startIndex)
{
    int openBracePos = content.IndexOf('{', startIndex);
    if (openBracePos == -1)
    {
        // Если это стрелочная однострочная функция без фигурных скобок, забираем её до конца строки
        int endOfLine = content.IndexOf('\n', startIndex);
        return endOfLine != -1 ? endOfLine - startIndex : content.Length - startIndex;
    }

    int braceCount = 1;
    int i = openBracePos + 1;

    for (; i < content.Length; i++)
    {
        if (content[i] == '{') braceCount++;
        else if (content[i] == '}')
        {
            braceCount--;
            if (braceCount == 0)
            {
                i++; // Включаем закрывающую скобку } в диапазон
                break;
            }
        }
    }

    if (i > content.Length) i = content.Length;
    return i - startIndex;
}

// 3. УМНАЯ РЕАЛИЗАЦИЯ ДЛЯ HTML (Захват полноценных тегов вместе с их содержимым)
public class HtmlSyntaxParser : ISyntaxParser
{
    public async Task<List<SyntaxEntry>> ParseFileAsync(string absolutePath, string relativePath)
    {
        var result = new List<SyntaxEntry>();
        try
        {
            string content = await File.ReadAllTextAsync(absolutePath);
            // Находим теги с id логических блоков разметки
            var tagRegex = new

```

```

Regex(@"<(section|header|footer|main|div)\s+[^>]*id=["'"] ([^"'"']+) ["'"] [^>]*>",
RegexOptions.Compiled | RegexOptions.IgnoreCase);
var matches = tagRegex.Matches(content);

foreach (Match m in matches)
{
    string tagName = m.Groups[1].Value;
    string idValue = m.Groups[2].Value;

    if (string.IsNullOrEmpty(tagName) || string.IsNullOrEmpty(idValue)) continue;

    // Находим точную позицию закрывающего тега разметки </tag>, учитывая вложенность
    int totalLength = FindHtmlBlockLength(content, m.Index, tagName);

    result.Add(new SyntaxEntry
    {
        FilePath = relativePath,
        EntryPath = $"{tagName}#{idValue}",
        DisplayName = $"<{tagName} id=\"{idValue}\">",
        Type = EntryType.Section,
        SpanInfo = $"{m.Index},{totalLength}" // ТЕПЕРЬ HTML ВЫГРУЖАЕТСЯ БЛОКОМ ЦЕЛИКОМ!
    });
}
catch { }
return result;
}

// Алгоритм поиска парного закрывающего HTML-тега
private int FindHtmlBlockLength(string content, int startIndex, string tagName)
{
    string openPattern = $"<{tagName}";
    string closePattern = $"</{tagName}>";

    int currentPos = startIndex + openPattern.Length;
    int tagCount = 1;

    while (currentPos < content.Length)
    {
        // Ищем следующее открытие или закрытие этого тега
        int nextOpen = content.IndexOf(openPattern, currentPos,
StringComparison.OrdinalIgnoreCase);
        int nextClose = content.IndexOf(closePattern, currentPos,
StringComparison.OrdinalIgnoreCase);

        // Если закрывающих тегов больше нет — забираем до конца файла (защита от битой
разметки)
        if (nextClose == -1) return content.Length - startIndex;

        // Если нашли открытие вложенного такого же тега ДО закрытия текущего
        if (nextOpen != -1 && nextOpen < nextClose)
        {
            tagCount++;
            currentPos = nextOpen + openPattern.Length;
        }
        else
        {
            tagCount--;
            if (tagCount == 0)
            {
                // Нашли парный закрывающий тег. Включаем его длину в диапазон
                return nextClose + closePattern.Length - startIndex;
            }
            currentPos = nextClose + closePattern.Length;
        }
    }
}

```

```

}

return content.Length - startIndex;
}
}

// 4. ЛЕГКОВЕСНАЯ РЕАЛИЗАЦИЯ ДЛЯ PHP (Объектно-ориентированный и процедурный код)
// 4. УМНАЯ РЕАЛИЗАЦИЯ ДЛЯ PHP (С автоматическим вычислением тела функций и классов)
public class PhpSyntaxParser : ISyntaxParser
{
    public async Task<List<SyntaxEntry>> ParseFileAsync(string absolutePath, string
relativePath)
    {
        var result = new List<SyntaxEntry>();
        try
        {
            string content = await File.ReadAllTextAsync(absolutePath);

            // Находит namespace, class, interface, trait и функции
            var phpRegex = new Regex(@"(?:namespace\s+([a-zA-Z0-9_\\]+\s+);)|(?:class|interface|trait)\s+([a-zA-Z0-9_]+\s+)|(?:function\s+([a-zA-Z0-9_]+\s+)\s*\()", RegexOptions.Compiled | RegexOptions.IgnoreCase);
            var matches = phpRegex.Matches(content);

            string currentParentPath = string.Empty;

            foreach (Match m in matches)
            {
                // 1. Обработка Namespace
                if (m.Groups[1].Success)
                {
                    string nsName = m.Groups[1].Value;
                    currentParentPath = nsName;
                    result.Add(new SyntaxEntry
                    {
                        FilePath = relativePath,
                        EntryPath = nsName,
                        DisplayName = nsName,
                        Type = EntryType.Namespace,
                        SpanInfo = $"{m.Index},{m.Length}"
                    });
                }
                // 2. Обработка Класса / Интерфейса
                else if (m.Groups[2].Success)
                {
                    string className = m.Groups[2].Value;
                    string fullPath = string.IsNullOrEmpty(currentParentPath) ? className :
                    $"{currentParentPath}.{className}";
                    currentParentPath = fullPath;

                    // Вычисляем длину класса по скобкам
                    int totalLength = FindBlockLength(content, m.Index);

                    result.Add(new SyntaxEntry
                    {
                        FilePath = relativePath,
                        EntryPath = fullPath,
                        DisplayName = className,
                        Type = EntryType.Class,
                        SpanInfo = $"{m.Index},{totalLength}"
                    });
                }
                // 3. Обработка Методов и Функций PHP
                else if (m.Groups[3].Success)
            }

```

```

{
string funcName = m.Groups[3].Value;
string fullPath = string.IsNullOrEmpty(currentParentPath) ? funcName :
${currentParentPath}.${funcName}";

// Находим реальную длину тела функции бэкенда со всеми внутренностями
int totalLength = FindBlockLength(content, m.Index);

result.Add(new SyntaxEntry
{
FilePath = relativePath,
EntryPath = fullPath,
DisplayName = "${funcName}()",
Type = EntryType.Function,
SpanInfo = $"{m.Index},{totalLength}" // ТЕПЕРЬ ПЕРЕДАЕТСЯ ПОЛНЫЙ ВЕС ФУНКЦИИ!
});
}
}
}
catch { }
return result;
}

// Алгоритм баланса скобок: находит конец программного блока { ... }
private int FindBlockLength(string content, int startIndex)
{
int openBracePos = content.IndexOf('{', startIndex);
if (openBracePos == -1) return content.Length - startIndex; // Если скобок нет, берем до
конца строки

int braceCount = 1;
int i = openBracePos + 1;

for (; i < content.Length; i++)
{
if (content[i] == '{') braceCount++;
else if (content[i] == '}')
{
braceCount--;
if (braceCount == 0)
{
i++; // Включаем закрывающую скобку в диапазон
break;
}
}
}

if (i > content.Length) i = content.Length;
return i - startIndex;
}
}

// 5. УМНАЯ РЕАЛИЗАЦИЯ ДЛЯ SQL ФАЙЛОВ (С автоматическим вычислением тела таблиц и
процедур)
public class SqlSyntaxParser : ISyntaxParser
{
public async Task<List<SyntaxEntry>> ParseFileAsync(string absolutePath, string
relativePath)
{
var result = new List<SyntaxEntry>();
try

```

```

{
string content = await File.ReadAllTextAsync(absolutePath);

// Ищем конструкции: CREATE TABLE имя, CREATE PROCEDURE имя, CREATE VIEW имя, CREATE
// FUNCTION имя
var sqlRegex = new
Regex(@"CREATE\s+(?:OR\s+ALTER\s+)?(TABLE|PROCEDURE|PROC|VIEW|FUNCTION)\s+([a-zA-Z0-9_""\.\`[\]]+)", RegexOptions.Compiled | RegexOptions.IgnoreCase);
var matches = sqlRegex.Matches(content);

foreach (Match m in matches)
{
if (m.Groups.Count < 3) continue;

string objectType = m.Groups[1].Value.ToUpper();
string objectName = m.Groups[2].Value.Replace("[", "").Replace("]", "").Replace("`",
 "").Replace("\\"", ""); // Очищаем от экранирования

if (string.IsNullOrEmpty(objectName)) continue;

// 1. СНАЧАЛА вычисляем длину блока SQL-объекта
int totalLength = FindSqlBlockLength(content, m.Index, objectType);

// 2. Переводим тип в понятный для нашего дерева префикс (связываем с ключами
// локализации)
EntryType type = EntryType.Section;
string localKey = "";

if (objectType == "PROCEDURE" || objectType == "PROC" || objectType == "FUNCTION")
{
type = EntryType.Function;
localKey = "Str_Type_SqlFunction";
}
else if (objectType == "TABLE")
{
type = EntryType.Section;
localKey = "Str_Type_SqlTable";
}
else if (objectType == "VIEW")
{
type = EntryType.Section;
localKey = "Str_Type_SqlView";
}

// 3. Вытаскиваем локализованную строку префикса из ресурсов приложения
string displayPrefix = $"[{objectType}]";
if (!string.IsNullOrEmpty(localKey) &&
System.Windows.Application.Current.Resources.Contains(localKey))
{
displayPrefix = System.Windows.Application.Current.Resources[localKey] as string ??
displayPrefix;
}

// 4. Безопасно добавляем запись в итоговый синтаксический список
result.Add(new SyntaxEntry
{
FilePath = relativePath,
EntryPath = objectName,
DisplayName = $"{displayPrefix} {objectName}", // Строка формируется локализованной
Type = type,
SpanInfo = $"{m.Index},{totalLength}" // Ошибка CS0103 полностью устранена
});
}
}

```

```

}
catch { }
return result;
}

// Алгоритм определения границ SQL объектов
private int FindSqlBlockLength(string content, int startIndex, string objectType)
{
    if (objectType == "TABLE")
    {
        // Для таблиц ищем первую открывающую круглую скобку после слова CREATE TABLE
        int openParenthesisPos = content.IndexOf('(', startIndex);
        if (openParenthesisPos == -1) return content.Length - startIndex;

        int parenthesisCount = 1;
        int i = openParenthesisPos + 1;

        // Идем по тексту и считаем баланс круглых скобок (учитывая вложенные типы данных,
        // например DECIMAL(10,2))
        for (; i < content.Length; i++)
        {
            if (content[i] == '(') parenthesisCount++;
            else if (content[i] == ')')
            {
                parenthesisCount--;
                if (parenthesisCount == 0)
                {
                    // Нашли закрывающую скобку таблицы. Продвигаемся чуть дальше, чтобы захватить точку с
                    // запятой, если она есть
                    i++;
                    if (i < content.Length && content[i] == ';') i++;
                    break;
                }
            }
        }
        if (i > content.Length) i = content.Length;
        return i - startIndex;
    }
    else
    {
        // Для процедур, функций и представлений забираем текст до следующей точки с запятой
        // или до начала следующего оператора CREATE (что наступит раньше)
        int nextSemicolon = content.IndexOf(';', startIndex + 10);

        // Проверяем, нет ли поблизости следующего оператора CREATE
        var nextCreateMatch = Regex.Match(content.Substring(startIndex + 10), @"\bCREATE\b",
            RegexOptions.IgnoreCase);
        int nextCreatePos = nextCreateMatch.Success ? startIndex + 10 + nextCreateMatch.Index :
            -1;

        int endPos = content.Length;

        if (nextSemicolon != -1 && nextCreatePos != -1)
            endPos = Math.Min(nextSemicolon + 1, nextCreatePos);
        else if (nextSemicolon != -1)
            endPos = nextSemicolon + 1;
        else if (nextCreatePos != -1)
            endPos = nextCreatePos;

        return endPos - startIndex;
    }
}

public class TextDocumentParser : ISyntaxParser
{
    private const int TargetPageSize = 1800;

```

```

public async Task<List<SyntaxEntry>> ParseFileAsync(string absolutePath, string
relativePath)
{
var result = new List<SyntaxEntry>();

if (!File.Exists(absolutePath)) return result;

// Асинхронно читаем весь текст файла на диске
string text = await File.ReadAllTextAsync(absolutePath);
if (string.IsNullOrEmpty(text)) return result;

// ШАГ 1: Поиск системных разрывов страниц (\f)
var pageBreakMatches = Regex.Matches(text, @"\f");
if (pageBreakMatches.Count > 0)
{
int lastIndex = 0;
for (int i = 0; i <= pageBreakMatches.Count; i++)
{
int nextIndex = i < pageBreakMatches.Count ? pageBreakMatches[i].Index : text.Length;
int length = nextIndex - lastIndex;
string partPrefix = System.Windows.Application.Current.Resources["Str_TextPart"] as
string ?? "ЧАСТЬ";
if (length > 0)
{
result.Add(new SyntaxEntry
{
FilePath = relativePath,
EntryPath = $"PageBreak/Part_{i + 1}",
DisplayName = $"[{partPrefix} {i + 1}]",
Type = EntryType.Section,
SpanInfo = $"{lastIndex},{length}"
});
}
lastIndex = nextIndex + 1;
}
return result;
}

// ШАГ 2: ФОЛБЭК А4 – Умная нарезка с контролем переноса слов и знаков препинания
int currentIndex = 0;
int pageNumber = 1;

while (currentIndex < text.Length)
{
int remainingLength = text.Length - currentIndex;
if (remainingLength <= TargetPageSize)
{
result.Add(CreatePageEntry(relativePath, pageNumber, currentIndex, remainingLength));
break;
}

int potentialBreak = currentIndex + TargetPageSize;
int actualBreak = potentialBreak;

// Ищем пробел перед концом страницы, чтобы не резать слово в центре
while (actualBreak > currentIndex + TargetPageSize / 2)
{
char c = text[actualBreak];
if (char.IsWhiteSpace(c) && actualBreak > 0 && !char.IsPunctuation(text[actualBreak -
1]))
{
break;
}
actualBreak--;
}
}

```

```

// Если не нашли хорошей границы — сдвигаемся вперед до ближайшего пробела
if (actualBreak <= currentIndex + TargetPageSize / 2)
{
    actualBreak = potentialBreak;
    while (actualBreak < text.Length && !char.IsWhiteSpace(text[actualBreak]))
    {
        actualBreak++;
    }
}

int pageLength = actualBreak - currentIndex;
result.Add(CreatePageEntry(relativePath, pageNumber, currentIndex, pageLength));

currentIndex = actualBreak;
while (currentIndex < text.Length && char.IsWhiteSpace(text[currentIndex]))
{
    currentIndex++;
}
pageNumber++;
}

return result;
}

private SyntaxEntry CreatePageEntry(string path, int pageNum, int start, int len)
{
    string pagePrefix = System.Windows.Application.Current.Resources["Str_TextPage"] as
    string ?? "Страница";

    return new SyntaxEntry
    {
        FilePath = path,
        EntryPath = $"A4/Page_{pageNum}",
        DisplayName = $"{pagePrefix} {pageNum}",
        Type = EntryType.Section,
        SpanInfo = $"{start},{len}"
    };
}
}

```

=== ПОЛНЫЙ ТЕКСТ ДОКУМЕНТА: GoogleDownloader.cs ===

```
using System;
using System.IO;
using System.Net.Http;
using System.Net.Http.Headers;
using System.Text.RegularExpressions;
using System.Threading.Tasks;
using Newtonsoft.Json.Linq;

namespace ContextSlicer.Google;

public class GoogleDownloader
{
    private static readonly HttpClient _httpClient = new HttpClient { Timeout =
    TimeSpan.FromSeconds(15) };
    private const string CacheFolderName = "googlecache";
    private static readonly TimeSpan CacheLifetime = TimeSpan.FromMinutes(15);

    public static string ExtractDocumentId(string url)
    {
        if (string.IsNullOrEmpty(url)) return string.Empty;
        var match = Regex.Match(url, @"^/document/d/([a-zA-Z0-9-]+)");
        return match.Success ? match.Groups[1].Value : string.Empty;
    }

    public bool IsCacheExpired(string url, string projectLocation)
    {
        string docId = ExtractDocumentId(url);
        if (string.IsNullOrEmpty(docId)) return true;
        string metaFilePath = Path.Combine(projectLocation, CacheFolderName, $"{docId}.meta");
        string cacheFilePath = Path.Combine(projectLocation, CacheFolderName, $"{docId}.json");
        if (!File.Exists(cacheFilePath) || !File.Exists(metaFilePath)) return true;
        try
        {
            string metaContent = File.ReadAllText(metaFilePath);
            if (DateTime.TryParse(metaContent, out DateTime lastDownloadTime))
            {
                return (DateTime.Now - lastDownloadTime) > CacheLifetime;
            }
        }
        catch { return true; }
        return false;
    }

    // ОБНОВЛЕНО: Теперь в третий аргумент передается путь к скачанному JSON-файлу ключа
    public async Task<bool> DownloadToCacheAsync(string url, string projectLocation, string
    authJsonPath)
    {
        string docId = ExtractDocumentId(url);
        if (string.IsNullOrEmpty(docId) || string.IsNullOrEmpty(authJsonPath) ||
        !File.Exists(authJsonPath)) return false;

        string cacheDir = Path.Combine(projectLocation, CacheFolderName);
        if (!Directory.Exists(cacheDir)) Directory.CreateDirectory(cacheDir);

        string cacheFilePath = Path.Combine(cacheDir, $"{docId}.json");
        string metaFilePath = Path.Combine(cacheDir, $"{docId}.meta");

        try
        {
            // 1. Читаем параметры из JSON-файла сервисного аккаунта
            string credentialsText = await File.ReadAllTextAsync(authJsonPath);
            var creds = JObject.Parse(credentialsText);
            string clientEmail = creds["client_email"]?.ToString() ?? string.Empty;
```

```

string privateKeyRaw = creds["private_key"]?.ToString() ?? string.Empty;

if (string.IsNullOrEmpty(clientEmail) || string.IsNullOrEmpty(privateKeyRaw)) return
false;

// 2. ПОЛУЧАЕМ ACCESS TOKEN ЧЕРЕЗ JWT (Автономная реализация без внешних библиотек)
string accessToken = await GetGoogleAccessTokenAsync(clientEmail, privateKeyRaw);
if (string.IsNullOrEmpty(accessToken)) return false;

// 3. СКАЧИВАЕМ ДОКУМЕНТ С ОФИЦИАЛЬНЫМ ТОКЕНОМ АВТОРИЗАЦИИ
// ИСПРАВЛЕНО: Сборка канонического адреса Docs API по частям без использования слэшей в
строке
var request = new HttpRequestMessage(HttpMethod.Get, "https" + ":" + "/" + "docs" + "." +
+ "googleapis" + "." + "com" + "/" + "v1" + "/" + "documents" + "/" + docId +
"?includeTabsContent=true");

request.Headers.Authorization = new AuthenticationHeaderValue("Bearer", accessToken);

var response = await _httpClient.SendAsync(request);
if (!response.IsSuccessStatusCode)
{
    string errLog = await response.Content.ReadAsStringAsync();
    System.Diagnostics.Debug.WriteLine($"[Google API Error] {response.StatusCode}:
{errLog}");
    return false;
}

string jsonResponse = await response.Content.ReadAsStringAsync();
var docStructure = JObject.Parse(jsonResponse);
if (docStructure["documentId"] == null) return false;

// Сохраняем структуру и обновляем время жизни кэша
await File.WriteAllTextAsync(cacheFilePath, jsonResponse);
await File.WriteAllTextAsync(metaFilePath, DateTime.Now.ToString("o"));
return true;
}
catch (Exception ex)
{
    System.Diagnostics.Debug.WriteLine($"[Google Downloader Exception] {ex.Message}");
    return false;
}
}

// Вспомогательный метод генерации OAuth2 токена на основе RSA-подписи ключа
private async Task<string> GetGoogleAccessTokenAsync(string clientEmail, string
privateKeyRaw)
{
    try
    {
        // Формируем стандартные JSON-блоки для JWT Claims по спецификации Google
        var now = (int)(DateTime.UtcNow - new DateTime(1970, 1, 1)).TotalSeconds;

        var header = new JObject { ["alg"] = "RS256", ["typ"] = "JWT" };
        var payload = new JObject
        {
            ["iss"] = clientEmail,
            // ИСПРАВЛЕНО: Полный корректный scope доступа к документам
            ["scope"] = "https" + ":" + "/" + "www" + "." + "googleapis" + "." + "com" + "/" +
            "auth" + "/" + "documents" + "." + "readonly",
            // ИСПРАВЛЕНО: Точный адрес aud, совпадающий с адресом отправки POST-запроса токена
            ["aud"] = "https" + ":" + "/" + "oauth2" + "." + "googleapis" + "." + "com" + "/" +
            "token",
            ["exp"] = now + 3600,
            ["iat"] = now
        };
    }
}

```

```

// Кодируем блоки в формат Base64Url
string encodeHeader =
Convert.ToBase64String(System.Text.Encoding.UTF8.GetBytes(header.ToString())).Replace("+",
"-").Replace("/", "_").Replace("=", "");
string encodePayload =
Convert.ToBase64String(System.Text.Encoding.UTF8.GetBytes(payload.ToString())).Replace("+",
"-").Replace("/", "_").Replace("=", "");
string assertionInput = $"{encodeHeader}.{encodePayload}";

// Очищаем сырой RSA Private Key от заголовков PEM-формата для системного
криптопровайдера .NET
string pemBody = privateKeyRaw
.Replace("-----BEGIN PRIVATE KEY-----", "")
.Replace("-----END PRIVATE KEY-----", "")
.Replace("\n", "").Replace("\r", "").Trim();

byte[] privateKeyBytes = Convert.FromBase64String(pemBody);

// Подписываем токен алгоритмом RS256
using (var rsa = System.Security.Cryptography.RSA.Create())
{
rsa.ImportPkcs8PrivateKey(privateKeyBytes, out _);
byte[] inputBytes = System.Text.Encoding.UTF8.GetBytes(assertionInput);
byte[] signatureBytes = rsa.SignData(inputBytes,
System.Security.Cryptography.HashAlgorithmName.SHA256,
System.Security.Cryptography.RSASignaturePadding.Pkcs1);

string signature = Convert.ToBase64String(signatureBytes).Replace("+", "-").Replace("/",
"_").Replace("=", "");
string jwtToken = $"{assertionInput}.{signature}";

// Обмениваем подписанный JWT-токен на реальный рабочий AccessToken
var dict = new System.Collections.Generic.Dictionary<string, string>
{
{ "grant_type", "urn:ietf:params:oauth:grant-type:jwt-bearer" },
{ "assertion", jwtToken }
};

var content = new FormUrlEncodedContent(dict);
var tokenResponse = await _httpClient.PostAsync("https" + ":" + "/" + "oauth2" + "." +
"googleapis" + "." + "com" + "/" + "token", content);

if (!tokenResponse.IsSuccessStatusCode) return string.Empty;

string tokenJson = await tokenResponse.Content.ReadAsStringAsync();
var tokenObj = JObject.Parse(tokenJson);
return tokenObj["access_token"]?.ToString() ?? string.Empty;
}
catch
{
return string.Empty;
}
}

```

=== ПОЛНЫЙ ТЕКСТ ДОКУМЕНТА: SecureCredentialStorage.cs ===

```
using System;
using System.Text;
using System.Security.Cryptography;

namespace ContextSlicer.Google;

public static class SecureCredentialStorage
{
    private static readonly byte[] Entropy = new byte[] { 0x47, 0x6f, 0x6f, 0x67, 0x6c,
        0x65, 0x4b, 0x65, 0x79 };

    public static string EncryptString(string plainText)
    {
        if (string.IsNullOrEmpty(plainText)) return string.Empty;
        try
        {
            byte[] plainBytes = Encoding.UTF8.GetBytes(plainText);
            byte[] encryptedBytes = ProtectedData.Protect(plainBytes, Entropy,
                DataProtectionScope.CurrentUser);
            return Convert.ToBase64String(encryptedBytes);
        }
        catch
        {
            return string.Empty;
        }
    }

    public static string DecryptString(string encryptedText)
    {
        if (string.IsNullOrEmpty(encryptedText)) return string.Empty;
        try
        {
            byte[] encryptedBytes = Convert.FromBase64String(encryptedText);
            byte[] plainBytes = ProtectedData.Unprotect(encryptedBytes, Entropy,
                DataProtectionScope.CurrentUser);
            return Encoding.UTF8.GetString(plainBytes);
        }
        catch
        {
            return string.Empty;
        }
    }
}
```

=== ПОЛНЫЙ ТЕКСТ ДОКУМЕНТА: Strings.en.xaml ===

```
<ResourceDictionary xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:system="clr-namespace:System;assembly=System.Runtime">
<!-- Заголовки блоков -->
<system:String x:Key="Str_ProjManagement">1. Project Management</system:String>
<system:String x:Key="Str_ModuleManagement">2. Slicing Modules / AI
Contexts</system:String>
<system:String x:Key="Str_FilesHeader">Current Module Files</system:String>
<!-- Разделенные заголовки правил prompt -->
<system:String x:Key="Str_PromptHeader">Project General Rules (Prompt)</system:String>
<system:String x:Key="Str_ModulePromptHeader">Current Module Rules
(Prompt)</system:String>

<!-- Элементы блока проекта -->
<system:String x:Key="Str_SelectProj">Select Project:</system:String>
<system:String x:Key="Str_ProjName">Project Name:</system:String>
<system:String x:Key="Str_CreateProj">Create Project</system:String>
<system:String x:Key="Str_Autosave">Autosave on exit</system:String>
<system:String x:Key="Str_DarkTheme">Dark theme</system:String>
<system:String x:Key="Str_RootPath">Root folder (code):</system:String>
<system:String x:Key="Str_OutputPath">Output folder (contexts):</system:String>
<system:String x:Key="Str_Browse">Browse...</system:String>
<system:String x:Key="Str_IncludeStructure">Include directory structure</system:String>

<!-- Элементы блока модулей -->
<system:String x:Key="Str_SelectModule">Select Module:</system:String>
<system:String x:Key="Str_ModuleName">Module Name:</system:String>
<system:String x:Key="Str_AddModule">Add Module</system:String>
<system:String x:Key="Str_FileFormat">File Format:</system:String>

<!-- Главные кнопки -->
<system:String x:Key="Str_SaveProject">SAVE PROJECT CHANGES</system:String>
<system:String x:Key="Str_Generate">GENERATE MODULE CONTEXT FILE</system:String>

<!-- Оверлей прогресса -->
<system:String x:Key="Str_Cancel">CANCEL OPERATION</system:String>

<!-- Фолбэк для заголовка экспандера -->
<system:String x:Key="Str_ProjNotSelected">Project not selected</system:String>
<system:String x:Key="Str_AppTitle">Code Context Slicer for AI</system:String>
<system:String x:Key="Str_DeleteModule">Delete Module</system:String>
<system:String x:Key="Str_GlobalFilters">Global Filters</system:String>
<system:String x:Key="Str_MnuIgnoreItem">Add to exclusion filter</system:String>
<!-- Расчёт токенов-->
<system:String x:Key="Str_StatsChars">Total characters:</system:String>
<system:String x:Key="Str_StatsTokens">Estimated tokens:</system:String>
<system:String x:Key="Str_Calculating">Calculating size...</system:String>
<!-- Syntax tree element prefixes -->
<system:String x:Key="Str_Type_Namespace">[NAMESPACE]</system:String>
<system:String x:Key="Str_Type_Class">[CLASS]</system:String>
<system:String x:Key="Str_Type_Struct">[STRUCT]</system:String>
<system:String x:Key="Str_Type_Interface">[INTERFACE]</system:String>
<system:String x:Key="Str_Type_Enum">[ENUM]</system:String>
<system:String x:Key="Str_Type_Function">[METHOD]</system:String>
<system:String x:Key="Str_Type_Section">[SECTION]</system:String>
<system:String x:Key="Str_LoadingStub">Loading structure...</system:String>
<!-- Specific SQL prefixes -->
<system:String x:Key="Str_Type_SqlTable">[TABLE]</system:String>
<system:String x:Key="Str_Type_SqlView">[VIEW]</system:String>
<system:String x:Key="Str_Type_SqlFunction">[SQL FUNCTION]</system:String>

<!-- Module rename block (Overlay) -->
<system:String x:Key="Str_ModuleNameLabel">Module Name:</system:String>
<system:String x:Key="Str_RenameTitle">Rename Module</system:String>
<system:String x:Key="Str_BtnCancel">Cancel</system:String>
```

```

<system:String x:Key="Str_BtnAccept">Accept</system:String>
<system:String x:Key="Str_TooltipRename">Rename module</system:String>
<!-- Tooltips for project and module control buttons -->
<system:String x:Key="Str_TooltipProjectEdit">Project settings</system:String>
<system:String x:Key="Str_TooltipProjectSync">Synchronize / Reread
source</system:String>
<system:String x:Key="Str_TooltipModuleAdd">Add new module</system:String>
<system:String x:Key="Str_TooltipModuleEdit">Rename current module</system:String>
<system:String x:Key="Str_TooltipModuleDelete">Delete current module</system:String>
<!-- Localization of text document nodes -->
<system:String x:Key="Str_TextPart">PART</system:String>
<system:String x:Key="Str_TextPage">Page</system:String>
<!-- Shortened prefixes for literary text -->
<system:String x:Key="Str_Type_Tab">[TAB]</system:String>
<system:String x:Key="Str_Type_Heading">[CH]</system:String>

<system:String x:Key="Str_SourceDataLabel">Data Source:</system:String>
<!-- Network cloud interaction errors -->
<system:String x:Key="Str_Err_GoogleDocUnavailable">Google Document is unavailable!
Please check the link access, API key validity, and your network
connection.</system:String>
<system:String x:Key="Str_Err_NetworkTitle">Network Error</system:String>
<!-- Google Service Accounts Manager Localization -->
<system:String x:Key="Str_Err_InvalidServiceAccount">The selected file is not a valid
Google Service Account key!</system:String>
<system:String x:Key="Str_Err_ValidationTitle">Validation Error</system:String>
<system:String x:Key="Str_Err_ImportFailed">Failed to read or import the
key:</system:String>
<system:String x:Key="Str_Err_ImportTitle">Import Error</system:String>
<system:String x:Key="Str_Msg_EmailCopied">The service account email has been
successfully copied to the clipboard:&#x0a;&#x0a;{0}&#x0a;&#x0a;Paste it into the access
settings of your Google Document!</system:String>
<system:String x:Key="Str_Msg_EmailCopiedTitle">Success</system:String>
<system:String x:Key="Str_Err_ExtractEmailFailed">Failed to extract
Email:</system:String>
<system:String x:Key="Str_Err_GeneralErrorTitle">Error</system:String>
<!-- System key dialog parameters -->
<system:String x:Key="Str_Dialog_JsonFilter">Google API Key (*.json)|*.json|All Files
(*.*)|*.*</system:String>
<system:String x:Key="Str_Dialog_JsonTitle">Select the downloaded service account JSON
key file</system:String>
<system:String x:Key="Str_TextLabel">Text</system:String>
<!-- Project update overlay statuses -->
<system:String x:Key="Str_Update_ReadDir">Analyzing directory
structure...</system:String>
<system:String x:Key="Str_Update_GoogleApi">Synchronizing with Google
Drive...</system:String>
<system:String x:Key="Str_Update_ConvertDocx">Converting documents...</system:String>

<!-- Error messages -->
<system:String x:Key="Str_Err_Update_ReadDir">File access error. Make sure the storage
media or local drive is connected.</system:String>
<system:String x:Key="Str_Err_Update_Network">Network failure. Please check your
internet connection.</system:String>
<system:String x:Key="Str_Err_Update_Convert">Failed to convert file. The archive might
be corrupted.</system:String>
<system:String x:Key="Str_Update_BtnRetry">Retry</system:String>
<system:String x:Key="Str_Update_BtnCancel">Cancel</system:String>
<system:String x:Key="Str_Update_Title">Project Update</system:String>
<!-- NEW STRINGS FOR ELIMINATING HARDCODE -->
<system:String x:Key="Str_Update_LoadingGoogleDoc">Loading Google Document
structure...</system:String>
<system:String x:Key="Str_Update_BuildingTree">Building syntax tree...</system:String>
<system:String x:Key="Str_Err_GoogleAuthFailed">Google authentication failed: Check the
service account key file.</system:String>
<system:String x:Key="Str_Err_GoogleParseError">Failed to parse Google Document
structure. Invalid JSON.</system:String>
<system:String x:Key="Str_Status_ProjectSaved">Project settings saved
successfully.</system:String>

```

```
<!-- Project Context Menu Items -->
<system:String x:Key="Str_Mnu_UpdateProject">Update Structure</system:String>
<system:String x:Key="Str_Mnu_DeleteProject">Delete Project</system:String>
<system:String x:Key="Str_Mnu_RecreateGoogleDoc">Clear and Recreate
Project</system:String>

<!-- Confirmation messages -->
<system:String x:Key="Str_Msg_ConfirmDeleteProj">Are you sure you want to completely
delete project "{0}" and all its modules?</system:String>
<system:String x:Key="Str_Msg_ConfirmRecreateGoogle">Warning! This action will
completely delete the local cache, clear the tree, and force re-download the document.
Continue?</system:String>
<system:String x:Key="Str_Title_Confirmation">Confirmation Required</system:String>

</ResourceDictionary>
```

=== ПОЛНЫЙ ТЕКСТ ДОКУМЕНТА: Strings.ru.xaml ===

```
<ResourceDictionary xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:system="clr-namespace:System;assembly=System.Runtime">
<!-- Заголовки блоков -->
<system:String x:Key="Str_ProjManagement">1. Управление Проектом</system:String>
<system:String x:Key="Str_ModuleManagement">2. Нарезаемые Модули / Контексты
ИИ</system:String>
<system:String x:Key="Str_FilesHeader">Файлы текущего модуля</system:String>
<system:String x:Key="Str_PromptHeader">Общие правила проекта (Prompt)</system:String>
<system:String x:Key="Str_ModulePromptHeader">Правила текущего модуля
(Prompt)</system:String>

<!-- Элементы блока проекта -->
<system:String x:Key="Str_SelectProj">Выбор проекта:</system:String>
<system:String x:Key="Str_ProjName">Название проекта:</system:String>
<system:String x:Key="Str_CreateProj">Создать проект</system:String>
<system:String x:Key="Str_Autosave">Автосохранение при выходе</system:String>
<system:String x:Key="Str_DarkTheme">Тёмная тема</system:String>
<system:String x:Key="Str_RootPath">Корневая папка (код):</system:String>
<system:String x:Key="Str_OutputPath">Выходная папка (ТЗ):</system:String>
<system:String x:Key="Str_Browse">Обзор...</system:String>
<system:String x:Key="Str_IncludeStructure">Включать структуру каталогов</system:String>

<!-- Элементы блока модулей -->
<system:String x:Key="Str_SelectModule">Выбор модуля:</system:String>
<system:String x:Key="Str_ModuleName">Название модуля:</system:String>
<system:String x:Key="Str_AddModule">Добавить модуль</system:String>
<system:String x:Key="Str_FileFormat">Формат файла:</system:String>

<!-- Главные кнопки -->
<system:String x:Key="Str_SaveProject">СОХРАНИТЬ ИЗМЕНЕНИЯ ПРОЕКТА</system:String>
<system:String x:Key="Str_Generate">СГЕНЕРИРОВАТЬ ФАЙЛ КОНТЕКСТА МОДУЛЯ</system:String>

<!-- Оверлей прогресса -->
<system:String x:Key="Str_Cancel">ОТМЕНИТЬ ОПЕРАЦИЮ</system:String>

<!-- Фолбэк для заголовка экспандера -->
<system:String x:Key="Str_ProjNotSelected">Проект не выбран</system:String>
<system:String x:Key="Str_AppTitle">Нарезка контекста кода для ИИ</system:String>
<system:String x:Key="Str_DeleteModule">Удалить модуль</system:String>
<system:String x:Key="Str_GlobalFilters">Глобальные фильтры</system:String>
<system:String x:Key="Str_MnuIgnoreItem">Добавить в фильтр исключений</system:String>
<!-- Расчёт токенов-->
<system:String x:Key="Str_StatsChars">Всего символов:</system:String>
<system:String x:Key="Str_StatsTokens">Примерно токенов:</system:String>
<system:String x:Key="Str_Calculating">Пересчет объема...</system:String>
<!-- Префиксы элементов синтаксического дерева -->
<system:String x:Key="Str_Type_Namespace">[ПРОСТРАНСТВО ИМЕН]</system:String>
<system:String x:Key="Str_Type_Class">[КЛАСС]</system:String>
<system:String x:Key="Str_Type_Struct">[СТРУКТУРА]</system:String>
<system:String x:Key="Str_Type_Interface">[ИНТЕРФЕЙС]</system:String>
<system:String x:Key="Str_Type_Enum">[ПЕРЕЧИСЛЕНИЕ]</system:String>
<system:String x:Key="Str_Type_Function">[МЕТОД]</system:String>
<system:String x:Key="Str_Type_Section">[СЕКЦИЯ]</system:String>
<system:String x:Key="Str_LoadingStub">Загрузка структуры...</system:String>
<!-- Специфичные SQL префиксы -->
<system:String x:Key="Str_Type_SqlTable">[ТАБЛИЦА]</system:String>
<system:String x:Key="Str_Type_SqlView">[ПРЕДСТАВЛЕНИЕ]</system:String>
<system:String x:Key="Str_Type_SqlFunction">[SQL ФУНКЦИЯ]</system:String>
<!-- Блок переименования модуля (Оверлей) -->
<system:String x:Key="Str_ModuleNameLabel">Название модуля:</system:String>
<system:String x:Key="Str_RenameTitle">Переименование модуля</system:String>
<system:String x:Key="Str_BtnCancel">Отмена</system:String>
<system:String x:Key="Str_BtnAccept">Принять</system:String>
```

```

<system:String x:Key="Str_TooltipRename">Переименовать модуль</system:String>
<!-- Подсказки для кнопок управления проектами и модулями -->
<system:String x:Key="Str_TooltipProjectEdit">Настройки проекта</system:String>
<system:String x:Key="Str_TooltipProjectSync">Синхронизировать / Перечитать
источник</system:String>
<system:String x:Key="Str_TooltipModuleAdd">Добавить новый модуль</system:String>
<system:String x:Key="Str_TooltipModuleEdit">Переименовать текущий
модуль</system:String>
<system:String x:Key="Str_TooltipModuleDelete">Удалить текущий модуль</system:String>
<system:String x:Key="Str_TextPart">ЧАСТЬ</system:String>
<system:String x:Key="Str_TextPage">Страница</system:String>
<!-- Сокращенные префиксы художественного текста -->
<system:String x:Key="Str_Type_Tab">[БКЛ]</system:String>
<system:String x:Key="Str_Type_Heading">[ГЛ]</system:String>

<system:String x:Key="Str_SourceDataLabel">Источник данных:</system:String>
<!-- Ошибки сетевого взаимодействия с облаком -->
<system:String x:Key="Str_Err_GoogleDocUnavailable">Google Документ недоступен!
Проверьте доступ по ссылке, валидность API-ключа и сетевое подключение.</system:String>
<system:String x:Key="Str_Err_NetworkTitle">Ошибка сети</system:String>
<!-- Локализация менеджера сервисных аккаунтов Google -->
<system:String x:Key="Str_Err_InvalidServiceAccount">Выбранный файл не является валидным
ключом Google Service Account!</system:String>
<system:String x:Key="Str_Err_ValidationTitle">Ошибка валидации</system:String>
<system:String x:Key="Str_Err_ImportFailed">Не удалось прочитать или импортировать
ключ:</system:String>
<system:String x:Key="Str_Err_ImportTitle">Ошибка импорта</system:String>
<system:String x:Key="Str_Msg_EmailCopied">Email сервисного аккаунта успешно скопирован
в буфер обмена:&#x0a;&#x0a;{0}&#x0a;&#x0a;Вставьте его в настройки доступа вашего Google
Документа!</system:String>
<system:String x:Key="Str_Msg_EmailCopiedTitle">Успех</system:String>
<system:String x:Key="Str_Err_ExtractEmailFailed">Не удалось извлечь
Email:</system:String>
<system:String x:Key="Str_Err_GeneralErrorTitle">Ошибка</system:String>
<!-- Параметры системного диалога выбора ключа -->
<system:String x:Key="Str_Dialog_JsonFilter">Ключ Google API (*.json)|*.json|Все файлы
(*.*)|*.*</system:String>
<system:String x:Key="Str_Dialog_JsonTitle">Выберите скачанный JSON-ключ сервисного
аккаунта</system:String>
<system:String x:Key="Str_TextLabel">Текст</system:String>
<!-- Статусы оверлея обновления проекта -->
<system:String x:Key="Str_Update_ReadDir">Анализ структуры каталогов...</system:String>
<system:String x:Key="Str_Update_GoogleApi">Синхронизация с Google
Drive...</system:String>
<system:String x:Key="Str_Update_ConvertDocx">Конвертация документов...</system:String>

<!-- Сообщения об ошибках -->
<system:String x:Key="Str_Err_Update_ReadDir">Ошибка доступа к файлам. Убедитесь, что
носитель или локальный диск подключены.</system:String>
<system:String x:Key="Str_Err_Update_Network">Сбой сети. Проверьте интернет-
соединение.</system:String>
<system:String x:Key="Str_Err_Update_Convert">Не удалось сконвертировать файл. Возможно,
архив поврежден.</system:String>
<system:String x:Key="Str_Update_BtnRetry">Повторить</system:String>
<system:String x:Key="Str_Update_BtnCancel">Отмена</system:String>
<system:String x:Key="Str_Update_Title">Обновление проекта</system:String>

<!-- НОВЫЕ СТРОКИ ДЛЯ ИСКЛЮЧЕНИЯ ХАРДКОДА -->
<system:String x:Key="Str_Update_LoadingGoogleDoc">Загрузка структуры Google
Документа...</system:String>
<system:String x:Key="Str_Update_BuildingTree">Построение синтаксического
дерева...</system:String>
<system:String x:Key="Str_Err_GoogleAuthFailed">Ошибка авторизации Google: Проверьте
файл ключа сервисного аккаунта.</system:String>
<system:String x:Key="Str_Err_GoogleParseError">Ошибка разбора структуры Google
Документа. Некорректный JSON.</system:String>

```

```
<system:String x:Key="Str_Status_ProjectSaved">Настройки проекта успешно
сохранены.</system:String>
<!-- Элементы контекстного меню проектов -->
<system:String x:Key="Str_Mnu_UpdateProject">Обновить структуру</system:String>
<system:String x:Key="Str_Mnu_DeleteProject">Удалить проект</system:String>
<system:String x:Key="Str_Mnu_RecreateGoogleDoc">Очистить и пересоздать
проект</system:String>

<!-- Сообщения подтверждения -->
<system:String x:Key="Str_Msg_ConfirmDeleteProj">Вы уверены, что хотите полностью
удалить проект "{0}" и все его модули?</system:String>
<system:String x:Key="Str_Msg_ConfirmRecreateGoogle">Внимание! Это действие полностью
удалит локальный кеш, очистит дерево и принудительно скачает документ заново.
Продолжить?</system:String>
<system:String x:Key="Str_Title_Confirmation">Подтверждение действия</system:String>

</ResourceDictionary>
```

=== ПОЛНЫЙ ТЕКСТ ДОКУМЕНТА: Settings.Designer.cs ===

```
//-----  
// <auto-generated>  
// Этот код создан программой.  
// Исполняемая версия:4.0.30319.42000  
//  
// Изменения в этом файле могут привести к неправильной работе и будут потеряны в случае  
// повторной генерации кода.  
// </auto-generated>  
//-----  
  
namespace ContextSlicer.Properties {  
  
    [global::System.Runtime.CompilerServices.CompilerGeneratedAttribute()]  
    [global::System.CodeDom.Compiler.GeneratedCodeAttribute("Microsoft.VisualStudio.Editors.Settings",  
"17.14.0.0")]  
    internal sealed partial class Settings :  
    global::System.Configuration.ApplicationSettingsBase {  
  
        private static Settings defaultInstance =  
        ((Settings) (global::System.Configuration.ApplicationSettingsBase.Synchronized(new  
Settings())));  
  
        public static Settings Default {  
        get {  
        return defaultInstance;  
        }  
        }  
  
        [global::System.Configuration.UserScopedSettingAttribute()]  
        [global::System.Diagnostics.DebuggerNonUserCodeAttribute()]  
        [global::System.Configuration.DefaultSettingValueAttribute("True")]  
        public bool AutoSaveOnExit {  
        get {  
        return ((bool) (this["AutoSaveOnExit"]));  
        }  
        set {  
        this["AutoSaveOnExit"] = value;  
        }  
        }  
  
        [global::System.Configuration.UserScopedSettingAttribute()]  
        [global::System.Diagnostics.DebuggerNonUserCodeAttribute()]  
        [global::System.Configuration.DefaultSettingValueAttribute("True")]  
        public bool IsDarkTheme {  
        get {  
        return ((bool) (this["IsDarkTheme"]));  
        }  
        set {  
        this["IsDarkTheme"] = value;  
        }  
        }  
  
        [global::System.Configuration.UserScopedSettingAttribute()]  
        [global::System.Diagnostics.DebuggerNonUserCodeAttribute()]  
        [global::System.Configuration.DefaultSettingValueAttribute("True")]  
        public bool IsProjectBlockExpanded {  
        get {  
        return ((bool) (this["IsProjectBlockExpanded"]));  
        }  
        set {  
        this["IsProjectBlockExpanded"] = value;  
        }  
        }  
    }  
}
```

```
}
```

```
[global::System.Configuration.UserScopedSettingAttribute()]  
[global::System.Diagnostics.DebuggerNonUserCodeAttribute()]  
[global::System.Configuration.DefaultSettingValueAttribute("True")]  
public bool IncludeDirectoryStructure {  
    get {  
        return ((bool) (this["IncludeDirectoryStructure"]));  
    }  
    set {  
        this["IncludeDirectoryStructure"] = value;  
    }  
}
```

```
[global::System.Configuration.UserScopedSettingAttribute()]  
[global::System.Diagnostics.DebuggerNonUserCodeAttribute()]  
[global::System.Configuration.DefaultSettingValueAttribute("")]  
public string EncryptedGoogleApiKey {  
    get {  
        return ((string) (this["EncryptedGoogleApiKey"]));  
    }  
    set {  
        this["EncryptedGoogleApiKey"] = value;  
    }  
}
```

```
[global::System.Configuration.UserScopedSettingAttribute()]  
[global::System.Diagnostics.DebuggerNonUserCodeAttribute()]  
[global::System.Configuration.DefaultSettingValueAttribute("")]  
public string DefaultServiceAccountEmail {  
    get {  
        return ((string) (this["DefaultServiceAccountEmail"]));  
    }  
    set {  
        this["DefaultServiceAccountEmail"] = value;  
    }  
}  
}  
}
```

=== ПОЛНЫЙ ТЕКСТ ДОКУМЕНТА: Settings.settings ===

```
<?xml version='1.0' encoding='utf-8'?>
<SettingsFile xmlns="http://schemas.microsoft.com/VisualStudio/2004/01/settings"
CurrentProfile="(Default)" GeneratedClassNamespace="ContextSlicer.Properties"
GeneratedClassName="Settings">
<Profiles />
<Settings>
<Setting Name="AutoSaveOnExit" Type="System.Boolean" Scope="User">
<Value Profile="(Default)">True</Value>
</Setting>
<Setting Name="IsDarkTheme" Type="System.Boolean" Scope="User">
<Value Profile="(Default)">True</Value>
</Setting>
<Setting Name="IsProjectBlockExpanded" Type="System.Boolean" Scope="User">
<Value Profile="(Default)">True</Value>
</Setting>
<Setting Name="IncludeDirectoryStructure" Type="System.Boolean" Scope="User">
<Value Profile="(Default)">True</Value>
</Setting>
<Setting Name="EncryptedGoogleApiKey" Type="System.String" Scope="User">
<Value Profile="(Default)" />
</Setting>
<Setting Name="DefaultServiceAccountEmail" Type="System.String" Scope="User">
<Value Profile="(Default)" />
</Setting>
</Settings>
</SettingsFile>
```

=== ПОЛНЫЙ ТЕКСТ ДОКУМЕНТА: DarkTheme.xaml ===

```
<ResourceDictionary xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
<!-- Цветовая палитра тёмной темы -->
<SolidColorBrush x:Key="WindowBackgroundBrush" Color="#1E1E1E"/>
<SolidColorBrush x:Key="ControlBackgroundBrush" Color="#252526"/>
<SolidColorBrush x:Key="TextBoxBackgroundBrush" Color="#2D2D30"/>
<SolidColorBrush x:Key="TextBrush" Color="#F1F1F1"/>
<SolidColorBrush x:Key="SplitterBrush" Color="#3E3E42"/>
<SolidColorBrush x:Key="BorderBrush" Color="#434346"/>
<!-- Синхронизация псевдонимов кистей для нового интерфейса (Темная тема) -->
<SolidColorBrush x:Key="AccentBrush" Color="#007ACC"/>
<SolidColorBrush x:Key="PanelBackgroundBrush" Color="#252526"/>
<SolidColorBrush x:Key="InputBackgroundBrush" Color="#2D2D30"/>

<!-- Новые адаптивные цвета для контуров кнопок управления (OK) -->
<SolidColorBrush x:Key="ButtonEditBrush" Color="#007ACC"/>
<SolidColorBrush x:Key="ButtonSyncBrush" Color="#4CAF50"/>
<SolidColorBrush x:Key="ButtonDeleteBrush" Color="#E53935"/>

<!-- Принудительные стили для WCEX элементов на форме в тёмной теме -->
<Style TargetType="TextBlock">
<Setter Property="Foreground" Value="#DCDCDC"/>
</Style>

<Style TargetType="GroupBox">
<Setter Property="Foreground" Value="#F1F1F1"/>
<Setter Property="BorderBrush" Value="#3E3E42"/>
</Style>

<Style TargetType="CheckBox">
<Setter Property="Foreground" Value="#DCDCDC"/>
</Style>

<Style TargetType="TextBox">
<Setter Property="Background" Value="{StaticResource TextBoxBackgroundBrush}"/>
<Setter Property="Foreground" Value="{StaticResource TextBrush}"/>
<Setter Property="BorderBrush" Value="{StaticResource BorderBrush}"/>
<Setter Property="BorderThickness" Value="1"/>
<Setter Property="Padding" Value="3,2"/>
<Setter Property="CaretBrush" Value="#F1F1F1"/>
</Style>

<Style TargetType="TreeView">
<Setter Property="Background" Value="{StaticResource ControlBackgroundBrush}"/>
<Setter Property="BorderBrush" Value="{StaticResource BorderBrush}"/>
</Style>

<!-- Стилизация ComboBox, чтобы текст внутри него и в выпадающем списке был белым -->
<Style TargetType="ComboBox">
<Setter Property="Background" Value="{StaticResource TextBoxBackgroundBrush}"/>
<Setter Property="Foreground" Value="{StaticResource TextBrush}"/>
<Setter Property="BorderBrush" Value="{StaticResource BorderBrush}"/>
</Style>

<Style TargetType="ComboBoxItem">
<Setter Property="Background" Value="{StaticResource TextBoxBackgroundBrush}"/>
<Setter Property="Foreground" Value="{StaticResource TextBrush}"/>
</Style>

<!-- Принудительно красим обычные кнопки "Обзор" и "Создать", чтобы они не оставались серыми -->
<Style TargetType="Button">
<Setter Property="Background" Value="#2D2D30"/>
```

```
<Setter Property="Foreground" Value="#F1F1F1"/>
<Setter Property="BorderBrush" Value="#434346"/>
<Setter Property="Padding" Value="8,2"/>
</Style>
<!-- Стили элементов управления для Темной Темы -->
<Style x:Key="ThemeComboBoxItemStyle" TargetType="ComboBoxItem">
<Setter Property="Background" Value="#252526"/>
<Setter Property="Foreground" Value="#F1F1F1"/>
</Style>

<Style x:Key="ThemeProjectEditButtonStyle" TargetType="Button" BasedOn="{StaticResource
{x:Type Button}}">
<Setter Property="Foreground" Value="#007ACC"/>
<Setter Property="Background" Value="Transparent"/>
</Style>
<Style x:Key="ThemeProjectSyncButtonStyle" TargetType="Button" BasedOn="{StaticResource
{x:Type Button}}">
<Setter Property="Foreground" Value="#4CAF50"/>
<Setter Property="Background" Value="Transparent"/>
</Style>
<Style x:Key="ThemeModuleDeleteButtonStyle" TargetType="Button" BasedOn="{StaticResource
{x:Type Button}}">
<Setter Property="Foreground" Value="#E53935"/>
<Setter Property="Background" Value="Transparent"/>
</Style>

</ResourceDictionary>
```

=== ПОЛНЫЙ ТЕКСТ ДОКУМЕНТА: LightTheme.xaml ===

```
<ResourceDictionary xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
<!-- Базовые кисти для светлой темы -->
<SolidColorBrush x:Key="WindowBackgroundBrush" Color="#F5F5F5"/>
<SolidColorBrush x:Key="ControlBackgroundBrush" Color="#FFFFFF"/>
<SolidColorBrush x:Key="TextBoxBackgroundBrush" Color="#FFFFFF"/>
<SolidColorBrush x:Key="TextBrush" Color="#000000"/>
<SolidColorBrush x:Key="SplitterBrush" Color="#F0F0F0"/>
<SolidColorBrush x:Key="BorderBrush" Color="#CCCCCC"/>
<!-- Синхронизация псевдонимов кистей для нового интерфейса (Светлая тема) -->
<SolidColorBrush x:Key="AccentBrush" Color="#005A9E"/>
<SolidColorBrush x:Key="PanelBackgroundBrush" Color="#FFFFFF"/>
<SolidColorBrush x:Key="InputBackgroundBrush" Color="#FFFFFF"/>

<!-- Адаптивные контрастные цвета для контуров кнопок (Светлая тема) -->
<SolidColorBrush x:Key="ButtonEditBrush" Color="#005A9E"/>
<SolidColorBrush x:Key="ButtonSyncBrush" Color="#2E7D32"/>
<SolidColorBrush x:Key="ButtonDeleteBrush" Color="#E53935"/>

<!-- Неявные стили для сброса на стандартные цвета -->
<Style TargetType="TextBlock">
<Setter Property="Foreground" Value="#333333"/>
</Style>
<Style TargetType="GroupBox">
<Setter Property="Foreground" Value="#000000"/>
<Setter Property="BorderBrush" Value="#CCCCCC"/>
</Style>
<Style TargetType="CheckBox">
<Setter Property="Foreground" Value="#333333"/>
</Style>
<Style TargetType="TextBox">
<Setter Property="Background" Value="{StaticResource TextBoxBackgroundBrush}"/>
<Setter Property="Foreground" Value="{StaticResource TextBrush}"/>
</Style>
<Style TargetType="TreeView">
<Setter Property="Background" Value="{StaticResource ControlBackgroundBrush}"/>
</Style>
<!-- Стили элементов управления для Светлой Темы -->
<Style x:Key="ThemeComboBoxItemStyle" TargetType="ComboBoxItem">
<Setter Property="Background" Value="White"/>
<Setter Property="Foreground" Value="#1E1E1E"/>
</Style>

<Style x:Key="ThemeProjectEditButtonStyle" TargetType="Button" BasedOn="{StaticResource
{x:Type Button}}">
<Setter Property="Foreground" Value="#005A9E"/>
<Setter Property="Background" Value="Transparent"/>
</Style>
<Style x:Key="ThemeProjectSyncButtonStyle" TargetType="Button" BasedOn="{StaticResource
{x:Type Button}}">
<Setter Property="Foreground" Value="#2E7D32"/>
<Setter Property="Background" Value="Transparent"/>
</Style>
<Style x:Key="ThemeModuleDeleteButtonStyle" TargetType="Button" BasedOn="{StaticResource
{x:Type Button}}">
<Setter Property="Foreground" Value="#C62828"/>
<Setter Property="Background" Value="Transparent"/>
</Style>

</ResourceDictionary>
```

=== ПОЛНЫЙ ТЕКСТ ДОКУМЕНТА: App.config ===

```
<?xml version="1.0" encoding="utf-8" ?>
<configuration>
<configSections>
<sectionGroup name="userSettings" type="System.Configuration.UserSettingsGroup, System,
Version=4.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089" >
<section name="ContextSlicer.Properties.Settings"
type="System.Configuration.ClientSettingsSection, System, Version=4.0.0.0,
Culture=neutral, PublicKeyToken=b77a5c561934e089"
allowExeDefinition="MachineToLocalUser" requirePermission="false" />
</sectionGroup>
</configSections>
<userSettings>
<ContextSlicer.Properties.Settings>
<setting name="AutoSaveOnExit" serializeAs="String">
<value>True</value>
</setting>
<setting name="IsDarkTheme" serializeAs="String">
<value>True</value>
</setting>
<setting name="IsProjectBlockExpanded" serializeAs="String">
<value>True</value>
</setting>
<setting name="IncludeDirectoryStructure" serializeAs="String">
<value>True</value>
</setting>
<setting name="EncryptedGoogleApiKey" serializeAs="String">
<value />
</setting>
<setting name="DefaultServiceAccountEmail" serializeAs="String">
<value />
</setting>
</ContextSlicer.Properties.Settings>
</userSettings>
</configuration>
```

=== ПОЛНЫЙ ТЕКСТ ДОКУМЕНТА: App.xaml ===

```
<Application x:Class="ContextSlicer.App"
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:local="clr-namespace:ContextSlicer.Filesystem"
StartupUri="MainWindow.xaml">
<Application.Resources>
<ResourceDictionary>
<ResourceDictionary.MergedDictionaries>
<!-- По умолчанию при старте всегда загружаем Светлую тему -->
<ResourceDictionary x:Name="ThemeDictionary">
<SolidColorBrush x:Key="WindowBackgroundBrush" Color="#F5F5F5"/>
<SolidColorBrush x:Key="ControlBackgroundBrush" Color="#FFFFFF"/>
<SolidColorBrush x:Key="TextBoxBackgroundBrush" Color="#FFFFFF"/>
<SolidColorBrush x:Key="TextBrush" Color="#000000"/>
<SolidColorBrush x:Key="SplitterBrush" Color="#E0E0E0"/>
<SolidColorBrush x:Key="BorderBrush" Color="#CCCCCC"/>
</ResourceDictionary>
</ResourceDictionary.MergedDictionaries>
<local:NullToBoolConverter x:Key="NullToBoolConverter"/>
</ResourceDictionary>

</Application.Resources>
</Application>
```

=== ПОЛНЫЙ ТЕКСТ ДОКУМЕНТА: App.xaml.cs ===

```
using System;
using System.Globalization;
using System.Windows;

namespace ContextSlicer
{
    public partial class App : Application
    {
        protected override void OnStartup(StartupEventArgs e)
        {
            base.OnStartup(e);

            // Получаем текущий язык операционной системы Windows
            string currentCulture = CultureInfo.CurrentUICulture.TwoLetterISOLanguageName;

            // Задаем базовое имя файла локализации
            string themeFile = currentCulture.Equals("ru", StringComparison.OrdinalIgnoreCase)
                ? "Strings.ru.xaml"
                : "Strings.en.xaml";

            try
            {
                var dict = new ResourceDictionary
                {
                    Source = new Uri($"pack://application:,,,/Localization/{themeFile}", UriKind.Absolute)
                };

                // Внедряем выбранный языковой словарь в глобальные ресурсы приложения
                Resources.MergedDictionaries.Add(dict);
            }
            catch (Exception ex)
            {
                MessageBox.Show($"Failed to load localization file: {ex.Message}", "Localization Error",
                    MessageBoxButton.OK, MessageBoxImage.Warning);
            }
        }
    }
}
```

=== ПОЛНЫЙ ТЕКСТ ДОКУМЕНТА: AssemblyInfo.cs ===

```
using System.Windows;
```

```
[assembly: ThemeInfo(  
ResourceDictionaryLocation.None, //where theme specific resource dictionaries are  
located  
//(used if a resource is not found in the page,  
// or application resource dictionaries)  
ResourceDictionaryLocation.SourceAssembly //where the generic resource dictionary is  
located  
//(used if a resource is not found in the page,  
// app, or any theme specific resource dictionaries)  
)]
```

=== ПОЛНЫЙ ТЕКСТ ДОКУМЕНТА: ContextBuilderService.cs ===

```
using System;
using System.IO;
using System.Text;
using System.Collections.Generic;
using System.Threading;
using System.Threading.Tasks;
using ContextSlicer.Filesystem;

namespace ContextSlicer;

// Структура для отправки отчетов о прогрессе в UI
public struct ProgressReport
{
    public int CurrentIndex { get; set; }
    public int TotalCount { get; set; }
    public string CurrentFileName { get; set; }
}

public static class ContextBuilderService
{
    // Динамическая проверка папок по глобальному списку исключений
    private static bool IsFolderExcluded(string folderName)
    {
        if (GlobalFilterService.Current?.ExcludedFolders == null) return false;
        foreach (var f in GlobalFilterService.Current.ExcludedFolders)
        {
            if (folderName.Equals(f, StringComparison.OrdinalIgnoreCase)) return true;
        }
        return false;
    }

    // Динамическая проверка расширений файлов по глобальному списку исключений
    private static bool IsExtensionExcluded(string filePath)
    {
        if (GlobalFilterService.Current?.ExcludedExtensions == null) return false;
        string ext = Path.GetExtension(filePath);
        foreach (var e in GlobalFilterService.Current.ExcludedExtensions)
        {
            if (ext.Equals(e, StringComparison.OrdinalIgnoreCase)) return true;
        }
        return false;
    }

    // Метод рекурсивного сканирования дерева
    public static FileSystemNode BuildTree(string rootPath, List<string> savedCheckedFiles)
    {
        var rootInfo = new DirectoryInfo(rootPath);
        var rootNode = new FileSystemNode { Name = rootInfo.Name, FullPath = rootInfo.FullName,
        RelativePath = "" };
        FillNodesRecursive(rootInfo, rootNode, rootPath, savedCheckedFiles);
        return rootNode;
    }

    private static void FillNodesRecursive(DirectoryInfo dir, FileSystemNode parentNode,
    string rootPath, List<string> savedCheckedFiles)
    {
        try
        {
            foreach (var subDir in dir.GetDirectories())
            {
                if (IsFolderExcluded(subDir.Name)) continue;

                var childNode = new FileSystemNode { Name = subDir.Name, FullPath = subDir.FullName,
```

```

RelativePath = Path.GetRelativePath(rootPath, subDir.FullName), IsFile = false, Parent =
parentNode };
parentNode.Children.Add(childNode);
FillNodesRecursive(subDir, childNode, rootPath, savedCheckedFiles);
}
foreach (var file in dir.GetFiles())
{
if (IsExtensionExcluded(file.FullName)) continue;
var relPath = Path.GetRelativePath(rootPath, file.FullName);

var childNode = new FileSystemNode
{
Name = file.Name,
FullPath = file.FullName,
RelativePath = relPath,
IsFile = true,
Parent = parentNode
};

if (savedCheckedFiles.Contains(relPath)) childNode.IsChecked = true;

// НОВОЕ: Если расширение файла поддерживается синтаксическими парсерами,
// добавляем фиктивную ноду-заглушку, чтобы WPF отобразил стрелочку раскрытия узла [ ]
string ext = Path.GetExtension(file.FullName);
if (SyntaxParserFactory.IsSupported(ext))
{
childNode.Children.Add(new FileSystemNode { Name = "LoadingStub...", Parent = childNode
});
}

parentNode.Children.Add(childNode);
}

}
catch (UnauthorizedAccessException) { }
}

public static void GetCheckedFiles(FileSystemNode node, List<FileSystemNode> result)
{
// ЖЕСТКАЯ ЗАЩИТА: Если корень дерева или текущий узел аннулирован, прерываем рекурсию
if (node == null || result == null) return;

if (node.IsFile && node.IsChecked == true)
{
if (!IsExtensionExcluded(node.FullPath))
{
if (!result.Contains(node)) result.Add(node);
}
}

// Безопасный обход дочерних элементов через локальную копию
var childrenCopy = new List<FileSystemNode>(node.Children);
foreach (var child in childrenCopy)
{
if (child != null)
{
GetCheckedFiles(child, result);
}
}
}

public static void GetCheckedFilesExtended(FileSystemNode node, List<FileSystemNode>
result)
{
// ЖЕСТКАЯ ЗАЩИТА: Ликвидация System.NullReferenceException при закрытии/смене проекта

```

```

if (node == null || result == null) return;

if (node.IsFile && (node.IsChecked == true || node.IsChecked == null))
{
    if (!IsExtensionExcluded(node.FullPath))
    {
        if (!result.Contains(node)) result.Add(node);
    }
}

var childrenCopy = new List<FileSystemNode>(node.Children);
foreach (var child in childrenCopy)
{
    if (child != null)
    {
        GetCheckedFilesExtended(child, result);
    }
}
}

// ===== -->
// ЭТАЛОННЫЙ СВОРЩИК ЛИТЕРАТУРНОГО КОНТЕКСТА GOOGLE DOCS (XML-ТЕГИ) -->
// ===== -->
public static async Task<StringBuilder> BuildTextContentAsync(
    string projectRules,
    string moduleRules,
    bool includeDirectoryStructure,
    List<FileSystemNode> checkedFiles,
    List<SyntaxEntry> savedEntries,
    IProgress<ProgressReport> progress,
    CancellationToken token)
{
    var sb = new StringBuilder();

    // 1. Выгрузка системных правил и промптов
    if (!string.IsNullOrEmpty(projectRules) || !string.IsNullOrEmpty(moduleRules))
    {
        sb.AppendLine("=== ПРАВИЛА ОБРАЩЕНИЯ С ТЕКСТОМ И КОДОМ ===");
        if (!string.IsNullOrEmpty(projectRules))
        {
            sb.AppendLine("<project_rules>");
            sb.AppendLine(projectRules.Trim());
            sb.AppendLine("</project_rules>\n");
        }
        if (!string.IsNullOrEmpty(moduleRules))
        {
            sb.AppendLine("<module_rules>");
            sb.AppendLine(moduleRules.Trim());
            sb.AppendLine("</module_rules>\n");
        }
    }

    // 2. ИСПРАВЛЕНО: Безопасный вывод структуры оглавления книги на основе сохраненных
    элементов синтаксиса
    if (includeDirectoryStructure && savedEntries != null && savedEntries.Count > 0)
    {
        sb.AppendLine("=== СТРУКТУРА ВЫБРАННОГО ЛИТЕРАТУРНОГО КОНТЕКСТА ===");
        var uniqueTabs = new HashSet<string>(StringComparer.OrdinalIgnoreCase);

        foreach (var entry in savedEntries)
        {
            // Строим красивое Markdown оглавление выбранных вкладок и глав
            int slashIdx = entry.EntryPath.IndexOf('/');
            if (slashIdx > 0)
            {

```

```

string tabName = entry.EntryPath.Substring(0, slashIdx);
if (uniqueTabs.Add(tabName)) sb.AppendLine($"□ [Вкладка] {tabName}");
sb.AppendLine($"└─ □ {entry.DisplayName}");
}
else
{
sb.AppendLine($"└─ □ {entry.DisplayName}");
}
}
sb.AppendLine();
}

sb.AppendLine("=== СОДЕРЖИМОЕ ВЫБРАННЫХ РАЗДЕЛОВ ===");

if (savedEntries == null || savedEntries.Count == 0) return sb;

// Группируем выбранные главы по файлам (на случай если проектов несколько)
var entriesByFile = savedEntries.GroupBy(e => e.FilePath,
StringComparer.Ordinal.IgnoreCase);

foreach (var fileGroup in entriesByFile)
{
token.ThrowIfCancellationRequested();

// Восстанавливаем физический путь к файлу кэша JSON в нашей папке googlecache
string googleCacheDir = Path.Combine(AppDomain.CurrentDomain.BaseDirectory,
"googlecache");
string docId = Path.GetFileNameWithoutExtension(fileGroup.Key);
// Если FilePath был пустой (наша оптимизация), берем ID из настроек или имени
if (string.IsNullOrEmpty(docId) && checkedFiles.Count > 0) docId =
Path.GetFileNameWithoutExtension(checkedFiles[0].FullPath);

string fullCachePath = Path.Combine(googleCacheDir, $"{docId}.json");
if (!File.Exists(fullCachePath)) continue;

// Загружаем объект JSON документа
string jsonContent = await File.ReadAllTextAsync(fullCachePath, Encoding.UTF8, token);
var docJson = Newtonsoft.Json.Linq.JObject.Parse(jsonContent);

var totalEntries = fileGroup.Count();
var entriesList = fileGroup.ToList();

// 3. БРИТВЕННО-ЧИСТЫЙ СБОР ХУДОЖЕСТВЕННОЙ ПРОЗЫ ПО ИМЕНАМ ГЛАВ
for (int i = 0; i < totalEntries; i++)
{
var entry = entriesList[i];
token.ThrowIfCancellationRequested();

progress?.Report(new ProgressReport
{
CurrentIndex = i + 1,
TotalCount = totalEntries,
CurrentFileName = entry.DisplayName
});

// Находим нужную вкладку и извлекаем текст конкретной главы
string pureChapterText = ExtractChapterTextFromGoogleJson(docJson, entry.EntryPath,
entry.DisplayName);

if (!string.IsNullOrEmpty(pureChapterText))
{
// Заворачиваем в идеальные семантические XML-теги, которые обожает ИИ!
sb.AppendLine($"<heading path=\"{entry.EntryPath}\">");

```

```

sb.AppendLine(pureChapterText.Trim());
sb.AppendLine($"</heading>\n");
}
}
}

return sb;
}

/// <summary>
/// Извлекает бритвенно-чистый текст параграфов, идущих строго внутри выбранной главы
/// </summary>
private static string ExtractChapterTextFromGoogleJson(Newtonsoft.Json.Linq.JObject
docJson, string entryPath, string chapterName)
{
var sb = new StringBuilder();

// Ищем имя вкладки (все что до первого слэша)
string targetTabTitle = entryPath.Contains('/') ? entryPath.Substring(0,
entryPath.IndexOf('/')) : string.Empty;

if (docJson["tabs"] is Newtonsoft.Json.Linq.JArray tabsArray)
{
foreach (var tab in tabsArray)
{
string tabTitle = tab["tabProperties"]?["title"]?.ToString() ?? "";
// Если документ с вкладками – ищем строго нашу целевую вкладку
if (!string.IsNullOrEmpty(targetTabTitle) && !tabTitle.Equals(targetTabTitle,
StringComparison.OrdinalIgnoreCase)) continue;

var content = tab["documentTab"]?["body"]?["content"] as Newtonsoft.Json.Linq.JArray;
if (content != null) CollectTextForHeading(content, chapterName, sb);
}
}
else if (docJson["body"]?["content"] is Newtonsoft.Json.Linq.JArray content)
{
// Плоский документ без вкладок
CollectTextForHeading(content, chapterName, sb);
}

return sb.ToString();
}

private static void CollectTextForHeading(Newtonsoft.Json.Linq.JArray contentArray,
string chapterName, StringBuilder sb)
{
bool insideTargetHeading = false;

foreach (var element in contentArray)
{
var paragraph = element["paragraph"];
if (paragraph == null) continue;

string namedStyle = paragraph["paragraphStyle"]?["namedStyleType"]?.ToString() ??
string.Empty;

// Собираем текст текущего параграфа из его элементов textRun
var textBuilder = new StringBuilder();
if (paragraph["elements"] is Newtonsoft.Json.Linq.JArray elements)
{
foreach (var el in elements)
{
textBuilder.Append(el["textRun"]?["content"]?.ToString() ?? string.Empty);
}
}
}

```

```

}
string currentText = textBuilder.ToString();

// Если встретили ЛЮБОЙ заголовок HEADING_
if (namedStyle.StartsWith("HEADING_"))
{
    if (insideTargetHeading)
    {
        // Если мы уже были внутри нашей главы и наткнулись на СЛЕДУЮЩИЙ заголовок — сбор
        // окончен!
        break;
    }

    // Проверяем, совпадает ли этот заголовок с искомой главой
    if (currentText.Trim().Equals(chapterName.Trim(), StringComparison.OrdinalIgnoreCase))
    {
        insideTargetHeading = true;
        // Записываем сам заголовок главы в начало
        sb.AppendLine(currentText.Trim());
        continue;
    }
}

// Если мы находимся внутри границ выбранной главы — бережно собираем её чистую прозу
// абзацев!
if (insideTargetHeading)
{
    sb.Append(currentText);
}
}

// Вспомогательный метод для рекурсивного сбора чекнутых синтаксических нод внутри файла
private static void FindCheckedSyntaxNodes(FileSystemNode node, List<FileSystemNode>
result)
{
    if (node.IsSyntaxNode && node.IsChecked == true)
    {
        result.Add(node);
    }
    foreach (var child in node.Children)
    {
        FindCheckedSyntaxNodes(child, result);
    }
}

// ===== -->
// ЧАСТЬ 2: ПОСТРАНИЧНЫЙ PDF-РЕНДЕРЕР С УЧЕТОМ КАСКАДА СТРУКТУРЫ -->
// ===== -->
public static async Task GeneratePdfContextFileAsync(
    string outputPath,
    string fileName,
    string projectRules,
    string moduleRules,
    bool includeDirectoryStructure,
    FileSystemNode rootNode,
    List<SyntaxEntry> savedEntries,
    IProgress<ProgressReport> progress,
    CancellationToken token)
{
    var checkedFiles = new List<FileSystemNode>();
    GetCheckedFilesExtended(rootNode, checkedFiles);

    // 1. Собираем базовый текстовый каркас (правила, промпты, оглавление Markdown)

```

```

var sb = await BuildTextContentAsync(projectRules, moduleRules,
includeDirectoryStructure, checkedFiles, savedEntries, progress, token);
string metaDataText = sb.ToString();
token.ThrowIfCancellationRequested();

// 2. Инициализируем объект документа MigraDoc
var document = new MigraDoc.DocumentObjectModel.Document();

// Настраиваем шрифты для правильного отображения кириллицы
var style = document.Styles["Normal"];
if (style?.Font != null)
{
style.Font.Name = "Courier New";
style.Font.Size = 9.5;
}

// Создаем первую секцию для вводных правил и оглавления книги
var currentSection = document.AddSection();
SetSectionMargins(currentSection);

// Выводим системную метаинформацию (правила, промпты, оглавление)
using (var reader = new StringReader(metaDataText))
{
string? line;
while ((line = await reader.ReadLineAsync()) != null)
{
token.ThrowIfCancellationRequested();

// Если дошли до маркера содержимого — останавливаем плоский вывод линий
if (line.StartsWith("=== СОДЕРЖИМОЕ ВЫБРАННЫХ РАЗДЕЛОВ ===")) break;

var p = currentSection.AddParagraph(line);
if (string.IsNullOrEmpty(line)) p.Format.SpaceBefore =
MigraDoc.DocumentObjectModel.Unit.FromPoint(5);
}
}

// 3. ПОГЛАВНЫЙ СБОР И НАРЕЗКА СТРУКТУРЫ НА СТРАНИЦЫ
int total = checkedFiles.Count;
for (int i = 0; i < total; i++)
{
var file = checkedFiles[i];
if (file == null || string.IsNullOrEmpty(file.FullPath)) continue;

try
{
token.ThrowIfCancellationRequested();
string fullContent = await File.ReadAllTextAsync(file.FullPath, Encoding.UTF8, token);

var fileEntries = savedEntries?.FindAll(e => e.FilePath.Equals(file.RelativePath,
StringComparison.OrdinalIgnoreCase)) ?? new List<SyntaxEntry>();

if (file.IsChecked == true)
{
// ФОЛБЭК: Если файл выбран целиком (парсер выдал один сплошной текст), выводим
монолитом с новой страницы
currentSection = document.AddSection();
SetSectionMargins(currentSection);

var headerP = currentSection.AddParagraph($"=== ПОЛНЫЙ ТЕКСТ ДОКУМЕНТА: {file.Name}
===");
headerP.Format.Font.Bold = true;
headerP.Format.SpaceAfter = MigraDoc.DocumentObjectModel.Unit.FromPoint(10);

```

```

AppendTextToSection(fullContent, currentSection);
}
else if (fileEntries.Count > 0)
{
// СЛЕДОВАНИЕ СТРУКТУРЕ: Обходим выбранные автором главы, вкладки или страницы A4
foreach (var entry in fileEntries)
{
token.ThrowIfCancellationRequested();
if (entry.Length == 0) continue; // Пропускаем пустые контейнеры вкладок

if (entry.StartIndex >= 0 && entry.StartIndex + entry.Length <= fullContent.Length)
{
string fragment = fullContent.Substring(entry.StartIndex, entry.Length).Trim();

// БИТЬЕ НА СТРАНИЦЫ: Каждая новая выбранная глава, вкладка или часть A4 НАЧИНАЕТСЯ С
НОВОЙ СТРАНИЦЫ!
currentSection = document.AddSection();
SetSectionMargins(currentSection);

// Формируем красивый, читаемый заголовок-маркер для автора и ИИ
string markerTitle = $"<{entry.Type.ToString().ToLower()} path=\"{entry.EntryPath}\">";
var markerP = currentSection.AddParagraph(markerTitle);
markerP.Format.Font.Bold = true;
markerP.Format.Font.Color = MigraDoc.DocumentObjectModel.Colors.DarkBlue; // Мягкий
темно-синий цвет разметки
markerP.Format.SpaceAfter = MigraDoc.DocumentObjectModel.Unit.FromPoint(12);

// Заливаем текст фрагмента прозы на страницу
AppendTextToSection(fragment, currentSection);

// Закрывающий тег для ИИ в самом низу страницы фрагмента
var closeP = currentSection.AddParagraph($"</{entry.Type.ToString().ToLower()}>");
closeP.Format.Font.Bold = true;
closeP.Format.Font.Color = MigraDoc.DocumentObjectModel.Colors.DarkBlue;
closeP.Format.SpaceBefore = MigraDoc.DocumentObjectModel.Unit.FromPoint(8);
}
}
}
}
catch { }
}

// 4. ФИЗИЧЕСКОЕ СОХРАНЕНИЕ СКОМПИЛИРОВАННОГО PDF НА ДИСК (ваша исходная логика защиты
процессов)
string fullOutputPath = Path.Combine(outputPath, fileName.EndsWith(".pdf") ? fileName :
fileName + ".pdf");
if (File.Exists(fullOutputPath))
{
try { File.Delete(fullOutputPath); }
catch (IOException)
{
System.Windows.MessageBox.Show("Не удалось перезаписать PDF. Закройте его в стороннем
просмотрщике и повторите попытку.", "Ошибка доступа",
System.Windows.MessageBoxButton.OK, System.Windows.MessageBoxImage.Warning);
return;
}
}

await Task.Run(() =>
{
var renderer = new MigraDoc.Rendering.PdfDocumentRenderer();
renderer.Document = document;
renderer.RenderDocument();
renderer.PdfDocument.Save(fullOutputPath);
}, token);

```

```

}

// Вспомогательный метод выставления компактных литературных отступов полей А4
private static void SetSectionMargins(MigraDoc.DocumentObjectModel.Section sec)
{
    sec.PageSetup.TopMargin = MigraDoc.DocumentObjectModel.Unit.FromCentimeter(1.5);
    sec.PageSetup.BottomMargin = MigraDoc.DocumentObjectModel.Unit.FromCentimeter(1.5);
    sec.PageSetup.LeftMargin = MigraDoc.DocumentObjectModel.Unit.FromCentimeter(1.8);
    sec.PageSetup.RightMargin = MigraDoc.DocumentObjectModel.Unit.FromCentimeter(1.5);
}

// Вспомогательный построчный перенос абзацев прозы внутрь PDF-секции
private static void AppendTextToSection(string text,
MigraDoc.DocumentObjectModel.Section sec)
{
    using (var reader = new StringReader(text))
    {
        string? line;
        while ((line = reader.ReadLine()) != null)
        {
            var paragraph = sec.AddParagraph(line);
            // Задаем красивый абзацный отступ для строк художественного текста рассказов
            if (string.IsNullOrEmpty(line))
            {
                paragraph.Format.SpaceBefore = MigraDoc.DocumentObjectModel.Unit.FromPoint(6);
            }
        }
    }
}

// Асинхронная генерация обычного TXT с поддержкой синтаксических записей
public static async Task GenerateContextFileAsync(
    string outputPath,
    string fileName,
    string projectRules,
    string moduleRules,
    bool includeDirectoryStructure,
    FileSystemNode rootNode,
    List<SyntaxEntry> savedEntries, // ИСПРАВЛЕНО: Добавлен параметр в сигнатуру метода
    IProgress<ProgressReport> progress,
    CancellationToken token)
{
    var checkedFiles = new List<FileSystemNode>();
    GetCheckedFilesExtended(rootNode, checkedFiles); // Используем расширенный сбор

    // ИСПРАВЛЕНО: Передаем savedEntries, полученный из параметров метода
    var sb = await BuildTextContentAsync(projectRules, moduleRules,
    includeDirectoryStructure,
    checkedFiles, savedEntries, progress, token);

    string fullOutputPath = Path.Combine(outputPath, fileName.EndsWith(".txt") ? fileName :
    fileName + ".txt");
    await File.WriteAllTextAsync(fullOutputPath, sb.ToString(), Encoding.UTF8, token);
}

}

```

=== ПОЛНЫЙ ТЕКСТ ДОКУМЕНТА: ContextSlicer.csproj ===

```
<Project Sdk="Microsoft.NET.Sdk">
```

```
<PropertyGroup>
  <OutputType>WinExe</OutputType>
  <TargetFramework>net9.0-windows</TargetFramework>
  <Nullable>enable</Nullable>
  <ImplicitUsings>enable</ImplicitUsings>
  <UseWPF>true</UseWPF>
  <ApplicationIcon>ContentSlicer.ico</ApplicationIcon>
</PropertyGroup>
```

```
<ItemGroup>
  <Content Include="ContentSlicer.ico">
    <CopyToOutputDirectory>Always</CopyToOutputDirectory>
  </Content>
</ItemGroup>
```

```
<ItemGroup>
  <PackageReference Include="CommunityToolkit.Mvvm" Version="8.4.2" />
  <PackageReference Include="Microsoft-WindowsAPICodePack-Shell" Version="1.1.5" />
  <PackageReference Include="Microsoft.CodeAnalysis.CSharp" Version="5.9.0" />
  <PackageReference Include="Newtonsoft.Json" Version="13.0.4" />
  <PackageReference Include="PDFsharp-MigraDoc" Version="6.2.4" />
</ItemGroup>
```

```
<ItemGroup>
  <Compile Update="Properties\Settings.Designer.cs">
    <DesignTimeSharedInput>True</DesignTimeSharedInput>
    <AutoGen>True</AutoGen>
    <DependentUpon>Settings.settings</DependentUpon>
  </Compile>
</ItemGroup>
```

```
<ItemGroup>
  <None Update="Properties\Settings.settings">
    <Generator>SettingsSingleFileGenerator</Generator>
    <LastGenOutput>Settings.Designer.cs</LastGenOutput>
  </None>
</ItemGroup>
```

```
</Project>
```

=== ПОЛНЫЙ ТЕКСТ ДОКУМЕНТА: FilterViewModel.cs ===

```
using System.Collections.ObjectModel;
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;
using ContextSlicer.Filesystem;

namespace ContextSlicer
{
    public partial class FilterViewModel : ObservableObject
    {
        // Прямые ссылки на глобальные ObservableCollection из нашего сервиса
        public ObservableCollection<string> ExcludedFolders =>
            GlobalFilterService.Current.ExcludedFolders;
        public ObservableCollection<string> ExcludedExtensions =>
            GlobalFilterService.Current.ExcludedExtensions;

        // Поля ввода для добавления новых элементов
        [ObservableProperty] private string _newFolderInput = string.Empty;
        [ObservableProperty] private string _newExtensionInput = string.Empty;

        // Выбранные элементы в списках (для удаления)
        [ObservableProperty] private string? _selectedFolder;
        [ObservableProperty] private string? _selectedExtension;

        // Команда добавления новой папки в черный список
        [RelayCommand]
        private void AddFolder()
        {
            if (string.IsNullOrWhiteSpace(NewFolderInput)) return;

            string cleanFolder = NewFolderInput.Trim();
            if (!ExcludedFolders.Contains(cleanFolder))
            {
                ExcludedFolders.Add(cleanFolder);
                GlobalFilterService.SaveFilters(); // Сразу пишем изменения в json
                NewFolderInput = string.Empty; // Очищаем поле ввода
            }
        }

        // Команда удаления папки из списка
        [RelayCommand]
        private void DeleteFolder()
        {
            if (SelectedFolder == null) return;

            ExcludedFolders.Remove(SelectedFolder);
            GlobalFilterService.SaveFilters();
            SelectedFolder = null;
        }

        // Команда добавления нового расширения в черный список
        [RelayCommand]
        private void AddExtension()
        {
            if (string.IsNullOrWhiteSpace(NewExtensionInput)) return;

            string cleanExt = NewExtensionInput.Trim();
            // Если пользователь забыл поставить точку в начале (например, написал "bak"), ставим её
            // принудительно
            if (!cleanExt.StartsWith("."))
            {
                cleanExt = "." + cleanExt;
            }
        }
    }
}
```

```
if (!ExcludedExtensions.Contains(cleanExt))
{
    ExcludedExtensions.Add(cleanExt);
    GlobalFilterService.SaveFilters();
    NewExtensionInput = string.Empty;
}

// Команда удаления расширения из списка
[RelayCommand]
private void DeleteExtension()
{
    if (SelectedExtension == null) return;

    ExcludedExtensions.Remove(SelectedExtension);
    GlobalFilterService.SaveFilters();
    SelectedExtension = null;
}
}
```

=== ПОЛНЫЙ ТЕКСТ ДОКУМЕНТА: FilterWindow.xaml ===

```
<Window x:Class="ContextSlicer.FilterWindow"
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
xmlns:local="clr-namespace:ContextSlicer"
mc:Ignorable="d"
Title="Глобальные фильтры исключений"
Height="500" Width="650"
WindowStartupLocation="CenterOwner"
Background="{DynamicResource WindowBackgroundBrush}"
Foreground="{DynamicResource TextBrush}" Icon="/ContentSlicer.ico">

<Window.Resources>
<Style TargetType="Button">
<Setter Property="Background" Value="{DynamicResource TextBoxBackgroundBrush}"/>
<Setter Property="Foreground" Value="{DynamicResource TextBrush}"/>
<Setter Property="BorderBrush" Value="{DynamicResource BorderBrush}"/>
<Setter Property="Padding" Value="10,2"/>
<Setter Property="Height" Value="24"/>
<Setter Property="Template">
<Setter.Value>
<ControlTemplate TargetType="Button">
<Border x:Name="mainBorder" Background="{TemplateBinding Background}"
BorderBrush="{TemplateBinding BorderBrush}" BorderThickness="1" CornerRadius="3">
<Grid>
<Border x:Name="hoverOverlay" Background="Transparent" CornerRadius="2"/>
<ContentPresenter HorizontalAlignment="Center" VerticalAlignment="Center"
Margin="{TemplateBinding Padding}"/>
</Grid>
</Border>
<ControlTemplate.Triggers>
<Trigger Property="IsMouseOver" Value="True">
<Setter TargetName="mainBorder" Property="BorderBrush" Value="#007ACC"/>
<Setter TargetName="hoverOverlay" Property="Background" Value="#15FFFFFF"/>
</Trigger>
<Trigger Property="IsPressed" Value="True">
<Setter TargetName="hoverOverlay" Property="Background" Value="#30000000"/>
</Trigger>
</ControlTemplate.Triggers>
</ControlTemplate>
</Setter.Value>
</Setter>
</Style>

<!-- Стилизация списков ListBox под тему оформления -->
<Style TargetType="ListBox">
<Setter Property="Background" Value="{DynamicResource ControlBackgroundBrush}"/>
<Setter Property="Foreground" Value="{DynamicResource TextBrush}"/>
<Setter Property="BorderBrush" Value="{DynamicResource BorderBrush}"/>
<Setter Property="BorderThickness" Value="1"/>
</Style>
</Window.Resources>

<Grid Margin="15">
<Grid.ColumnDefinitions>
<ColumnDefinition Width="*"/>
<ColumnDefinition Width="15"/>
<!-- Разделитель -->
<ColumnDefinition Width="*"/>
</Grid.ColumnDefinitions>
<Grid.RowDefinitions>
<RowDefinition Height="*"/>
<RowDefinition Height="Auto"/>
</Grid.RowDefinitions>
```

```

<!-- ЛЕВАЯ КОЛОНКА: ИСКЛЮЧАЕМЫЕ ПАПКИ -->
<GroupBox Header="Игнорируемые папки (каталоги)" Grid.Column="0" Grid.Row="0"
Margin="0,0,0,10">
<Grid Margin="5">
<Grid.RowDefinitions>
<RowDefinition Height="*/>
<RowDefinition Height="Auto"/>
</Grid.RowDefinitions>

<ListBox ItemsSource="{Binding ExcludedFolders}" SelectedItem="{Binding SelectedFolder}"
Grid.Row="0" Margin="0,0,0,8"/>

<Grid Grid.Row="1">
<Grid.ColumnDefinitions>
<ColumnDefinition Width="*/>
<ColumnDefinition Width="Auto"/>
<ColumnDefinition Width="Auto"/>
</Grid.ColumnDefinitions>
<TextBox Text="{Binding NewFolderInput, UpdateSourceTrigger=PropertyChanged}"
Grid.Column="0" Height="24" VerticalContentAlignment="Center" Margin="0,0,5,0"
Background="{DynamicResource TextBoxBackgroundBrush}" Foreground="{DynamicResource
TextBrush}"/>
<Button Content="□" Command="{Binding AddFolderCommand}" Grid.Column="1" Width="26"
Padding="0" FontWeight="Bold" Margin="0,0,5,0" Foreground="#007ACC"/>
<Button Content="□" Command="{Binding DeleteFolderCommand}" Grid.Column="2" Width="26"
Padding="0" Foreground="#E53935" FontWeight="Bold"/>
</Grid>
</Grid>
</GroupBox>

<!-- ПРАВАЯ КОЛОНКА: ИСКЛЮЧАЕМЫЕ РАСШИРЕНИЯ -->
<GroupBox Header="Игнорируемые расширения файлов" Grid.Column="2" Grid.Row="0"
Margin="0,0,0,10">
<Grid Margin="5">
<Grid.RowDefinitions>
<RowDefinition Height="*/>
<RowDefinition Height="Auto"/>
</Grid.RowDefinitions>

<ListBox ItemsSource="{Binding ExcludedExtensions}" SelectedItem="{Binding
SelectedExtension}" Grid.Row="0" Margin="0,0,0,8"/>

<Grid Grid.Row="1">
<Grid.ColumnDefinitions>
<ColumnDefinition Width="*/>
<ColumnDefinition Width="Auto"/>
<ColumnDefinition Width="Auto"/>
</Grid.ColumnDefinitions>
<TextBox Text="{Binding NewExtensionInput, UpdateSourceTrigger=PropertyChanged}"
Grid.Column="0" Height="24" VerticalContentAlignment="Center" Margin="0,0,5,0"
Background="{DynamicResource TextBoxBackgroundBrush}" Foreground="{DynamicResource
TextBrush}"/>
<Button Content="□" Command="{Binding AddExtensionCommand}" Grid.Column="1" Width="26"
Padding="0" FontWeight="Bold" Margin="0,0,5,0" Foreground="#007ACC"/>
<Button Content="□" Command="{Binding DeleteExtensionCommand}" Grid.Column="2"
Width="26" Padding="0" Foreground="#E53935" FontWeight="Bold"/>
</Grid>
</Grid>
</GroupBox>

<!-- КНОПКА ЗАКРЫТИЯ ОКНА -->
<Button Content="ЗАКРЫТЬ НАСТРОЙКИ" Grid.Row="1" Grid.Column="0" Grid.ColumnSpan="3"
Height="36"
Click="CloseButton_Click" Background="#007ACC" Foreground="White" FontWeight="Bold"
Style="{x:Null}"/>

```

```
</Grid>  
</Window>
```

=== ПОЛНЫЙ ТЕКСТ ДОКУМЕНТА: FilterWindow.xaml.cs ===

```
using System.Windows;

namespace ContextSlicer
{
    public partial class FilterWindow : Window
    {
        public FilterWindow()
        {
            InitializeComponent();
            DataContext = new FilterViewModel();

            // Окно фильтров автоматически подстраивается под состояние темы главного приложения
            this.SourceInitialized += (s, e) => {
                var mainWin = System.Windows.Application.Current.MainWindow as MainWindow;
                bool isDark = (mainWin?.DataContext as MainViewModel)?.IsDarkTheme ?? false;
                InverseBooleanConverter.ApplyTitleBarColor(this, isDark);
            };
        }

        private void CloseButton_Click(object sender, RoutedEventArgs e)
        {
            // Просто закрываем окно — автосохранение списков в JSON происходит на лету
            this.Close();
        }
    }
}
```

=== ПОЛНЫЙ ТЕКСТ ДОКУМЕНТА: InverseBooleanConverter.cs ===

```
using System;
using System.Globalization;
using System.Windows.Data;

namespace ContextSlicer;

// Конвертер для инверсии булевых значений (True <-> False) в разметке RadioButton
public class InverseBooleanConverter : IValueConverter
{
    // Статический экземпляр для прямого вызова через {x:Static} без регистрации в ресурсах
    public static readonly InverseBooleanConverter Instance = new InverseBooleanConverter();

    // Из ViewModel в Интерфейс (если IsPdfFormat == false, то Радиокнопка TXT нажимается в True)
    public object Convert(object value, Type targetType, object parameter, CultureInfo culture)
    {
        if (value is bool booleanValue)
        {
            return !booleanValue;
        }
        return false;
    }

    // Из Интерфейса во ViewModel (если пользователь кликнул на TXT, во ViewModel IsPdfFormat пишется как False)
    public object ConvertBack(object value, Type targetType, object parameter, CultureInfo culture)
    {
        if (value is bool booleanValue)
        {
            return !booleanValue;
        }
        return false;
    }

    // Универсальный метод перекрашивания заголовка любого WPF-окна
    public static void ApplyTitleBarColor(System.Windows.Window window, bool isDark)
    {
        try
        {
            var interopHelper = new System.Windows.Interop.WindowInteropHelper(window);
            IntPtr hwnd = interopHelper.Handle;
            if (hwnd == IntPtr.Zero) return;

            int attribute = 35; // DWMWA_CAPTION_COLOR
            int colorBGR = isDark ? 0x1E1E1E : 0xF5F5F5;
            DwmSetWindowAttribute(hwnd, attribute, ref colorBGR, sizeof(int));

            int textAttribute = 36; // DWMWA_TEXT_COLOR
            int textColorBGR = isDark ? 0xF1F1F1 : 0x000000;
            DwmSetWindowAttribute(hwnd, textAttribute, ref textColorBGR, sizeof(int));
        }
        catch { }
    }

    [System.Runtime.InteropServices.DllImport("dwmapi.dll")]
    private static extern int DwmSetWindowAttribute(IntPtr hwnd, int attr, ref int attrValue, int attrSize);
}
```

=== ПОЛНЫЙ ТЕКСТ ДОКУМЕНТА: MainViewModel.cs ===

```
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;
using ContextSlicer.Filesystem;
using ContextSlicer.Google;
using Microsoft.WindowsAPICodePack.Dialogs;
using MigraDoc;
using Newtonsoft.Json;
using Newtonsoft.Json.Linq;
using PdfSharp.Fonts;
using System;
using System.Collections.Generic;
using System.Collections.ObjectModel;
using System.IO;
using System.Linq;
using System.Net;
using System.Net.Http;
using System.Net.NetworkInformation;
using System.Reflection;
using System.Windows;
using System.Windows.Media;

namespace ContextSlicer;

public partial class MainViewModel : ObservableObject
{
    private readonly string _configFilePath;
    [ObservableProperty] private bool _isDarkTheme;
    [ObservableProperty] private ObservableCollection<ProjectConfig> _projects = new();
    [ObservableProperty] private ProjectConfig? _selectedProject;
    [ObservableProperty] private bool _isAutoSaveEnabled;
    // Список модулей выбранного проекта
    [ObservableProperty] private bool _isPdfFormat; // Если true – рендерим в PDF, если
    false – в TXT
    [ObservableProperty] private bool _isProjectBlockExpanded;
    [ObservableProperty] private bool _isProjectNameInvalid;
    [ObservableProperty] private bool _isModuleNameInvalid;
    [ObservableProperty] private bool _includeDirectoryStructure = true;
    [ObservableProperty] private FileSystemNode? _rootNode;
    [ObservableProperty] private string _moduleRules = string.Empty;

    // Поля ввода UI
    [ObservableProperty] private string _projectNameInput = string.Empty;
    [ObservableProperty] private string _rootPath = string.Empty;
    [ObservableProperty] private string _outputPath = string.Empty;
    [ObservableProperty] private string _promptRules = string.Empty;

    [ObservableProperty] private string _moduleNameInput = string.Empty;
    [ObservableProperty] private string _contextFileName = string.Empty;
    [ObservableProperty]
    private ObservableCollection<ContextModule> _modules = new();
    //Подсчёт токенов
    [ObservableProperty] private long _totalCharacters;
    [ObservableProperty] private long _estimatedTokens;
    [ObservableProperty] private bool _isCalculatingSize;
    // Внимательно проверьте написание этой переменной:
    [ObservableProperty]
    private ContextModule? _selectedModule;
    private bool _isUpdatingFields = false;
    [ObservableProperty] private string _newModuleNameInput = string.Empty;
    // Редактирование названия модуля
    [ObservableProperty] private bool _isEditOverlayVisible;
    [ObservableProperty] private string _editModuleNameInput = string.Empty;
    // Одна универсальная команда для контекстного меню TreeView
    [CommunityToolkit.Mvvm.Input.RelayCommand]
    private void AddNodeToFilters(FileSystemNode? node)
    {

```

```

if (node == null) return;

if (node.IsFile)
{
    // Если кликнули по файлу – отправляем его расширение в фильтр
    string ext = System.IO.Path.GetExtension(node.FullPath).ToLower();
    if (!string.IsNullOrEmpty(ext) &&
        !GlobalFilterService.Current.ExcludedExtensions.Contains(ext))
    {
        GlobalFilterService.Current.ExcludedExtensions.Add(ext);
        GlobalFilterService.SaveFilters();
        TriggerTreeRefresh();
    }
}
else
{
    // Если кликнули по папке – отправляем её имя в фильтр
    string folderName = node.Name;
    if (!GlobalFilterService.Current.ExcludedFolders.Contains(folderName))
    {
        GlobalFilterService.Current.ExcludedFolders.Add(folderName);
        GlobalFilterService.SaveFilters();
        TriggerTreeRefresh();
    }
}

// Вспомогательный метод перезагрузки дерева файлов (оставляем старый)
// Вспомогательный метод перезагрузки дерева файлов (ИСПРАВЛЕНО: Полный принудительный
пересчет)
private void TriggerTreeRefresh()
{
    if (SelectedProject != null && !string.IsNullOrEmpty(RootPath))
    {
        // Запускаем стандартный метод сборки дерева, который у вас вызывается при выборе папки.
        // Передаем текущий корневой путь и список уже сохраненных чекнутых файлов модуля
        var savedChecked = new List<string>();
        if (SelectedModule?.CheckedFiles != null)
        {
            savedChecked = new List<string>(SelectedModule.CheckedFiles);
        }

        // Перестраиваем структуру дерева с учетом НОВЫХ глобальных фильтров
        RootNode = ContextBuilderService.BuildTree(RootPath, savedChecked);

        // Уведомляем интерфейс WPF, что дерево файлов полностью обновилось
        OnPropertyChanged(nameof(RootNode));
    }
}

[RelayCommand]
private void OpenFiltersWindow()
{
    var filterWin = new FilterWindow();
    // Устанавливаем главное окно владельцем, чтобы новое окно красиво центрировалось поверх
    него
    filterWin.Owner = System.Windows.Application.Current.MainWindow;

    // ShowDialog() полностью блокирует поток выполнения до тех пор, пока пользователь не
    закроет окно фильтров.
    // Как только пользователь нажмет кнопку «ЗАКРЫТЬ НАСТРОЙКИ» или крестик – код пойдет
    дальше.
    filterWin.ShowDialog();
}

```

```

// ИСПРАВЛЕНО: Прямо здесь вызываем наш железно работающий метод полной пересборки
дерева файлов.
// Это заставит утилиту мгновенно убрать с экрана все только что добавленные
папки/расширения
// или вернуть обратно те, что пользователь вручную удалил из списков!
TriggerTreeRefresh();
}
[RelayCommand]
private void ShowEditOverlay()
{
if (SelectedModule == null) return;

// Копируем текущее имя модуля в буферное поле ввода
EditModuleNameInput = SelectedModule.ModuleName;
IsEditOverlayVisible = true;
}
[RelayCommand]
private void CancelEditOverlay()
{
IsEditOverlayVisible = false;
IsModuleNameInvalid = false; // Сбрасываем красную рамку ошибки, если она была
}
[RelayCommand]
private void ConfirmRenameModule()
{
if (SelectedProject == null || SelectedModule == null) return;

string newName = EditModuleNameInput?.Trim() ?? string.Empty;

// 1. Проверка на пустую строку
if (string.IsNullOrEmpty(newName))
{
System.Windows.MessageBox.Show("Имя модуля не может быть пустым!", "Ошибка валидации",
MessageBoxButton.OK, MessageBoxImage.Warning);
return;
}

// 2. Проверка на дубликаты
bool isDuplicate = SelectedProject.Modules.Any(m =>
m != SelectedModule &&
m.ModuleName.Equals(newName, StringComparison.OrdinalIgnoreCase));

if (isDuplicate)
{
System.Windows.MessageBox.Show($"Модуль с названием \"{newName}\" уже существует в этом
проекте!", "Ошибка валидации",
MessageBoxButton.OK, MessageBoxImage.Warning);
return;
}

// 3. Проверка на спецсимволы файловой системы Windows
if (!IsValidName(newName, out string validationError))
{
System.Windows.MessageBox.Show(validationError, "Ошибка валидации", MessageBoxButton.OK,
MessageBoxImage.Warning);
return;
}

// ИСПРАВЛЕНО: Запоминаем ссылку на текущий переименовываемый модуль
var currentModule = SelectedModule;

_isUpdatingFields = true;
try
{
currentModule.ModuleName = newName;

```

```

currentModule.ContextFileName = $"{GetSafeFileName(newName)}.txt";
ContextFileName = currentModule.ContextFileName;
currentModule.ModuleRules = ModuleRules;

// Синхронизируем строку в коллекции
var index = Modules.IndexOf(currentModule);
if (index >= 0)
{
    Modules[index] = currentModule;
}

ModuleNameInput = newName;
}
finally
{
    _isUpdatingFields = false;
}

// ИСПРАВЛЕНО: Откладываем восстановление фокуса ComboBox на следующий такт UI-потока.
// Это гарантирует, что ComboBox выберет именно переименованный пункт, а не сбросится на
индекс 0!
System.Windows.Application.Current.Dispatcher.BeginInvoke(new Action(() =>
{
    _isUpdatingFields = true;
    try
    {
        SelectedModule = currentModule;
    }
    finally
    {
        _isUpdatingFields = false;
    }
}

// Принудительно заставляем селектор кнопок пересчитать состояние
OnPropertyChanged(nameof(IsModuleSelectorEnabled)));
}), System.Windows.Threading.DispatcherPriority.Background);

// Закрываем оверлей и сохраняем проект на диск
IsEditOverlayVisible = false;
SilentSave();
}

// Свойство доступности блока модулей (теперь со строгим уведомлением для интерфейса)
// Настраиваемый черный список папок (дефолтные значения)
[ObservableProperty]
private string _excludedFoldersInput = "bin, obj, .vs, publish, .git, .idea,
node_modules";

// Настраиваемый черный список мусорных и бинарных расширений (дефолтные значения)
[ObservableProperty]
private string _excludedExtensionsInput = ".png, .jpg, .jpeg, .gif, .ico, .bmp, .webp,
.mp3, .wav, .ogg, .flac, .aac, .mp4, .avi, .mkv, .mov, .zip, .rar, .7z, .tar, .gz, .dll,
.exe, .pdb, .suo, .user, .fbx, .obj, .max, .blend, .3ds, .bak, .tmp, .temp, .log";

[RelayCommand]
private async Task RecalculateContextSizeAsync()
{
    if (RootNode == null)
    {
        TotalCharacters = 0;
        EstimatedTokens = 0;
        return;
    }
}

IsCalculatingSize = true;

```

```

await Task.Run(() =>
{
var checkedFiles = new List<FileSystemNode>();
ContextBuilderService.GetCheckedFiles(RootNode, checkedFiles);

long charCount = 0;

// 1. Считаем символы в выбранных файлах исходного кода
foreach (var file in checkedFiles)
{
if (file != null && File.Exists(file.FullPath))
{
try
{
var fi = new FileInfo(file.FullPath);
charCount += fi.Length;

// НОВОЕ: Если структура каталогов включена, добавляем вес строк разметки структуры папок
if (IncludeDirectoryStructure)
{
// Примерно 15 символов на строку вида " [Файл] Relative\Path\File.cs\n"
charCount += (file.RelativePath?.Length ?? 0) + 10;
}
}
}
catch { }
}

// 2. Добавляем символы из полей правил и служебную разметку тегов
charCount += (PromptRules?.Length ?? 0);
charCount += (ModuleRules?.Length ?? 0);
charCount += 150; // Запас на служебные теги

// 3. Считаем токены. Код (английский) ~ 4 символа на токен.
// Русский текст в правилах ~ 2 символа на токен. Делаем взвешенную безопасную оценку:
long rulesLength = (PromptRules?.Length ?? 0) + (ModuleRules?.Length ?? 0);
long codeLength = charCount - rulesLength;
if (codeLength < 0) codeLength = 0;

long tokensFromCode = codeLength / 4;
long tokensFromRules = rulesLength / 2; // Более тяжелые токены для кириллицы

TotalCharacters = charCount;
EstimatedTokens = tokensFromCode + tokensFromRules;
});

IsCalculatingSize = false;
}
partial void OnPromptRulesChanged(string value) => _ = RecalculateContextSizeAsync();
partial void OnModuleRulesChanged(string value) => _ = RecalculateContextSizeAsync();
// Автоматический пересчет токенов при клике на чекбокс структуры каталогов
partial void OnIncludeDirectoryStructureChanged(bool value) => _ =
RecalculateContextSizeAsync();

[RelayCommand]
private void DeleteCurrentModule()
{
if (SelectedModule == null) return;

// Спрашиваем подтверждение удаления на языке системы
var result = System.Windows.MessageBox.Show(

```

```

"Вы уверены, что хотите полностью удалить этот модуль и все его сохраненные настройки
файлов?",
"Удаление модуля",
MessageBoxButton.YesNo,
MessageBoxImage.Question);

if (result == MessageBoxResult.Yes)
{
var moduleToRemove = SelectedModule;
SelectedModule = null; // Сбрасываем выбор

Modules.Remove(moduleToRemove);
OnPropertyChanged(nameof(IsModuleSelectorEnabled));

// Автоматически сохраняем изменения на диск
SaveProject();
}
}

// public bool IsModuleSelectorEnabled => SelectedProject != null && Modules != null &&
Modules.Count > 0;

// Свойство для динамического вывода имени проекта в заголовок Expander
// Динамическое свойство для заголовка экспандера (Исключает наложение текста)
public string DisplayProjectName
{
get
{
// Если проект выбран и у него есть имя — выводим его
if (SelectedProject != null && !string.IsNullOrEmpty(SelectedProject.ProjectName))
{
return SelectedProject.ProjectName;
}

// Если проект не выбран — безопасно вытаскиваем локализованную строку из ресурсов окна
if (Application.Current?.MainWindow?.Resources != null &&
Application.Current.MainWindow.Resources.Contains("Str_ProjNotSelected"))
{
return Application.Current.MainWindow.Resources["Str_ProjNotSelected"] as string ?? "---
";
}

// Фолбэк на случай, если ресурсы еще не успели прогрузиться при самом первом старте
return "---";
}
}

public MainViewModel()
{
// Точный и правильный путь к настройке шрифтов Windows в PDFsharp/MigraDoc
PdfSharp.Fonts.GlobalFontSettings.UseWindowsFontsUnderWindows = true;

// Оставляем провайдер кодировок для кириллицы
System.Text.Encoding.RegisterProvider(System.Text.CodePagesEncodingProvider.Instance);

// ... остальной код конструктора без изменений ...
string appData = Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData);
string appDir = Path.Combine(appData, "ContextSlicer");
Directory.CreateDirectory(appDir);
_configFilePath = Path.Combine(appDir, "projects.json");
IncludeDirectoryStructure = Properties.Settings.Default.IncludeDirectoryStructure;

```

```

IsAutoSaveEnabled = Properties.Settings.Default.AutoSaveOnExit;
IsDarkTheme = Properties.Settings.Default.IsDarkTheme;
IsProjectBlockExpanded = Properties.Settings.Default.IsProjectBlockExpanded;
OnIsDarkThemeChanged(IsDarkTheme);
// Ваша существующая инициализация (InitializeLanguage и т.д.)
LoadAvailableServiceAccounts();

LoadProjects();
}

// Логика при выборе ПРОЕКТА
// Модифицированная логика автоматической фоновой проверки кэша при смене проекта
// 3. Модифицированная логика автоматической фоновой проверки кэша
(OnSelectedProjectChanged)
partial void OnSelectedProjectChanged(ProjectConfig? value)
{
    if (value != null)
    {
        ProjectNameInput = value.ProjectName;
        RootPath = value.RootPath;
        OutputPath = value.OutputPath;
        PromptRules = value.PromptRules;
        IncludeDirectoryStructure = value.IncludeDirectoryStructure;

        Modules = new ObservableCollection<ContextModule>(value.Modules);

        if (value.Type == ProjectType.GoogleDoc)
        {
            System.Windows.Application.Current.Dispatcher.BeginInvoke(new Func<Task>(async () =>
            {
                var downloader = new GoogleDownloader();
                string projectBaseDir = AppDomain.CurrentDomain.BaseDirectory;
                if (downloader.IsCacheExpired(value.RootPath, projectBaseDir))
                {
                    string keyPath = GetKeyPathByEmail(value.GoogleApiKey);
                    if (!string.IsNullOrEmpty(keyPath) && File.Exists(keyPath))
                    {
                        await downloader.DownloadToCacheAsync(value.RootPath, projectBaseDir, keyPath);
                    }
                }
                RefreshTreeView(value.Modules.FirstOrDefault()?.CheckedFiles ?? new List<string>());
                SelectedModule = Modules.FirstOrDefault();
            }), System.Windows.Threading.DispatcherPriority.Background);
        }

        else
        {
            RefreshTreeView(new List<string>());
            SelectedModule = Modules.FirstOrDefault();
        }
        else
        {
            Modules.Clear();
            RootNode = null;
        }
        OnPropertyChanged(nameof(IsModuleSelectorEnabled));
        OnPropertyChanged(nameof(DisplayProjectName));
    }
}

private string GetKeyPathByEmail(string email)
{
    if (string.IsNullOrEmpty(email)) return string.Empty;

```

```

string systemStorageDir = Path.Combine(AppDomain.CurrentDomain.BaseDirectory,
"ServiceAccounts");
if (!Directory.Exists(systemStorageDir)) return string.Empty;

var files = Directory.GetFiles(systemStorageDir, "*.json");
foreach (var file in files)
{
    try
    {
        string content = File.ReadAllText(file);
        var json = JObject.Parse(content);
        if (json["client_email"]?.ToString() == email)
        {
            return file; // Нашли физический файл ключа
        }
    }
    catch { }
}
return string.Empty;
}

// ОБНОВЛЕННАЯ КОМАНДА СИНХРОНИЗАЦИИ (Кнопка V): Принудительный сброс кэша и
перечитывание
// Внутри MainViewModel.cs
[RelayCommand]
private async Task SyncProjectSource()
{
    if (SelectedProject == null || string.IsNullOrWhiteSpace(RootPath)) return;

    // Если текущий проект — Google Документ, выполняем жесткий перезапрос структуры из
    облака
    if (SelectedProject.Type == ProjectType.GoogleDoc)
    {
        string encryptedKey = Properties.Settings.Default.EncryptedGoogleApiKey ?? string.Empty;
        string apiKey = SecureCredentialStorage.DecryptString(encryptedKey);
        if (string.IsNullOrWhiteSpace(apiKey))
        {
            string authErr = Application.Current.Resources["Str_Err_GoogleAuthFailed"] as string
            ?? "Google API Key отсутствует или поврежден!";
            string authTitle = Application.Current.Resources["Str_Err_ValidationTitle"] as string
            ?? "Ошибка авторизации";
            MessageBox.Show(authErr, authTitle, MessageBoxButton.OK, MessageBoxImage.Warning);
            return;
        }

        // Активируем полноэкранный адаптивный оверлей обновления структуры проекта
        IsProjectLoading = true;
        try
        {
            // Интеграция с подсистемой отображения статусов
            await StartProjectUpdateAsync(UpdateType.GoogleApiLoad, RootPath);

            var downloader = new GoogleDownloader();
            string projectBaseDir = AppDomain.CurrentDomain.BaseDirectory;

            // Принудительно качаем свежую структуру, игнорируя проверку IsCacheExpired
            bool success = await downloader.DownloadToCacheAsync(RootPath, projectBaseDir, apiKey);
            if (!success)
            {
                string syncErr = Application.Current.Resources["Str_Err_GoogleDocUnavailable"] as string
                ?? "Не удалось обновить кэш из облака.";
                string syncTitle = Application.Current.Resources["Str_Err_NetworkTitle"] as string
                ?? "Ошибка синхронизации";
                MessageBox.Show(syncErr, syncTitle, MessageBoxButton.OK, MessageBoxImage.Error);
                return;
            }
        }
        catch { }
    }
}

```

```

    }
    }
    finally
    {
        IsProjectLoading = false;
        IsUpdateOverlayVisible = false; // Гарантированно гасим оверлей обновления
    }
}

// Полностью перестраиваем синтаксическое дерево (WPF перерисует ноды на экране на лету)
RefreshTreeView(SelectedModule?.CheckedFiles ?? new List<string>());
if (RootNode != null && SelectedModule != null)
{
    FastPreloadSavedEntries(SelectedModule, RootNode);
}

string successMsg = Application.Current.Resources["Str_Status_ProjectSaved"] as string
?? "Синхронизация структуры успешно завершена!";
string successTitle = Application.Current.Resources["Str_Msg_EmailCopiedTitle"] as
string
?? "Успех";
MessageBox.Show(successMsg, successTitle, MessageBoxButton.OK,
MessageBoxImage.Information);
}

// Логика при выборе конкретного МОДУЛЯ (контекста) внутри проекта
// Имя должно быть ОДИН В ОДИН как имя свойства после On...
partial void OnSelectedModuleChanged(ContextModule? value)
{
    if (_isUpdatingFields) return;

    _isUpdatingFields = true;
    try
    {
        {
            if (value != null)
            {
                ModuleNameInput = value.ModuleName;
                ContextFileName = value.ContextFileName;
                ModuleRules = value.ModuleRules;

                // 1. Строим базовую структуру файлов с диска
                RefreshTreeView(value.CheckedFiles);

                // 2. ИСПРАВЛЕНО: Мгновенно накатываем сохраненные структуры из JSON.
                // Папки покроются закрашенными квадратами в ту же секунду!
                if (RootNode != null)
                {
                    FastPreloadSavedEntries(value, RootNode);
                }
            }
            else
            {
                ModuleNameInput = string.Empty;
                ContextFileName = string.Empty;
                ModuleRules = string.Empty;
                RefreshTreeView(new List<string>());
            }
        }
        finally
        {
            _isUpdatingFields = false;
        }
    }
}

```

```

// Внутри MainViewModel.cs
// Внутри MainViewModel.cs

// ИСПРАВЛЕНО: Изменено на асинхронное выполнение для безопасного ожидания парсинга без
ContinueWith
private async void RefreshTreeView(List<string> checkedFiles)
{
    if (SelectedProject == null || string.IsNullOrWhiteSpace(RootPath))
    {
        RootNode = null;
        _ = RecalculateContextSizeAsync();
        return;
    }

    switch (SelectedProject.Type)
    {
        case ProjectType.GoogleDoc:
            string googleDocId = GoogleDownloader.ExtractDocumentId(RootPath);
            string googleCachePath = Path.Combine(AppDomain.CurrentDomain.BaseDirectory,
            "googlecache", $"{googleDocId}.json");

            RootNode = new FileSystemNode
            {
                Name = SelectedProject.ProjectName,
                FullPath = googleCachePath,
                RelativePath = googleCachePath,
                IsFile = false
            };

            try
            {
                RootNode.Children.Clear();
                // Создаем временную техническую ноду-заглушку
                RootNode.Children.Add(new FileSystemNode { Name = "LoadingStub...", Parent = RootNode,
                IsFile = true });
            }

            // ИСПРАВЛЕНО: Прямое и безопасное ожидание задачи парсинга вместо ContinueWith
            await PopulateSyntaxNodesAsync(RootNode);

            // Код ниже гарантированно выполнится в основном UI-потокe после успешного парсинга
            if (RootNode != null)
            {
                RootNode.VerifyCheckState();
                if (SelectedModule != null)
                {
                    FastPreloadSavedEntries(SelectedModule, RootNode);
                }
            }
            catch (OperationCanceledException)
            {
                System.Diagnostics.Debug.WriteLine("[Google Sync] Парсинг структуры был отменен.");
            }
            catch (Exception ex)
            {
                System.Diagnostics.Debug.WriteLine($"[Google Sync Parser Break] {ex.Message}");
            }
            break;

            case ProjectType.WordDoc:
                RootNode = new FileSystemNode
                {
                    Name = Path.GetFileName(RootPath),
                    FullPath = RootPath,
                    RelativePath = Path.GetFileName(RootPath),
                    IsFile = false
                }
    }
}

```

```

};
RootNode.Children.Clear();
RootNode.Children.Add(new FileSystemNode { Name = "LoadingStub...", Parent = RootNode
});
break;

default:
if (Directory.Exists(RootPath))
{
RootNode = ContextBuilderService.BuildTree(RootPath, checkedFiles);
if (RootNode != null) DeepVerifyCheckStates(RootNode);
}
else
{
RootNode = null;
}
break;
}

OnPropertyChanged(nameof(RootNode));
_ = RecalculateContextSizeAsync();
}

[RelayCommand]
private void CreateNewProject()
{
string name = string.IsNullOrWhiteSpace(ProjectNameInput)
? $"Проект {Projects.Count + 1}"
: ProjectNameInput.Trim();

// 1. ВАЛИДАЦИЯ НА СПЕЦСИМВОЛЫ
if (!IsValidName(name, out string validationError))
{
System.Windows.MessageBox.Show(validationError, "Ошибка валидации",
MessageBoxButton.OK, MessageBoxImage.Warning);
return;
}

// 2. ПРОВЕРКА НА ДУБЛИКАТЫ
bool isDuplicate = Projects.Any(p => p.ProjectName.Equals(name,
StringComparison.OrdinalIgnoreCase));
if (isDuplicate)
{
System.Windows.MessageBox.Show($"Проект с названием \"{name}\" уже существует!",
"Внимание",
MessageBoxButton.OK, MessageBoxImage.Warning);
return;
}

var newProject = new ProjectConfig { ProjectName = name };
Projects.Add(newProject);
SelectedProject = newProject;

// 3. ОЧИСТКА И СОХРАНЕНИЕ
ProjectNameInput = string.Empty;
IsProjectNameInvalid = false;
SilentSave();
}

[RelayCommand]
private void CreateNewModule()
{
if (SelectedProject == null)
{

```

```

System.Windows.MessageBox.Show("Сначала выберите или создайте проект!", "Внимание",
MessageBoxButton.OK, MessageBoxImage.Warning);
return;
}

// Читаем имя из ИЗОЛИРОВАННОГО поля ввода нового модуля
string name = string.IsNullOrEmpty(NewModuleNameInput)
? $"Модуль {Modules.Count + 1}"
: NewModuleNameInput.Trim();

bool isDuplicate = SelectedProject.Modules.Any(m =>
m.ModuleName.Equals(name, StringComparison.OrdinalIgnoreCase));

if (isDuplicate)
{
System.Windows.MessageBox.Show($"Модуль с названием \"{name}\" уже существует в этом
проекте!", "Внимание",
MessageBoxButton.OK, MessageBoxImage.Warning);
return;
}

var newModule = new ContextModule
{
ModuleName = name,
ContextFileName = $"{GetSafeFileName(name)}.txt",
CheckedFiles = new List<string>(),
CheckedEntries = new List<SyntaxEntry>()
};

_isUpdatingFields = true;
try
{
SelectedProject.Modules.Add(newModule);
Modules.Add(newModule);

// Полностью пересоздаем корень дерева файлов для изоляции памяти
RootNode = new FileSystemNode
{
Name = System.IO.Path.GetFileName(SelectedProject.RootPath),
FullPath = SelectedProject.RootPath,
IsFile = false
};
OnPropertyChanged(nameof(RootNode));
RefreshTreeView(new List<string>());

// ОЧИЩАЕМ ИЗОЛИРОВАННОЕ ПОЛЕ. Теперь это никак не затронет ComboBox!
NewModuleNameInput = string.Empty;
}
finally
{
_isUpdatingFields = false;
}

// Мягко переключаем фокус в UI-поток
System.Windows.Application.Current.Dispatcher.BeginInvoke(new Action(() =>
{
SelectedModule = newModule;
}), System.Windows.Threading.DispatcherPriority.Background);

SilentSave();
}

// public bool IsModuleSelectorEnabled => SelectedProject != null && Modules != null &&
Modules.Count > 0;

```

```

[RelayCommand]
private void UpdateCurrentModuleName()
{
    // ЖЕСТКАЯ ЗАЩИТА: Если мы программно обновляем интерфейс, или проект/модуль не выбраны
    - выходим сразу
    if (!_isUpdatingFields || SelectedProject == null || SelectedModule == null) return;

    string newName = ModuleNameInput?.Trim() ?? string.Empty;

    // ГАРАНТИЯ СТАБИЛЬНОСТИ: Если поле ввода пустое, содержит дефолтное "Модуль X"
    // или полностью совпадает с текущим именем модуля - НЕМЕДЛЕННО прекращаем выполнение.
    // Это заблокирует ложные срабатывания при очистке полей и смене фокуса!
    if (string.IsNullOrEmpty(newName) ||
        newName.StartsWith("Модуль ", StringComparison.OrdinalIgnoreCase) ||
        newName.Equals(SelectedModule.ModuleName, StringComparison.Ordinal))
    {
        return;
    }

    if (!IsValidName(newName, out string validationError))
    {
        IsModuleNameInvalid = true;
        System.Windows.MessageBox.Show(validationError, "Ошибка валидации модуля",
            MessageBoxButton.OK, MessageBoxImage.Warning);
        // Возвращаем старое имя обратно в поле, чтобы сбросить ошибку
        _isUpdatingFields = true;
        ModuleNameInput = SelectedModule.ModuleName;
        _isUpdatingFields = false;
        return;
    }

    IsModuleNameInvalid = false;

    // Создаем локальную копию, защищенную от сбросов UI
    var currentModule = SelectedModule;

    currentModule.ModuleName = newName;
    currentModule.ContextFileName = $"{GetSafeFileName(newName)}.txt";
    ContextFileName = currentModule.ContextFileName;
    currentModule.ModuleRules = ModuleRules;

    var index = Modules.IndexOf(currentModule);
    if (index >= 0)
    {
        _isUpdatingFields = true;
        try
        {
            Modules[index] = currentModule;
        }
        finally
        {
            _isUpdatingFields = false;
        }
    }

    SilentSave();
}

// Метод принудительной синхронизации текущего состояния дерева с моделью модуля
// ===== -->
// ИСПРАВЛЕНО: ЖЕСТКАЯ СИНХРОНИЗАЦИЯ ПУТЕЙ БЕЗ ИСКАЖЕНИЯ И ДУБЛИРОВАНИЯ-->

```

```

// ===== -->
// ===== -->
// ИСПРАВЛЕНО: ПОЛНАЯ АДАПТИВНАЯ ОЧИСТКА И СИНХРОНИЗАЦИЯ СТРУКТУРЫ -->
// ===== -->
public void SyncTreeWithModule()
{
    if (SelectedModule == null || RootNode == null) return;

    var checkedFilesList = new List<string>();
    var checkedEntriesList = new List<SyntaxEntry>();

    // 1. ОПРЕДЕЛЯЕМ ТИП ПРОЕКТА ДЛЯ ИЗОЛЯЦИИ ЛОГИКИ ТЕКСТА И КОДА
    bool isTextProject = SelectedProject?.Type == ProjectType.GoogleDoc ||
        SelectedProject?.Type == ProjectType.WordDoc;

    if (!isTextProject)
    {
        // ЛОКАЛЬНЫЙ ПРОЕКТ (КОД C#): Набиваем CheckedFiles списком путей к файлам на диске
        var flatFilesList = new List<FileSystemNode>();
        ContextBuilderService.GetCheckedFiles(RootNode, flatFilesList);
        foreach (var node in flatFilesList)
        {
            checkedFilesList.Add(node.RelativePath);
        }
    }
    else
    {
        // КНИЖНЫЙ ПРОЕКТ (GOOGLE DOCS): CheckedFiles остается чистым и пустым [],
        // так как ссылка на документ уже лежит в RootPath на верхнем уровне JSON!
    }

    // 2. СБОР ОТМЕЧЕННЫХ ГАЛОЧКАМИ СИНТАКСИЧЕСКИХ НОД (ВКЛАДКИ И ГЛАВ)
    var flatSyntaxList = new List<FileSystemNode>();
    CollectAllCheckedSyntaxNodes(RootNode, flatSyntaxList);

    foreach (var node in flatSyntaxList)
    {
        // Оптимизация: Для книг зануляем локальный жесткий FilePath, избавляя JSON от
        // раздувания
        string effFilePath = isTextProject ? string.Empty : node.RelativePath;

        // Берем оригинальный EntryPath, который построил наш JSON-парсер
        string cleanEntryPath = node.EntryPath;

        checkedEntriesList.Add(new SyntaxEntry
        {
            FilePath = effFilePath,
            EntryPath = cleanEntryPath,
            DisplayName = node.Name,
            Type = node.SyntaxType,
            SpanInfo = node.SyntaxSpanInfo
        });
    }

    // 3. ФИКСИРУЕМ ОЧИЩЕННЫЕ ДАННЫЕ В ТЕКУЩЕМ МОДУЛЕ ПРОЕКТА
    SelectedModule.CheckedFiles = checkedFilesList;
    SelectedModule.CheckedEntries = checkedEntriesList;
}

// ИСПРАВЛЕНО: Точечная очистка путей внутри ExecuteSave перед записью JSON
// ===== -->
// ВОССТАНОВЛЕНО: ЧИСТОЕ МОНОЛИТНОЕ СОХРАНЕНИЕ БЕЗ ХАРДКОДА СТРОК -->
// ===== -->

```

```

private void ExecuteSave(bool showMessage)
{
    if (SelectedProject == null)
    {
        if (!string.IsNullOrEmpty(ProjectNameInput))
        {
            var autoProject = new ProjectConfig { ProjectName = ProjectNameInput };
            Projects.Add(autoProject);
            SelectedProject = autoProject;
        }
        else
        {
            if (showMessage)
            {
                // ИСПРАВЛЕНО: Мультиязычный вызов предупреждения об пустом имени проекта
                string warnMsg = Application.Current.Resources["Str_Msg_EnterProjectName"] as string
                ?? "Введите имя проекта перед сохранением.";
                string warnTitle = Application.Current.Resources["Str_Title_Warning"] as string
                ?? "Внимание";
                System.Windows.MessageBox.Show(warnMsg, warnTitle, MessageBoxButton.OK,
                MessageBoxImage.Warning);
            }
            return;
        }
    }

    if (SelectedModule != null && RootNode != null)
    {
        var checkedFilesList = new List<string>();
        bool isTextProject = SelectedProject.Type == ProjectType.GoogleDoc ||
        SelectedProject.Type == ProjectType.WordDoc;

        if (!isTextProject)
        {
            var flatFilesList = new List<FileSystemNode>();
            ContextBuilderService.GetCheckedFiles(RootNode, flatFilesList);
            foreach (var node in flatFilesList)
            {
                checkedFilesList.Add(node.RelativePath);
            }
        }

        var checkedEntriesList = new List<SyntaxEntry>();
        var flatSyntaxList = new List<FileSystemNode>();
        CollectAllCheckedSyntaxNodes(RootNode, flatSyntaxList);

        foreach (var node in flatSyntaxList)
        {
            string effFilePath = isTextProject ? string.Empty : node.RelativePath;
            string cleanEntryPath = node.EntryPath;

            checkedEntriesList.Add(new SyntaxEntry
            {
                FilePath = effFilePath,
                EntryPath = cleanEntryPath,
                DisplayName = node.Name,
                Type = node.SyntaxType,
                SpanInfo = node.SyntaxSpanInfo
            });
        }

        SelectedModule.ModuleName = ModuleNameInput;
        SelectedModule.ContextFileName = GetSafeFileName(ModuleNameInput) + ".txt";
        SelectedModule.ModuleRules = ModuleRules;
        SelectedModule.CheckedFiles = checkedFilesList;
        SelectedModule.CheckedEntries = checkedEntriesList;
    }
}

```

```

var mIdx = Modules.IndexOf(SelectedModule);
if (mIdx >= 0) Modules[mIdx] = SelectedModule;
}

if (SelectedProject == null) return;

SelectedProject.ProjectName = ProjectNameInput;
SelectedProject.RootPath = RootPath;
SelectedProject.OutputPath = OutputPath;
SelectedProject.PromptRules = PromptRules;
SelectedProject.IncludeDirectoryStructure = IncludeDirectoryStructure;
SelectedProject.Modules = Modules.ToList();

var pIdx = Projects.IndexOf(SelectedProject);
if (pIdx >= 0)
{
    Projects[pIdx] = SelectedProject;
    SelectedProject = Projects[pIdx];
}

try
{
    string json = JsonConvert.SerializeObject(Projects, Formatting.Indented);
    File.WriteAllText(_configFilePath, json);

    if (showMessage)
    {
        // ИСПРАВЛЕНО: Локализованное окно успешного сохранения
        string successMsg = Application.Current.Resources["Str_Msg_SaveSuccess"] as string
        ?? "Всё успешно сохранено!";
        string successTitle = Application.Current.Resources["Str_Title_Success"] as string
        ?? "Успех";
        System.Windows.MessageBox.Show(successMsg, successTitle, MessageBoxButton.OK,
        MessageBoxImage.Information);
    }
}
catch (Exception ex)
{
    if (showMessage)
    {
        // ИСПРАВЛЕНО: Локализованное окно системной ошибки ввода-вывода
        string errorPrefix = Application.Current.Resources["Str_Err_SaveJson"] as string
        ?? "Ошибка сохранения JSON";
        string errorTitle = Application.Current.Resources["Str_Err_GeneralErrorTitle"] as string
        ?? "Ошибка";
        System.Windows.MessageBox.Show($"{errorPrefix}: {ex.Message}", errorTitle,
        MessageBoxButton.OK, MessageBoxImage.Error);
    }
}

// Вспомогательный метод для глубокой очистки синтаксических нод
private void ClearSyntaxNodesRecursive(FileSystemNode node)
{
    if (node == null) return;

    // Проходим по детям с конца, чтобы безопасно удалять элементы
    for (int i = node.Children.Count - 1; i >= 0; i--)
    {

```

```

var child = node.Children[i];
if (child.IsSyntaxNode)
{
node.Children.RemoveAt(i);
}
else
{
ClearSyntaxNodesRecursive(child);
}
}
}

// Универсальный метод валидации имени проекта или модуля
private bool IsValidName(string name, out string errorMessage)
{
errorMessage = string.Empty;
if (string.IsNullOrEmpty(name))
{
errorMessage = Application.Current.Resources["Str_Err_Update_ReadDir"] as string
?? "Имя не может быть пустым или состоять только из пробелов.";
return false;
}

char[] invalidChars = System.IO.Path.GetInvalidFileNameChars();
foreach (char c in invalidChars)
{
if (name.Contains(c))
{
string errTemplate = Application.Current.Resources["Str_Err_InvalidServiceAccount"] as
string
?? "Имя содержит недопустимый символ '{0}'.";
errorMessage = string.Format(errTemplate, c);
return false;
}
}
return true;
}

[RelayCommand]
private void UpdateCurrentProjectName()
{
if (SelectedProject == null) return;
string newName = ProjectNameInput?.Trim() ?? string.Empty;
if (newName.Equals(SelectedProject.ProjectName, StringComparison.Ordinal)) return;

if (!IsValidName(newName, out string validationError))
{
IsProjectNameInvalid = true; // ВКЛЮЧАЕМ ПОДСВЕТКУ
System.Windows.MessageBox.Show(validationError, "Ошибка валидации проекта",
MessageBoxButton.OK, MessageBoxImage.Warning);
ProjectNameInput = SelectedProject.ProjectName;
return;
}

IsProjectNameInvalid = false; // СБРАСЫВАЕМ ПОДСВЕТКУ
SelectedProject.ProjectName = newName;

var index = Projects.IndexOf(SelectedProject);
if (index >= 0) { Projects[index] = SelectedProject; SelectedProject = Projects[index];
}
OnPropertyChanged(nameof(DisplayProjectName));
SilentSave();
}

```

```
[RelayCommand]
```

```
partial void OnProjectNameInputChanged(string value) => IsProjectNameInvalid = false;  
partial void OnModuleNameInputChanged(string value) => IsModuleNameInvalid = false;
```

```
// Изменим сигнатуру метода, добавив необязательный параметр showMessage
```

```
[RelayCommand]  
public void SaveProject()  
{  
    ExecuteSave(showMessage: true);  
}
```

```
// Этот метод мы вызываем при закрытии, он работает без всплывающих окон
```

```
public void SilentSave()  
{  
    ExecuteSave(showMessage: false);  
}
```

```
// Основная логика сохранения
```

```
// Рекурсивный сбор всех выбранных элементов кода для сохранения в JSON  
// ИСПРАВЛЕНО: Рекурсивный сбор синтаксических нод БЕЗ мусора и дубликатов  
// Внутри MainViewModel.cs
```

```
// ===== -->
```

```
// ИСПРАВЛЕНО: АДТИВНЫЙ СБОР СИНТАКСИСА ПО ТИПАМ ПРОЕКТОВ -->
```

```
// ===== -->
```

```
private void CollectAllCheckedSyntaxNodes(FileSystemNode node, List<FileSystemNode>  
result)
```

```
{  
    if (node == null) return;
```

```
    if (node.IsSyntaxNode && (node.IsChecked == true || node.IsChecked == null))
```

```
{  
    // Проверяем тип текущего проекта, чтобы не сломать сохранение исходного кода  
    if (SelectedProject?.Type == ProjectType.GoogleDoc || SelectedProject?.Type ==  
        ProjectType.WordDoc)
```

```
{  
    // ДЛЯ КНИГ: Сохраняем любые отмеченные элементы (и вкладки, и главы-листья)  
    result.Add(node);  
}
```

```
else  
{  
    // ДЛЯ КОДА C#: Сохраняем только промежуточные ноды-контейнеры (!node.IsFile)  
    if (!node.IsFile)  
    {  
        result.Add(node);  
    }  
}
```

```
foreach (var child in node.Children)  
{  
    CollectAllCheckedSyntaxNodes(child, result);  
}
```

```
[RelayCommand]  
private void BrowseRootPath()  
{  
    var dialog = new CommonOpenFileDialog { IsFolderPicker = true };  
    if (dialog.ShowDialog() == CommonFileDialogResult.Ok)
```

```

{
RootPath = dialog.FileName;
RefreshTreeView(SelectedModule?.CheckedFiles ?? new List<string>());
}
}

[RelayCommand]
private void BrowseOutputPath()
{
var dialog = new CommonOpenFileDialog { IsFolderPicker = true };
if (dialog.ShowDialog() == CommonFileDialogResult.Ok)
{
OutputPath = dialog.FileName;
}
}

[RelayCommand]
private void BrowseProjectOutputFolder()
{
// Открываем ваш стандартный диалог выбора папок
var dialog = new CommonOpenFileDialog { IsFolderPicker = true };
if (dialog.ShowDialog() == CommonFileDialogResult.Ok)
{
// ИСПРАВЛЕНО: Записываем путь в буферное поле оверлея, активируя валидацию!
EditProjectOutputPathInput = dialog.FileName;
}
}

private void LoadProjects()
{
{
if (File.Exists(_configFilePath))
{
try
{
string json = File.ReadAllText(_configFilePath);
var list = JsonConvert.DeserializeObject<List<ProjectConfig>>(json);
if (list != null)
{
Projects = new ObservableCollection<ProjectConfig>(list);
if (Projects.Count > 0) SelectedProject = Projects[0];
}
}
catch { }
}
}

// Метод автоматической генерации безопасного имени файла
private string GetSafeFileName(string name)
{
if (string.IsNullOrWhiteSpace(name)) return "context";

string safeName = name.Replace(" ", "-");
char[] invalidChars = Path.GetInvalidFileNameChars();

foreach (char c in invalidChars)
{
safeName = safeName.Replace(c.ToString(), "");
}
return string.IsNullOrWhiteSpace(safeName) ? "context" : safeName.ToLower();
}

// Метод, вызываемый при закрытии приложения
public void OnWindowClosing()
{

```

```

// Сохраняем состояние темы в настройки по умолчанию
Properties.Settings.Default.IsDarkTheme = IsDarkTheme;
// Сохраняем состояние чекбокса в Settings приложения
Properties.Settings.Default.AutoSaveOnExit = IsAutoSaveEnabled;
Properties.Settings.Default.IsProjectBlockExpanded = IsProjectBlockExpanded;
Properties.Settings.Default.IncludeDirectoryStructure = IncludeDirectoryStructure; //
СОХРАНЯЕМ ГЛОБАЛЬНО
Properties.Settings.Default.Save();

// Если чекбокс активен — сохраняем проект без вывода MessageBox
if (IsAutoSaveEnabled)
{
    SilentSave();
}
}

// Этот метод срабатывает автоматически каждый раз, когда меняется галочка Тёмной темы
partial void OnIsDarkThemeChanged(bool value)
{
    // Очищаем старую палитру приложения
    Application.Current.Resources.MergedDictionaries.Clear();

    var themeDict = new ResourceDictionary();

    if (value)
    {
        // Накатываем каноничные цвета глубокой тёмной темы VS Code
        themeDict.Add("WindowBackgroundBrush", new SolidColorBrush(Color.FromRgb(0x1E, 0x1E, 0x1E)));
        themeDict.Add("ControlBackgroundBrush", new SolidColorBrush(Color.FromRgb(0x25, 0x25, 0x26)));
        themeDict.Add("TextBoxBackgroundBrush", new SolidColorBrush(Color.FromRgb(0x2D, 0x2D, 0x30)));
        themeDict.Add("TextBrush", new SolidColorBrush(Color.FromRgb(0xF1, 0xF1, 0xF1)));
        themeDict.Add("SplitterBrush", new SolidColorBrush(Color.FromRgb(0x3E, 0x3E, 0x42)));
        themeDict.Add("BorderBrush", new SolidColorBrush(Color.FromRgb(0x43, 0x43, 0x46)));
    }
    else
    {
        // Возвращаем чистые светлые цвета Windows
        themeDict.Add("WindowBackgroundBrush", new SolidColorBrush(Color.FromRgb(0xF5, 0xF5, 0xF5)));
        themeDict.Add("ControlBackgroundBrush", new SolidColorBrush(Color.FromRgb(0xFF, 0xFF, 0xFF)));
        themeDict.Add("TextBoxBackgroundBrush", new SolidColorBrush(Color.FromRgb(0xFF, 0xFF, 0xFF)));
        themeDict.Add("TextBrush", new SolidColorBrush(Color.FromRgb(0x00, 0x00, 0x00)));
        themeDict.Add("SplitterBrush", new SolidColorBrush(Color.FromRgb(0xE0, 0xE0, 0xE0)));
        themeDict.Add("BorderBrush", new SolidColorBrush(Color.FromRgb(0xCC, 0x2C, 0xCC)));
    }

    // Внедряем новую палитру в глобальный контекст WPF
    Application.Current.Resources.MergedDictionaries.Add(themeDict);

    // Красим заголовок главного окна на лету
    if (Application.Current.MainWindow is MainWindow mainWin)
    {
        InverseBooleanConverter.ApplyTitleBarColor(mainWin, value);
    }
}
}

```

```

// Маленький служебный класс-заглушка для связки MigraDoc с системными шрифтами
private CancellationTokenSource? _cts;

// Свойства для отображения оверлея прогресса
[ObservableProperty] private bool _isProcessing;
[ObservableProperty] private int _progressValue;
[ObservableProperty] private int _progressMax;
[ObservableProperty] private string _progressText = string.Empty;
[ObservableProperty] private string _currentFileText = string.Empty;

// Команда "Отмена"
[RelayCommand]
private void CancelGeneration()
{
    _cts?.Cancel();
    ProgressText = "Отмена операции...";
}
// Пробегает по всему дереву до самых глубоких файлов и заставляет папки рассчитать свои
квадратики снизу вверх
private void DeepVerifyCheckStates(FileSystemNode node)
{
    if (node == null) return;

    // Сначала уходим на самую глубину к файлам и синтаксическим элементам
    foreach (var child in node.Children)
    {
        DeepVerifyCheckStates(child);
    }

    // На обратном пути (снизу вверх) заставляем контейнеры обновить свое состояние
    if (!node.IsFile && node.Children.Count > 0)
    {
        node.VerifyCheckState();
    }
}

// Полностью заменяем старую команду GenerateContext на асинхронную
// ===== -->
// ЧАСТЬ 1: ИСПРАВЛЕННАЯ КОМАНДА ГЕНЕРАЦИИ С ЗАПУСКОМ ОВЕРЛЕЯ PROGRESS-->
// ===== -->
// Внутри MainViewModel.cs

// ИСПРАВЛЕНО: Привязываем команду к методу проверки прав на выполнение
[RelayCommand(CanExecute = nameof(CanGenerateContext))]
private async Task GenerateContext()
{
    if (SelectedModule == null || RootNode == null) return;

    // Считываем все галочки с экрана прямо в модель модуля перед сборкой
    SyncTreeWithModule();
    string extension = IsPdfFormat ? ".pdf" : ".txt";
    string safeFileName = GetSafeFileName(ModuleNameInput) + extension;

    if (string.IsNullOrEmpty(OutputPath) || string.IsNullOrEmpty(ModuleNameInput))
    {
        System.Windows.MessageBox.Show("Не заполнены критические данные: выходная папка или имя модуля.", "Ошибка", MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }

    var checkedFiles = new List<FileSystemNode>();
    ContextBuilderService.GetCheckedFilesExtended(RootNode, checkedFiles);

    if (checkedFiles.Count == 0)

```

```

{
    System.Windows.MessageBox.Show("Не выбрано ни одного фрагмента структуры для нарезки
контекста.", "Внимание", MessageBoxButton.OK, MessageBoxImage.Warning);
    return;
}

// Выводим флаги оверлея
IsProcessing = true;
ProgressValue = 0;
ProgressMax = checkedFiles.Count;
ProgressText = $"Подготовка к обработке {checkedFiles.Count} элементов...";
CurrentFileText = "";
_cts = new CancellationTokenSource();

var progressHandler = new Progress<ProgressReport>(report =>
{
    ProgressValue = report.CurrentIndex;
    ProgressText = $"Обработано: {report.CurrentIndex} из {report.TotalCount}";
    CurrentFileText = $"Раздел: {report.CurrentFileName}";
});

try
{
    string fullPath = Path.Combine(OutputPath, safeFileName);
    if (IsPdfFormat)
    {
        await ContextBuilderService.GeneratePdfContextFileAsync(OutputPath, safeFileName,
        PromptRules, ModuleRules, IncludeDirectoryStructure, RootNode,
        SelectedModule.CheckedEntries, progressHandler, _cts.Token);
    }
    else
    {
        await ContextBuilderService.GenerateContextFileAsync(OutputPath, safeFileName,
        PromptRules, ModuleRules, IncludeDirectoryStructure, RootNode,
        SelectedModule.CheckedEntries, progressHandler, _cts.Token);
    }

    if (File.Exists(fullPath))
    {
        System.Diagnostics.Process.Start("explorer.exe", $"/select,\"{fullPath}\"");
    }
}
catch (OperationCanceledException)
{
    System.Windows.MessageBox.Show("Операция сборки контекста была отменена пользователем.",
    "Отмена", MessageBoxButton.OK, MessageBoxImage.Information);
}
catch (Exception ex)
{
    System.Windows.MessageBox.Show($"Ошибка сборки файла: {ex.Message}", "Ошибка",
    MessageBoxButton.OK, MessageBoxImage.Error);
}
finally
{
    IsProcessing = false;
    _cts?.Dispose();
    _cts = null;
}

// Новый обязательный метод проверки для CanExecute
private bool CanGenerateContext()
{
    // Кнопка заблокирована, если идет загрузка проекта, обработка или пересчет размера
    токенов
    if (IsProjectLoading || IsProcessing || IsCalculatingSize) return false;
}

```

```

// Кнопка активна только если выбран модуль и построено дерево
return SelectedModule != null && RootNode != null;
}

// Асинхронно восстанавливает структуру только для тех файлов, которые были частично
выбраны в проекте
private async Task RestoreSavedSyntaxStructureAsync(ContextModule module, FileSystemNode
root)
{
if (module?.CheckedEntries == null || module.CheckedEntries.Count == 0 || root == null)
return;

// Собираем уникальные относительные пути файлов, для которых есть сохраненные записи
элементов
var filesToPreload = new HashSet<string>();
foreach (var entry in module.CheckedEntries)
{
if (!string.IsNullOrEmpty(entry.FilePath))
{
filesToPreload.Add(entry.FilePath);
}
}

// Запускаем фоновую задачу для каждого такого файла
foreach (var relPath in filesToPreload)
{
// Ищем узел этого файла в нашем построенном дереве
var fileNode = FindNodeByRelativePath(root, relPath);
if (fileNode != null)
{
// Асинхронно загружаем синтаксическую структуру файла в фоновом потоке
await PopulateSyntaxNodesAsync(fileNode);

// Точечно восстанавливаем галочки для элементов этого файла на основе JSON
RestoreEntriesCheckState(fileNode, module.CheckedEntries);
}
}
}

// Вспомогательный метод поиска узла файла в дереве по его относительному пути
private FileSystemNode? FindNodeByRelativePath(FileSystemNode current, string
relativePath)
{
if (current.IsFile && current.RelativePath.Equals(relativePath,
StringComparison.OrdinalIgnoreCase))
{
return current;
}

foreach (var child in current.Children)
{
var found = FindNodeByRelativePath(child, relativePath);
if (found != null) return found;
}

return null;
}

// Рекурсивный метод простановки галочек сохраненным элементам
private void RestoreEntriesCheckState(FileSystemNode node, List<SyntaxEntry>
savedEntries)
{
if (node.IsSyntaxNode)

```

```

{
// Проверяем, есть ли текущий элемент кода в списке сохраненных в JSON
var match = savedEntries.Find(e => e.FilePath.Equals(node.RelativePath,
StringComparison.OrdinalIgnoreCase)
&& e.EntryPath.Equals(node.EntryPath, StringComparison.Ordinal));
if (match != null)
{
// Восстанавливаем координаты и ставим галочку (без каскада вниз, чтобы не перетереть
детей)
node.SyntaxSpanInfo = match.SpanInfo;
node.SetChecked(true, updateChildren: false, updateParent: true);
}
}

// Проходим по детям (вглубь классов к методам)
// Делаем копию коллекции, чтобы избежать ошибок изменения во время перебора
var childrenCopy = new List<FileSystemNode>(node.Children);
foreach (var child in childrenCopy)
{
RestoreEntriesCheckState(child, savedEntries);
}
}

// Универсальный метод асинхронного парсинга файла и подселения синтаксических нод в
дерево
public async Task PopulateSyntaxNodesAsync(FileSystemNode fileNode)
{
// ИСПРАВЛЕНО: Проверяем, есть ли среди детей нода-заглушка "LoadingStub..."
bool hasStub = false;
for (int i = 0; i < fileNode.Children.Count; i++)
{
if (fileNode.Children[i].Name == "LoadingStub...")
{
hasStub = true;
fileNode.Children.RemoveAt(i); // Удаляем техническую заглушку перед парсингом
break;
}
}
}

// Если заглушки нет И в списке уже есть элементы – значит, файл уже был полностью
распарсен ранее, выходим
if (!hasStub && fileNode.Children.Count > 0) return;

// ===== -->
// ИСПРАВЛЕНО: Интеллектуальное определение расширения для кэша Google Docs-->
// ===== -->
// Если файл лежит в папке кэша googlecache или имеет виртуальное имя,
// принудительно выставляем ему расширение .gdoc, чтобы фабрика нашла нужный парсер
string ext = System.IO.Path.GetExtension(fileNode.FullPath);
if (fileNode.RelativePath.EndsWith(".gdoc", StringComparison.OrdinalIgnoreCase) ||
fileNode.FullPath.Contains("googlecache"))
{
ext = ".gdoc";
}

// Теперь фабрика увидит легитимный .gdoc и не сбросит выполнение!
if (!SyntaxParserFactory.IsSupported(ext)) return;

var parser = SyntaxParserFactory.GetParser(ext);
if (parser == null) return;

// Сюда выполнение теперь гарантированно дойдет, и точка останова загорится!
List<SyntaxEntry> entries = await parser.ParseFileAsync(fileNode.FullPath,
fileNode.RelativePath);

```

```

// Строим словарь уже существующих в UI виртуальных нод, чтобы не дублировать их при
парсинге
var existingNodes = new Dictionary<string, FileSystemNode>(StringComparer.Ordinal);
BuildExistingNodesMap(fileName, existingNodes);

// ===== -->
// ИСПРАВЛЕНО: ПОДКЛЮЧЕНИЕ МУЛЬТИЯЗЫЧНЫХ РЕСУРСОВ ДЛЯ ПРОЗЫ -->
// ===== -->
// ===== -->
// ЧАСТЬ 2: ИЕРАРХИЧЕСКОЕ ПОДСЕЛЕНИЕ ГЛАВ ВНУТРЬ РОДИТЕЛЬСКИХ ВКЛАДОК -->
// ===== -->
foreach (var entry in entries)
{
    if (existingNodes.ContainsKey(entry.EntryPath)) continue;

    // СТРОГОЕ ПРАВИЛО: Вкладка (Tab) – это папка со стрелочкой (IsFile = false).
    // Глава рассказа (Heading) или текстовая секция – это конечный элемент (IsFile = true).
    bool isLeaf = (entry.Type == EntryType.Heading || entry.Type == EntryType.Section);

    var newNode = new FileSystemNode
    {
        Name = entry.DisplayName.Trim(), // Чистое название без запекания префиксов
        RelativePath = entry.FilePath,
        FullPath = fileName.FullPath,
        IsFile = isLeaf, // Разделяем поведение папок и файлов в UI
        IsSyntaxNode = true,
        SyntaxType = entry.Type,
        SyntaxSpanInfo = entry.SpanInfo,
        EntryPath = entry.EntryPath,
        Parent = fileName,
        // АВТОРАЗВОРАЧИВАНИЕ: Если узел является вкладкой GoogleDoc, заставляем UI раскрыть его
        IsExpanded = (entry.Type == EntryType.Tab)
    };

    // Ищем родителя по косой черте (/) литературных вкладок рассказов
    string parentEntryPath = string.Empty;
    int lastSeparator = entry.EntryPath.LastIndexOf('/');

    if (lastSeparator > 0)
    {
        parentEntryPath = entry.EntryPath.Substring(0, lastSeparator);
    }

    // Пытаемся подселить к существующей родительской вкладке в UI
    if (!string.IsNullOrEmpty(parentEntryPath) && existingNodes.TryGetValue(parentEntryPath,
        out var parentUiNode))
    {
        newNode.Parent = parentUiNode;
        parentUiNode.Children.Add(newNode);
    }
    else
    {
        // Если родительской вкладки нет, это корень проекта
        fileName.Children.Add(newNode);
    }

    // Регистрируем ноду в карте, чтобы её могли найти её будущие дети (подглавы или главы)
    existingNodes[entry.EntryPath] = newNode;

    // Восстановление галочек
    if (fileName.IsChecked == true || (newNode.Parent != null && newNode.Parent.IsChecked ==
        true))
    {
        newNode.SetChecked(true, updateChildren: false, updateParent: false);
    }
}

```

```

}

fileNode.SetChecked(fileNode.IsChecked, updateChildren: false, updateParent: true);

}

// Вспомогательный метод для сбора карты уже существующих UI нод в файле
// Внутри MainViewModel.cs
private void BuildExistingNodesMap(FileSystemNode node, Dictionary<string,
FileSystemNode> map)
{
    if (node == null) return;

    // ИСПРАВЛЕНО: Регистрируем узел, если у него заполнен синтаксический путь,
    // либо если это корневой узел виртуального файла GoogleDoc/WordDoc
    if (!string.IsNullOrEmpty(node.EntryPath))
    {
        map[node.EntryPath] = node;
    }
    else if (!node.IsFile && node.IsSyntaxNode == false && !string.IsNullOrEmpty(node.Name))
    {
        // Фоллбэк-маркер для корневого контейнера книги
        map[node.Name] = node;
    }

    // Делаем локальную копию коллекции детей для безопасного прохода без конфликтов потоков
    var childrenCopy = new List<FileSystemNode>(node.Children);
    foreach (var child in childrenCopy)
    {
        BuildExistingNodesMap(child, map);
    }
}

// Внутри MainViewModel.cs
// Внутри MainViewModel.cs
// =====>
// ИСПРАВЛЕНО: БЕЗОПАСНЫЙ НАКАТ ГАЛОЧЕК ДЛЯ ПРОЗЫ БЕЗ ДУБЛИРОВАНИЯ НОД-->
// =====>
private void FastPreloadSavedEntries(ContextModule module, FileSystemNode fileNode)
{
    if (module == null || fileNode == null) return;

    // СЦЕНАРИЙ А: Для текстовых проектов (GoogleDoc / WordDoc) работаем по эталонному
    // дереву
    // СЦЕНАРИЙ А: Для текстовых проектов (GoogleDoc / WordDoc) работаем по эталонному
    // дереву
    if (SelectedProject?.Type == ProjectType.GoogleDoc || SelectedProject?.Type ==
    ProjectType.WordDoc)
    {
        if (module.CheckedEntries == null || module.CheckedEntries.Count == 0) return;

        var savedPaths = new HashSet<string>(
            module.CheckedEntries.Select(e => e.EntryPath.Replace('\\', '/')),
            StringComparer.OrdinalIgnoreCase
        );

        ApplyCheckedStatesFromConfigRecursive(fileNode, savedPaths);

        // ИСПРАВЛЕНО: Запускаем глубокий каскадный пересчет состояний снизу вверх
        // для абсолютно всех уровней вложенности вкладок и подвкладок!
        DeepVerifyAllCheckStates(fileNode);
    }
}

```

```

return;
}

// СЦЕНАРИЙ Б: Старая исходная логика генерации нод для проектов с исходным кодом C#
var savedEntries = module.CheckedEntries;
if (savedEntries == null || savedEntries.Count == 0) return;

fileNode.Children.Clear();
var nodesMap = new Dictionary<string, FileSystemNode>(StringComparer.OrdinalIgnoreCase);

foreach (var entry in savedEntries)
{
    bool isLeaf = (entry.Type == EntryType.Heading || entry.Type == EntryType.Section);

    var newNode = new FileSystemNode
    {
        Name = entry.DisplayName.Trim(),
        RelativePath = entry.FilePath,
        FullPath = fileNode.FullPath,
        IsFile = isLeaf,
        IsSyntaxNode = true,
        SyntaxType = entry.Type,
        SyntaxSpanInfo = entry.SpanInfo,
        EntryPath = entry.EntryPath,
        Parent = fileNode
    };

    string parentEntryPath = string.Empty;
    int lastDot = entry.EntryPath.LastIndexOf('.');
    if (lastDot > 0)
    {
        parentEntryPath = entry.EntryPath.Substring(0, lastDot);
    }

    if (!string.IsNullOrEmpty(parentEntryPath) && nodesMap.TryGetValue(parentEntryPath, out
    var parentUiNode))
    {
        newNode.Parent = parentUiNode;
        parentUiNode.Children.Add(newNode);
    }
    else
    {
        fileNode.Children.Add(newNode);
    }
    nodesMap[entry.EntryPath] = newNode;

    newNode.SetChecked(true, updateChildren: false, updateParent: false);
}

fileNode.VerifyCheckState();
}
/// <summary>
/// Рекурсивный каскадный пересчет состояний чекбоксов Tri-State снизу вверх
/// </summary>
private void DeepVerifyAllCheckStates(FileSystemNode node)
{
    if (node == null) return;

    // Сначала уходим в самую глубину дерева к последним потомкам
    foreach (var child in node.Children)
    {
        DeepVerifyAllCheckStates(child);
    }
}

```

```

// Когда вернулись снизу, принудительно заставляем текущий узел
// пересчитать свой статус на основе реального состояния его детей
node.VerifyCheckState();
}

// Вспомогательный рекурсивный метод поиска нод по EntryPath и активации галочек
// ===== -->
// ИСПРАВЛЕНО: ДВУХКАНАЛЬНАЯ ПРОВЕРКА ПУТЕЙ (С ТОЧКОЙ И СО СЛЭШЕМ) -->
// ===== -->
private void ApplyCheckedStatesFromConfigRecursive(FileSystemNode node, HashSet<string>
savedPaths)
{
    if (node == null) return;

    // 1. Нормализуем путь текущего узла из дерева (заменяем наклоны слэшей)
    string cleanNodePath = node.EntryPath.Replace('\\', '/');

    // 2. ИСПРАВЛЕНО: Создаем альтернативный вариант пути на случай, если в JSON
    // точка на конце превратилась в слэш: "Среди людей - основной текст/Глава 1/"
    string alternativeNodePath = cleanNodePath;
    if (cleanNodePath.EndsWith("."))
    {
        // Отрезаем точку и принудительно дописываем слэш, имитируя баг сохранения
        alternativeNodePath = cleanNodePath.Substring(0, cleanNodePath.Length - 1) + "/";
    }

    // 3. Проверяем вхождение по любому из двух каналов адресации
    if (node.IsSyntaxNode && (savedPaths.Contains(cleanNodePath) ||
        savedPaths.Contains(alternativeNodePath)))
    {
        node.SetChecked(true, updateChildren: false, updateParent: false);
    }

    // Рекурсивно уходим вглубь по коллекции вкладок и глав романа
    foreach (var child in node.Children)
    {
        ApplyCheckedStatesFromConfigRecursive(child, savedPaths);
    }
}

// Флаги управления видимостью и состоянием оверлея проектов
[ObservableProperty] private bool _isProjectOverlayVisible;
[ObservableProperty] private bool _isLocalProjectSelected = true; // RadioButton:
Локальная папка
[ObservableProperty] private bool _isCloudProjectSelected; // RadioButton: Google Doc

// Буферные поля для ввода данных внутри оверлея
[ObservableProperty] private string _editProjectNameInput = string.Empty;
[ObservableProperty] private string _editProjectRootPathInput = string.Empty; // Папка
или URL ссылки
[ObservableProperty] private string _editProjectOutputPathInput = string.Empty;

// Дополнительные свойства для трех типов проектов во ViewModel
[ObservableProperty] private bool _isLocalFolderSelected = true;
[ObservableProperty] private bool _isGoogleDocSelected;
[ObservableProperty] private bool _isWordDocSelected;

// Команда кнопки "Обзор" для источника данных (Папка или Word)
[RelayCommand]
private void BrowseProjectSource()
{

```

```

if (IsLocalFolderSelected)
{
    // Используем ваш стандартный диалог выбора папки (например, CommonOpenFileDialog)
    var dialog = new Microsoft.WindowsAPICodePack.Dialogs.CommonOpenFileDialog {
        IsFolderPicker = true };
    if (dialog.ShowDialog() ==
        Microsoft.WindowsAPICodePack.Dialogs.CommonFileDialogResult.Ok)
    {
        EditProjectRootPathInput = dialog.FileName;
    }
}
else if (IsWordDocSelected)
{
    // Открываем диалог выбора файлов .doc / .docx
    var dialog = new Microsoft.Win32.OpenFileDialog
    {
        Filter = "Документы MS Word (*.doc;*.docx)|*.doc;*.docx|Все файлы (*.*)|*.*"
    };
    if (dialog.ShowDialog() == true)
    {
        EditProjectRootPathInput = dialog.FileName;
    }
}

// Команда кнопки "Вставить" (Иконка  внутри текстового поля Google Doc)
[RelayCommand]
private void PasteUrlFromClipboard()
{
    if (Clipboard.ContainsText())
    {
        EditProjectRootPathInput = Clipboard.GetText()?.Trim() ?? string.Empty;
    }
}

// КНОПКА ОТМЕНА: Полная зачистка буферных полей и возврат в интерфейс
[RelayCommand]
private void CancelProjectOverlay()
{
    EditProjectNameInput = string.Empty;
    EditProjectRootPathInput = string.Empty;
    EditProjectOutputPathInput = string.Empty;
    IsProjectOverlayVisible = false;
}

// Дополнительный флаг для блокировки интерфейса оверлея во время скачивания кэша
[ObservableProperty] private bool _isProjectLoading;
// Новые свойства для отображения точного числового прогресса в оверлее
[ObservableProperty] private int _currentProgressValue = 0;
[ObservableProperty] private string _currentProgressStatus = string.Empty;
// КНОПКА ОК: Комплексная валидация с учетом перечисления ProjectType
[RelayCommand]
private async Task ConfirmSaveProjectConfig()
{
    string projName = EditProjectNameInput?.Trim() ?? string.Empty;
    string rootPath = EditProjectRootPathInput?.Trim() ?? string.Empty;
    string outputPath = EditProjectOutputPathInput?.Trim() ?? string.Empty;

    // 1. Валидация на пустые строки
    if (string.IsNullOrEmpty(projName) || string.IsNullOrEmpty(rootPath) ||
        string.IsNullOrEmpty(outputPath))
    {
        MessageBox.Show("Все поля обязательны для заполнения!", "Ошибка валидации",
            MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }
}

```

```

// 2. Проверка спецсимволы в имени проекта
char[] invalidChars = Path.GetInvalidFileNameChars();
if (projName.Any(c => invalidChars.Contains(c)))
{
    MessageBox.Show("Название проекта содержит недопустимые символы для имени файла!",
    "Ошибка валидации", MessageBoxButtons.OK, MessageBoxIcon.Warning);
    return;
}

// 3. Проверка на уникальность имени среди ДРУГИХ проектов
bool isDuplicate = Projects.Any(p => p.ProjectName.Equals(projName,
StringComparison.OrdinalIgnoreCase) && p != SelectedProject);
if (isDuplicate)
{
    MessageBox.Show($"Проект с названием \"{projName}\" уже существует!", "Ошибка
валидации", MessageBoxButtons.OK, MessageBoxIcon.Warning);
    return;
}

// 4. Проверка и создание выходной папки результатов
if (!Directory.Exists(outPath))
{
    try { Directory.CreateDirectory(outPath); }
    catch (Exception ex)
    {
        MessageBox.Show($"Не удалось создать выходную папку: {ex.Message}", "Ошибка диска",
        MessageBoxButtons.OK, MessageBoxIcon.Error);
        return;
    }
}

// ИСПРАВЛЕНО: Строго определяем тип проекта по актуальным радиокнопкам оверлея
ProjectType selectedType = ProjectType.Folder;
if (IsGoogleDocSelected)
{
    selectedType = ProjectType.GoogleDoc;
}
else if (IsWordDocSelected)
{
    selectedType = ProjectType.WordDoc;
}
else if (IsLocalFolderSelected)
{
    selectedType = ProjectType.Folder;
}

// 5. Проверка доступности источников перед фиксацией
if (selectedType == ProjectType.Folder && !Directory.Exists(rootPath))
{
    MessageBox.Show("Указанная локальная папка источника данных не существует!", "Ошибка
пути", MessageBoxButtons.OK, MessageBoxIcon.Error);
    return;
}

if (selectedType == ProjectType.WordDoc && !File.Exists(rootPath))
{
    MessageBox.Show("Указанный файл MS Word не найден на диске!", "Ошибка пути",
    MessageBoxButtons.OK, MessageBoxIcon.Error);
    return;
}

if (selectedType == ProjectType.GoogleDoc)
{
    if (string.IsNullOrEmpty(GoogleDownloader.ExtractDocumentId(rootPath)))
    {

```

```

string idErr = Application.Current.Resources["Str_Err_InvalidServiceAccount"] as string
?? "Строка не содержит валидный ID!";
MessageBox.Show(idErr, "ID Error", MessageBoxButton.OK, MessageBoxImage.Warning);
return;
}

IsProjectLoading = true;
try
{
// Подгружаем текст состояния из словаря локализации
ProgressValue = 40;
ProgressText = Application.Current.Resources["Str_Update_LoadingGoogleDoc"] as string ??
"Скачивание структуры из облака Google...";

var downloader = new GoogleDownloader();
string projectBaseDir = AppDomain.CurrentDomain.BaseDirectory;
bool success = await downloader.DownloadToCacheAsync(rootPath, projectBaseDir,
EditGoogleApiKeyInput?.Trim() ?? string.Empty);

if (!success)
{
IsProjectLoading = false;
string errorMsg = Application.Current.Resources["Str_Err_GoogleDocUnavailable"] as
string ?? "Google Документ недоступен!";
string errorTitle = Application.Current.Resources["Str_Err_NetworkTitle"] as string ??
"Ошибка сети";
MessageBox.Show(errorMsg, errorTitle, MessageBoxButton.OK, MessageBoxImage.Error);
return;
}
ProgressValue = 100;
ProgressText = Application.Current.Resources["Str_Update_BuildingTree"] as string ??
"Структура успешно синхронизирована!";
}
finally
{
IsProjectLoading = false;
}
}

// ... Завершение сохранения в модель конфигурации проекта и SilentSave ...
ProjectConfig? targetProject = SelectedProject;
if (targetProject == null || EditProjectNameInput != targetProject.ProjectName)
{
targetProject = new ProjectConfig();
Projects.Add(targetProject);
}
targetProject.ProjectName = projName;
targetProject.RootPath = rootPath;
targetProject.OutputPath = outputPath;
targetProject.Type = selectedType;
IsProjectOverlayVisible = false;
SelectedProject = targetProject;

RefreshTreeView(targetProject.Modules.FirstOrDefault()?.CheckedFiles ?? new
List<string>());
SilentSave();

}
// Изолированное буферное поле для безопасного ввода ключа в интерфейсе оверлея
[ObservableProperty] private string _editGoogleApiKeyInput = string.Empty;

// ===== -->
// ЧАСТЬ 1: РЕСТРУКТУРИЗАЦИЯ ОВЕРЛЕЕВ (ЧИСТАЯ ПРИВЯЗКА К CONFIG) -->
// ===== -->

```

```

// 1. Команда создания нового проекта: выставляет дефолтный аккаунт из настроек приложения
[RelayCommand]
private void CreateNewProjectConfig()
{
    // Сканируем системную папку, наполняя AvailableServiceAccounts чистыми Email
    LoadAvailableServiceAccounts();

    EditProjectNameInput = string.Empty;
    EditProjectRootPathInput = string.Empty;
    EditProjectOutputPathInput = string.Empty;

    // Подтягиваем сохраненный по умолчанию Email из общих настроек приложения
    string defaultEmail = Properties.Settings.Default.DefaultServiceAccountEmail ??
    string.Empty;

    // Если этот Email физически существует в нашей папке ключей — выбираем его
    if (!string.IsNullOrEmpty(defaultEmail) &&
    AvailableServiceAccounts.Contains(defaultEmail))
    {
        SelectedServiceAccount = defaultEmail;
    }
    else
    {
        // Если настроек нет или ключ удален — берем самый первый доступный из списка
        SelectedServiceAccount = AvailableServiceAccounts.FirstOrDefault() ?? string.Empty;
    }

    IsLocalFolderSelected = true;
    IsGoogleDocSelected = false;
    IsWordDocSelected = false;
    IsProjectOverlayVisible = true;
}

// 2. Команда редактирования проекта (Ромбик ◊): строго считывает ключ из ProjectConfig
[RelayCommand]
private void ShowProjectSettings()
{
    if (SelectedProject == null) return;

    LoadAvailableServiceAccounts();

    EditProjectNameInput = SelectedProject.ProjectName;
    EditProjectRootPathInput = SelectedProject.RootPath;
    EditProjectOutputPathInput = SelectedProject.OutputPath;

    IsLocalFolderSelected = (SelectedProject.Type == ProjectType.Folder);
    IsGoogleDocSelected = (SelectedProject.Type == ProjectType.GoogleDoc);
    IsWordDocSelected = (SelectedProject.Type == ProjectType.WordDoc);

    // СТРОГАЯ ПРИВЯЗКА: Вытаскиваем Email аккаунта напрямую из конфигурации выбранного проекта!
    if (SelectedProject.Type == ProjectType.GoogleDoc &&
    !string.IsNullOrEmpty(SelectedProject.GoogleApiKey))
    {
        SelectedServiceAccount = SelectedProject.GoogleApiKey;
    }
    else
    {
        SelectedServiceAccount = AvailableServiceAccounts.FirstOrDefault() ?? string.Empty;
    }

    IsProjectOverlayVisible = true;
}

```

```

// НОВАЯ КОМАНДА: Безопасный запуск браузера на странице создания ключей Google Console
[RelayCommand]
private void OpenGoogleConsole()
{
    try
    {
        string url = "https://console.cloud.google.com";

        // В .NET Core / .NET 9 для Process.Start требуется флаг UseShellExecute = true
        var psi = new System.Diagnostics.ProcessStartInfo
        {
            FileName = url,
            UseShellExecute = true
        };
        System.Diagnostics.Process.Start(psi);
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Не удалось открыть браузер: {ex.Message}", "Ошибка запуска",
            MessageBoxButton.OK, MessageBoxImage.Error);
    }
}

// ОБНОВЛЕННАЯ КОМАНДА ИМПОРТА: С контролем дубликатов и автовыбором
[RelayCommand]
private void BrowseGoogleAuthJson()
{
    string filterStr = System.Windows.Application.Current.Resources["Str_Dialog_JsonFilter"]
        as string ?? "Ключ Google API (*.json)|*.json";
    string titleStr = System.Windows.Application.Current.Resources["Str_Dialog_JsonTitle"]
        as string ?? "Выберите JSON-ключ";

    var dialog = new Microsoft.Win32.OpenFileDialog
    {
        Filter = filterStr,
        Title = titleStr
    };

    if (dialog.ShowDialog() == true)
    {
        try
        {
            string fileContent = File.ReadAllText(dialog.FileName);
            var json = JObject.Parse(fileContent);
            string clientEmail = json["client_email"]?.ToString() ?? string.Empty;
            string projectId = json["project_id"]?.ToString() ?? "project";

            if (string.IsNullOrEmpty(clientEmail))
            {
                string msg =
                    System.Windows.Application.Current.Resources["Str_Err_InvalidServiceAccount"] as string
                    ?? "Ошибка валидации ключа.";
                string title = System.Windows.Application.Current.Resources["Str_Err_ValidationTitle"]
                    as string ?? "Ошибка";
                MessageBox.Show(msg, title, MessageBoxButton.OK, MessageBoxImage.Warning);
                return;
            }

            string systemStorageDir = Path.Combine(AppDomain.CurrentDomain.BaseDirectory,
                "ServiceAccounts");
            if (!Directory.Exists(systemStorageDir)) Directory.CreateDirectory(systemStorageDir);

```

```

// КОНТРОЛЬ ДУБЛИКАТОВ: Если такой Email уже импортирован, просто переключаемся на него
if (AvailableServiceAccounts.Contains(clientEmail))
{
    SelectedServiceAccount = clientEmail; // Триггерит OnSelectedServiceAccountChanged
    return;
}

string targetFileName = $"{projectId}_account.json";
string targetPath = Path.Combine(systemStorageDir, targetFileName);

File.Copy(dialog.FileName, targetPath, overwrite: true);

// Перечитываем хранилище
LoadAvailableServiceAccounts();

// ФИНАЛЬНЫЙ ШТРИХ: Делаем импортированный Email выбранным в ComboBox на лету!
SelectedServiceAccount = clientEmail;
}
catch (Exception ex)
{
    string msg = System.Windows.Application.Current.Resources["Str_Err_ImportFailed"] as
    string ?? "Не удалось импортировать ключ:";
    string title = System.Windows.Application.Current.Resources["Str_Err_ImportTitle"] as
    string ?? "Ошибка";
    MessageBox.Show($"{msg} {ex.Message}", title, MessageBoxButton.OK,
    MessageBoxImage.Error);
}
}
}

// ОБНОВЛЕННАЯ КОМАНДА КОПИРОВАНИЯ EMAIL ЧЕРЕЗ РЕСУРСЫ: Полностью локализована
[RelayCommand]
private void CopyServiceAccountEmailToClipboard()
{
    if (string.IsNullOrEmpty(SelectedServiceAccount)) return;

    try
    {
        // Нам больше не нужно парсить JSON, Email уже выбран в ComboBox!
        Clipboard.SetText(SelectedServiceAccount);

        string rawTemplate = System.Windows.Application.Current.Resources["Str_Msg_EmailCopied"]
        as string ?? "Email скопирован: {0}";
        string title = System.Windows.Application.Current.Resources["Str_Msg_EmailCopiedTitle"]
        as string ?? "Успех";

        MessageBox.Show(string.Format(rawTemplate, SelectedServiceAccount), title,
        MessageBoxButton.OK, MessageBoxImage.Information);
    }
    catch (Exception ex)
    {
        string msg = System.Windows.Application.Current.Resources["Str_Err_ExtractEmailFailed"]
        as string ?? "Не удалось извлечь Email:";
        string title = System.Windows.Application.Current.Resources["Str_Err_GeneralErrorTitle"]
        as string ?? "Ошибка";
        MessageBox.Show($"{msg} {ex.Message}", title, MessageBoxButton.OK,
        MessageBoxImage.Error);
    }
}

// ИСПРАВЛЕНО: Явное объявление частных полей для генератора MVVM свойств

```

```

[ObservableProperty] private ObservableCollection<string> _availableServiceAccounts =
new();
[ObservableProperty] private string _selectedServiceAccount = string.Empty;

// ИСПРАВЛЕНО: Добавлен метод сканирования хранилища, который искал компилятор
// ИСПРАВЛЕНО: Сканируем файлы и наполняем ComboBox чистыми Email-адресами
private void LoadAvailableServiceAccounts()
{
    AvailableServiceAccounts.Clear();
    string systemStorageDir = Path.Combine(AppDomain.CurrentDomain.BaseDirectory,
    "ServiceAccounts");

    if (!Directory.Exists(systemStorageDir))
    {
        Directory.CreateDirectory(systemStorageDir);
    }

    var files = Directory.GetFiles(systemStorageDir, "*.json");
    foreach (var file in files)
    {
        try
        {
            string content = File.ReadAllText(file);
            var json = JObject.Parse(content);
            string clientEmail = json["client_email"]?.ToString() ?? string.Empty;

            if (!string.IsNullOrEmpty(clientEmail) &&
            !AvailableServiceAccounts.Contains(clientEmail))
            {
                AvailableServiceAccounts.Add(clientEmail);
            }
        }
        catch { }
    }

    // ИСПРАВЛЕНО: При выборе Email находим соответствующий ему файл на диске
    partial void OnSelectedServiceAccountChanged(string value)
    {
        if (string.IsNullOrEmpty(value))
        {
            EditGoogleApiKeyInput = string.Empty;
            return;
        }

        string systemStorageDir = Path.Combine(AppDomain.CurrentDomain.BaseDirectory,
        "ServiceAccounts");
        var files = Directory.GetFiles(systemStorageDir, "*.json");
        string foundFullPath = string.Empty;

        // Бежим по всем файлам в поисках того, который содержит выбранный Email
        foreach (var file in files)
        {
            try
            {
                string content = File.ReadAllText(file);
                var json = JObject.Parse(content);
                if (json["client_email"]?.ToString() == value)
                {
                    foundFullPath = file;
                    break;
                }
            }
            catch { }
        }
    }

```

```

if (string.IsNullOrEmpty(foundFullPath))
{
    EditGoogleApiKeyInput = string.Empty;
    return;
}

// Записываем точный физический путь для сетевого Downloader
EditGoogleApiKeyInput = foundFullPath;

// Асинхронная проверка токена на лету
System.Windows.Application.Current.Dispatcher.BeginInvoke(new Func<Task>(async () =>
{
    try
    {
        string fileContent = await File.ReadAllTextAsync(foundFullPath);
        var json = JObject.Parse(fileContent);
        string clientEmail = json["client_email"]?.ToString() ?? string.Empty;
        string privateKeyRaw = json["private_key"]?.ToString() ?? string.Empty;

        if (string.IsNullOrEmpty(clientEmail) || string.IsNullOrEmpty(privateKeyRaw)) return;

        // Здесь при необходимости можно оставить вызов GetGoogleAccessTokenAsync
    }
    catch { }
}), System.Windows.Threading.DispatcherPriority.Background);
// ===== -->
// ЧАСТЬ 1: ЧИСТЫЙ СБОР ПУТЕЙ ДЛЯ СОХРАНЕНИЯ БЕЗ ИСКАЖЕНИЯ ТОЧКАМИ -->
// ===== -->
private void UpdateCheckedEntriesForCurrentModule()
{
    if (SelectedModule == null || RootNode == null) return;

    // Очищаем старый список перед перезаписью
    SelectedModule.CheckedFiles.Clear();

    // Запускаем безопасный сбор путей по всему дереву нод
    CollectEntriesRecursive(RootNode, SelectedModule.CheckedFiles);
}

private void CollectEntriesRecursive(FileSystemNode node, List<string> checkedList)
{
    if (node == null) return;

    // Если нода отмечена (или находится в неопределенном состоянии, но имеет синтаксис)
    if (node.IsSyntaxNode && (node.IsChecked == true || node.IsChecked == null))
    {
        // СТРОГОЕ ПРАВИЛО: Сохраняем оригинальный EntryPath, который построил парсер.
        // Никаких ручных склеек через точки! Парсер уже заложил туда "Вкладка/Глава 1."
        if (!checkedList.Contains(node.EntryPath))
        {
            checkedList.Add(node.EntryPath);
        }
    }

    // Идем глубже по дереву к дочерним элементам
    foreach (var child in node.Children)
    {
        CollectEntriesRecursive(child, checkedList);
    }
}

private CancellationTokenSource? _updateCts;

```

```

private UpdateType _lastUpdateType;
private string _lastTargetPath = string.Empty;

// === СВОЙСТВА УПРАВЛЕНИЯ ОВЕРЛЕЕМ ===

[ObservableProperty] private bool _isUpdateOverlayVisible; // Видимость всего Grid-
оверля
[ObservableProperty] private bool _isUpdateProcessing; // Видимость полосы прогресса
(ProgressBar)
[ObservableProperty] private bool _isRetryButtonVisible; // Видимость кнопки "Повторить"
[ObservableProperty] private bool _isCancelButtonEnabled = true; // Активность кнопки
"Отмена"
[ObservableProperty] private string _updateOverlayMessage = string.Empty; // Текст
внутри оверля

// === БИЗНЕС-ЛОГИКА ОБНОВЛЕНИЯ ===

/// <summary>
/// Публичный метод для инициализации процесса обновления проекта из любой точки
приложения
/// </summary>

// Модифицированное свойство доступности селекторов и кнопок управления модулями
public bool IsModuleSelectorEnabled
{
    get
    {
        // Кнопки и комбобоксы блокируются, если проект находится в режиме
загрузки/синхронизации (IsProjectLoading == true)
        if (IsProjectLoading) return false;

        return SelectedProject != null && Modules != null && Modules.Count > 0;
    }
}

// Дополнительное каноничное свойство для привязки к Command или IsEnabled кнопки
генерации в XAML:
public bool IsGenerateButtonEnabled
{
    get
    {
        if (IsProjectLoading || IsProcessing || IsCalculatingSize) return false;
        return SelectedModule != null && RootNode != null;
    }
}
// Добавьте этот метод перехвата в MainViewModel.cs для мгновенного обновления кнопок в
UI
// Внутри MainViewModel.cs

// Срабатывает при изменении статуса загрузки/синхронизации проекта Google Doc
partial void OnIsProjectLoadingChanged(bool value)
{
    GenerateContextCommand.NotifyCanExecuteChanged();
    OnPropertyChanged(nameof(IsModuleSelectorEnabled));
}

// Срабатывает при старте/окончании процесса сборки контекста
partial void OnIsProcessingChanged(bool value)
{
    GenerateContextCommand.NotifyCanExecuteChanged();
}

// Срабатывает при начале/окончании асинхронного подсчета токенов

```

```

partial void OnIsCalculatingSizeChanged(bool value)
{
    GenerateContextCommand.NotifyCanExecuteChanged();
}
// Внутри MainViewModel.cs

[RelayCommand]
private void DeleteCurrentProject(ProjectConfig? project)
{
    var targetProject = project ?? SelectedProject;
    if (targetProject == null) return;

    string title = Application.Current.Resources["Str_Title_Confirmation"] as string ??
        "Подтверждение";
    string rawMsg = Application.Current.Resources["Str_Msg_ConfirmDeleteProj"] as string ??
        "Удалить проект \"{0}\"?";
    string message = string.Format(rawMsg, targetProject.ProjectName);

    var result = MessageBox.Show(message, title, MessageBoxButton.YesNo,
        MessageBoxImage.Question);
    if (result == MessageBoxResult.Yes)
    {
        if (SelectedProject == targetProject)
        {
            SelectedProject = null;
        }
        Projects.Remove(targetProject);
        SilentSave();
    }
}

// === КОМАНДЫ (COMMUNITY TOOLKIT MVVM) ===

// ИСПРАВЛЕНО: Явное публичное свойство для перехвата кликов backdrop
[RelayCommand]
public void Dummy()
{
    // Пустой метод. Защищает центральное окноBorder от закрытия при клике на него.
}

[RelayCommand]
public void CancelUpdate()
{
    if (!IsCancelButtonEnabled) return;

    // Нажатие "Отмена" прерывает фоновую задачу (в т.ч. сетевой HttpClient)
    _updateCts?.Cancel();

    // Сброс видимости элементов
    IsUpdateOverlayVisible = false;
    IsUpdateProcessing = false;
    IsRetryButtonVisible = false;
}

[RelayCommand]
public async Task RetryUpdate()
{
    // ИСПРАВЛЕНО: Защита от дребезга кнопок и повторных параллельных запусков
    if (IsUpdateProcessing) return;
    await StartProjectUpdateAsync(_lastUpdateType, _lastTargetPath);
}

// === ИСПРАВЛЕННАЯ БИЗНЕС-ЛОГИКА ===
// ===== -->

```

```

// ЧАСТЬ 2: ГАРАНТИРОВАННАЯ ИНИЦИАЛИЗАЦИЯ СТАТУСОВ И ВЫЗОВ GOOGLE API-->
// ===== -->
public async Task StartProjectUpdateAsync(UpdateType type, string targetPath)
{
    _lastUpdateType = type;
    _lastTargetPath = targetPath;
    _updateCts = new CancellationTokenSource();

    // 1. Инициализируем базовое состояние UI-оверлея
    IsUpdateOverlayVisible = true;
    IsUpdateProcessing = true;
    IsRetryButtonVisible = false;
    IsCancelButtonEnabled = true;

    // 2. ИСПРАВЛЕНО: Безопасное извлечение строк из словаря локализации ДО тяжелых задач
    string statusKey = type switch
    {
        UpdateType.ReadDirectory => "Str_Update_ReadDir",
        UpdateType.GoogleApiLoad => "Str_Update_GoogleApi",
        UpdateType.ConvertDocx => "Str_Update_ConvertDocx",
        _ => "Str_Calculating"
    };

    // Принудительно вытаскиваем и пингуем UI-поток текстом статуса
    UpdateOverlayMessage = System.Windows.Application.Current.Resources[statusKey] as string
    ?? "Обработка...";

    try
    {
        switch (type)
        {
            case UpdateType.ReadDirectory:
                var fileSystem = new FileSystemService();
                await fileSystem.ScanDirectoryAsync(targetPath);
                break;

            case UpdateType.GoogleApiLoad:
                IsCancelButtonEnabled = true;

                if (!System.Net.NetworkInformation.NetworkInterface.GetIsNetworkAvailable())
                {
                    throw new System.Net.WebException("No internet connection");
                }

                if (SelectedProject != null && SelectedProject.Type == ProjectType.GoogleDoc)
                {
                    var downloader = new Google.GoogleDownloader();
                    string projectBaseDir = AppDomain.CurrentDomain.BaseDirectory;

                    // ИСПРАВЛЕНО: Защита от затирания Email. Если поле в проекте пустое –
                    // принудительно восстанавливаем его из локального хранилища настроек приложения
                    Settings!
                    string serviceAccountEmail = SelectedProject.GoogleApiKey;
                    if (string.IsNullOrEmpty(serviceAccountEmail))
                    {
                        serviceAccountEmail = Properties.Settings.Default.DefaultServiceAccountEmail;
                        // Сразу восстанавливаем значение и в самом проекте, чтобы баг больше не повторялся
                        SelectedProject.GoogleApiKey = serviceAccountEmail;
                    }

                    // Теперь извлечение пути по Email сработает гарантированно!
                    string keyPath = GetKeyPathByEmail(serviceAccountEmail);

                    if (string.IsNullOrEmpty(keyPath) || !File.Exists(keyPath))

```

```

{
// Передаем keyPath в конструктор, чтобы в логах отладки (из прошлого шага)
// вы точно видели, какой именно физический путь проверяет система
throw new FileNotFoundException("Файл ключа авторизации Google Docs не найден на
диске.", keyPath ?? "ПОЛУЧЕНА_ПУСТАЯ_СТРОКА");
}

using (var linkedCts =
CancellationTokenSource.CreateLinkedTokenSource(_updateCts.Token))
{
linkedCts.CancelAfter(TimeSpan.FromSeconds(15));

bool success = await downloader.DownloadToCacheAsync(
SelectedProject.RootPath,
projectBaseDir,
keyPath);

if (!success)
{
throw new System.Net.WebException("Google Docs API returned failure.");
}
}
break;

case UpdateType.ConvertDocx:
// Заглушка для будущего парсера Word
await Task.Delay(2000, _updateCts.Token);
break;
}

// Успешный исход — гасим оверлей
IsUpdateOverlayVisible = false;
}
catch (OperationCanceledException)
{
IsUpdateOverlayVisible = false;
}
catch (FileNotFoundException fnfEx)
{
// ИСПРАВЛЕНО: Выводим в отладку точный путь, на котором споткнулась программа!
System.Diagnostics.Debug.WriteLine($"[Google API Auth Error] Ключ не найден по пути:
{fnfEx.FileName}");

IsUpdateProcessing = false;
IsCancelButtonEnabled = true;

// Читаем из ресурсов строку об отсутствии доступа/ключа (или используем общую)
string errKey = "Str_Err_GoogleDocUnavailable";
UpdateOverlayMessage = System.Windows.Application.Current.Resources[errKey] as string
?? "Файл ключа авторизации Google Docs не найден на диске.";
}
catch (Exception ex) when (ex is IOException || ex is UnauthorizedAccessException || ex
is FormatException)
{
System.Diagnostics.Debug.WriteLine($"[IO Error Detail] {ex.GetType().Name}:
{ex.Message}");
IsUpdateProcessing = false;
IsCancelButtonEnabled = true;
string errKey = (type == UpdateType.ConvertDocx) ? "Str_Err_Update_Convert" :
"Str_Err_Update_ReadDir";
UpdateOverlayMessage = System.Windows.Application.Current.Resources[errKey] as string ??
"Ошибка чтения/записи диска.";
}
catch (Exception ex) when (ex is System.Net.WebException || ex is TaskCanceledException)
{

```

```

System.Diagnostics.Debug.WriteLine($"[Network Error Detail] {ex.Message}");
IsUpdateProcessing = false;
IsCancelButtonEnabled = true;
IsRetryButtonVisible = true;
UpdateOverlayMessage =
System.Windows.Application.Current.Resources["Str_Err_Update_Network"] as string ??
"Ошибка сети.";
}
}

// ===== -->
// ИСПРАВЛЕНО: ПОЛНАЯ МУЛЬТИЯЗЫЧНОСТЬ И ОЧИСТКА ХАРДКОДА СТРОК -->
// ===== -->
[RelayCommand]
private async Task RecreateGoogleDoc()
{
if (SelectedProject == null || SelectedProject.Type != ProjectType.GoogleDoc) return;

// 1. Извлекаем строго локализованные строки из XAML-ресурсов приложения
string confirmMsg = Application.Current.Resources["Str_Msg_ConfirmRecreateGoogle"] as
string
?? "Внимание! Это действие полностью удалит локальный кеш, очистит дерево и
принудительно скачает документ заново. Продолжить?";

// Используем ваш легитимный ключ заголовка из страницы 22/24 PDF кода
string confirmTitle = Application.Current.Resources["Str_Title_Confirmation"] as string
?? "Подтверждение действия";

// Показываем автору предупреждающее мультиязычное окно
var dialogResult = MessageBox.Show(confirmMsg, confirmTitle, MessageBoxButton.YesNo,
MessageBoxImage.Warning);
if (dialogResult != MessageBoxResult.Yes) return;

try
{
// 2. Уничтожаем локальный кэш
string docId = Google.GoogleDownloader.ExtractDocumentId(SelectedProject.RootPath);
if (!string.IsNullOrEmpty(docId))
{
string projectBaseDir = AppDomain.CurrentDomain.BaseDirectory;
string cacheDir = Path.Combine(projectBaseDir, "googlecache");

string jsonCachePath = Path.Combine(cacheDir, $"{docId}.json");
string metaCachePath = Path.Combine(cacheDir, $"{docId}.meta");

if (File.Exists(jsonCachePath)) File.Delete(jsonCachePath);
if (File.Exists(metaCachePath)) File.Delete(metaCachePath);
}

// 3. Обнуляем дерево и модули
if (RootNode != null)
{
RootNode.Children.Clear();
RootNode.Children.Add(new FileSystemNode { Name = "LoadingStub...", Parent = RootNode,
IsFile = true });
}

if (SelectedModule != null)
{
SelectedModule.CheckedFiles?.Clear();
SelectedModule.CheckedEntries?.Clear();
}
}

```

```

// ===== -->
// ЧАСТЬ 1: ИСПРАВЛЕННЫЙ ПОРЯДОК ОЖИДАНИЯ АСИНХРОННОГО СКАЧИВАНИЯ -->
// ===== -->
OnPropertyChanged(nameof(RootNode));

// 4. ЗАПУСКАЕМ НАШ СЕТЕВОЙ GRID-ОВЕРЛЕЙ ДЛЯ СКАЧИВАНИЯ С НУЛЯ
// ИСПРАВЛЕНО: Ждём (await) полного завершения скачивания и закрытия оверлея!
await StartProjectUpdateAsync(UpdateType.GoogleApiLoad, SelectedProject.RootPath);

// 5. ИСПРАВЛЕНО: Строим дерево ТЕПЕРЬ, когда файл гарантированно лежит в googlesache
// Передаем пустой список выбранных файлов, так как мы полностью пересоздали проект
RefreshTreeView(new List<string>());
}
catch (Exception ex)
{
string generalErrorTitle = Application.Current.Resources["Str_Err_GeneralErrorTitle"] as
string ?? "Ошибка";
MessageBox.Show($"{ex.Message}", generalErrorTitle, MessageBoxButton.OK,
MessageBoxImage.Error);
}
}

}

```

=== ПОЛНЫЙ ТЕКСТ ДОКУМЕНТА: MainWindow.xaml ===

```
<Window x:Class="ContextSlicer.MainWindow"
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
xmlns:local="clr-namespace:ContextSlicer.Filesystem"
mc:Ignorable="d"
Title="{DynamicResource Str_AppTitle}" Height="450" Width="780"
MinWidth="780" MinHeight="450"
WindowStartupLocation="CenterScreen"
Closing="Window_Closing"
Background="{DynamicResource WindowBackgroundBrush}"
Foreground="{DynamicResource TextBrush}" Icon="/ContentSlicer.ico">

<Window.Resources>
<ResourceDictionary>
<ResourceDictionary.MergedDictionaries>
<!-- Подгружаем дефолтный словарь, чтобы дизайнер Студии сразу видел все надписи -->
<ResourceDictionary Source="pack://application:,,,/Localization/Strings.ru.xaml"/>
</ResourceDictionary.MergedDictionaries>
<!-- 1. Текстовые блоки (метки) -->
<Style TargetType="TextBlock">
<Setter Property="Foreground" Value="{DynamicResource TextBrush}"/>
</Style>

<!-- 2. Стилизация рамок GroupBox -->
<Style TargetType="GroupBox">
<Setter Property="OverridesDefaultStyle" Value="True"/>
<Setter Property="Foreground" Value="{DynamicResource TextBrush}"/>
<Setter Property="BorderBrush" Value="{DynamicResource BorderBrush}"/>
<Setter Property="Background" Value="{DynamicResource ControlBackgroundBrush}"/>
<Setter Property="Template">
<Setter.Value>
<ControlTemplate TargetType="GroupBox">
<Border Background="{DynamicResource ControlBackgroundBrush}"
BorderBrush="{DynamicResource BorderBrush}" BorderThickness="1" CornerRadius="3"
Padding="5">
<Grid Background="{DynamicResource ControlBackgroundBrush}">
<Grid.RowDefinitions>
<RowDefinition Height="Auto"/>
<RowDefinition Height="*/>
</Grid.RowDefinitions>
<Border BorderThickness="0,0,0,1" BorderBrush="{DynamicResource BorderBrush}"
Padding="0,0,0,4" Margin="0,0,0,5">
<ContentPresenter ContentSource="Header" TextBlock.FontWeight="Bold"/>
</Border>
<ContentPresenter Grid.Row="1"/>
</Grid>
</Border>
</ControlTemplate>
</Setter.Value>
</Setter>
</Style>

<!-- 3. Чекбоксы и Радиокнопки -->
<Style TargetType="CheckBox">
<Setter Property="Foreground" Value="{DynamicResource TextBrush}"/>
<Setter Property="VerticalAlignment" Value="Center"/>
</Style>
<Style TargetType="RadioButton">
<Setter Property="Foreground" Value="{DynamicResource TextBrush}"/>
<Setter Property="VerticalAlignment" Value="Center"/>
<Setter Property="Margin" Value="0,0,15,0"/>
</Style>

<!-- 4. Текстовые поля ввода -->
```

```

<Style TargetType="TextBox">
<Setter Property="Background" Value="{DynamicResource TextBoxBackgroundBrush}"/>
<Setter Property="Foreground" Value="{DynamicResource TextBrush}"/>
<Setter Property="BorderBrush" Value="{DynamicResource BorderBrush}"/>
<Setter Property="BorderThickness" Value="1"/>
<Setter Property="Padding" Value="4,2"/>
<Setter Property="CaretBrush" Value="{DynamicResource TextBrush}"/>
</Style>

<!-- 5. Дерево файлов (TreeView) -->
<Style TargetType="TreeView">
<Setter Property="Background" Value="{DynamicResource ControlBackgroundBrush}"/>
<Setter Property="BorderBrush" Value="{DynamicResource BorderBrush}"/>
<Setter Property="BorderThickness" Value="1"/>
</Style>
<BooleanToVisibilityConverter x:Key="BooleanToVisibilityConverter"/>

<!-- ЕДИНСТВЕННЫЙ И ИСПРАВЛЕННЫЙ СТИЛЬ ДЛЯ ВСЕХ КНОПОК ФОРМЫ -->
<Style TargetType="Button">
<Setter Property="Background" Value="{DynamicResource TextBoxBackgroundBrush}"/>
<Setter Property="Foreground" Value="{DynamicResource TextBrush}"/>
<Setter Property="BorderBrush" Value="{DynamicResource BorderBrush}"/>
<Setter Property="Padding" Value="10,2"/>
<Setter Property="Height" Value="24"/>
<Setter Property="Template">
<Setter.Value>
<ControlTemplate TargetType="Button">
<!-- Основной контейнер кнопки берет фон напрямую из свойства Background самого элемента -->
<Border x:Name="mainBorder" Background="{TemplateBinding Background}"
BorderBrush="{TemplateBinding BorderBrush}" BorderThickness="1" CornerRadius="3">
<Grid>
<!-- Скрытый слой затемнения/высветления для реализации красивого эффекта Hover -->
<Border x:Name="hoverOverlay" Background="Transparent" CornerRadius="2"/>
<ContentPresenter HorizontalAlignment="Center" VerticalAlignment="Center"
Margin="{TemplateBinding Padding}"/>
</Grid>
</Border>
<ControlTemplate.Triggers>
<!-- ЭФФЕКТ НАВЕДЕНИЯ МЫШИ (ИСПРАВЛЕНО: работает универсально для всех типов кнопок) -->
<Trigger Property="IsMouseOver" Value="True">
<Setter TargetName="mainBorder" Property="BorderBrush" Value="#007ACC"/>
<!-- Полупрозрачный белый слой делает кнопку мягко светлее при наведении (отлично для
темной темы) -->
<Setter TargetName="hoverOverlay" Property="Background" Value="#15FFFFFF"/>
</Trigger>
<!-- ЭФФЕКТ НАЖАТИЯ (КЛИКА) -->
<Trigger Property="IsPressed" Value="True">
<!-- Мягкое затемнение при клике -->
<Setter TargetName="hoverOverlay" Property="Background" Value="#30000000"/>
</Trigger>
</ControlTemplate.Triggers>
</ControlTemplate>
</Setter.Value>
</Setter>
</Style>

<!-- ИСПРАВЛЕНО: Безопасный стиль выделения строк дерева без конфликтов типов данных -->
<Style TargetType="TreeViewItem">
<Setter Property="Foreground" Value="{DynamicResource TextBrush}"/>
<Style.Resources>
<!-- Полностью обнуляем дефолтные системные системные цвета выделения Windows Aero, -->
<!-- чтобы они не окрашивали ноду в ядовито-синий или черный цвет при фокусе -->
<SolidColorBrush x:Key="{x:Static SystemColors.HighlightBrushKey}" Color="Transparent"/>
<SolidColorBrush x:Key="{x:Static SystemColors.InactiveSelectionHighlightBrushKey}"
Color="Transparent"/>
</Style.Resources>

```

```

<Style.Triggers>
<!-- Перекрашиваем текст ноды в красивый акцентный синий цвет при её выделении -->
<Trigger Property="IsSelected" Value="True">
<Setter Property="Foreground" Value="#007ACC"/>
<Setter Property="FontWeight" Value="SemiBold"/>
</Trigger>
</Style.Triggers>
</Style>

<Style x:Key="ThemeComboBoxStyle" TargetType="ComboBox">
<Setter Property="Background" Value="{DynamicResource TextBoxBackgroundBrush}"/>
<Setter Property="Foreground" Value="{DynamicResource TextBrush}"/>
<Setter Property="BorderBrush" Value="{DynamicResource BorderBrush}"/>
<Setter Property="BorderThickness" Value="1"/>
<Setter Property="Padding" Value="6,3"/>
<Setter Property="ScrollViewer.HorizontalScrollBarVisibility" Value="Auto"/>
<Setter Property="ScrollViewer.VerticalScrollBarVisibility" Value="Auto"/>
<Setter Property="ScrollViewer.CanContentScroll" Value="True"/>

<!-- Полная замена системного шаблона для удаления голубого Win32 градиента -->
<Setter Property="Template">
<Setter.Value>
<ControlTemplate TargetType="ComboBox">
<Grid x:Name="MainGrid">
<ToggleButton x:Name="ToggleButton"
BorderBrush="{TemplateBinding BorderBrush}"
Background="{TemplateBinding Background}"
Grid.Column="2"
Focusable="false"
IsChecked="{Binding IsDropDownOpen, Mode=TwoWay, RelativeSource={RelativeSource
TemplatedParent}}">
ClickMode="Press">
<ToggleButton.Template>
<ControlTemplate TargetType="ToggleButton">
<Border x:Name="Border"
Background="{TemplateBinding Background}"
BorderBrush="{TemplateBinding BorderBrush}"
BorderThickness="1"
CornerRadius="3">
<!-- Рисуем стрелочку выпадающего списка справа -->
<Grid HorizontalAlignment="Right" Width="18">
<Path x:Name="Arrow"
Fill="{DynamicResource TextBrush}"
HorizontalAlignment="Center"
VerticalAlignment="Center"
Data="M 0 0 L 4 4 L 8 0 Z"/>
</Grid>
</Border>
<ControlTemplate.Triggers>
<!-- ИСПРАВЛЕНО: При наведении мышки на ComboBox подсвечиваем только рамку -->
<Trigger Property="IsMouseOver" Value="true">
<Setter TargetName="Border" Property="BorderBrush" Value="{DynamicResource
AccentBrush}"/>
</Trigger>
</ControlTemplate.Triggers>
</ControlTemplate>
</ToggleButton.Template>
</ToggleButton>

<!-- Отображение выбранного текста в свернутом состоянии -->
<ContentPresenter x:Name="ContentSite"
IsHitTestVisible="False"
Content="{TemplateBinding SelectionBoxItem}"
ContentTemplate="{TemplateBinding SelectionBoxItemTemplate}"
ContentTemplateSelector="{TemplateBinding ItemTemplateSelector}"
Margin="{TemplateBinding Padding}"
VerticalAlignment="Center"

```

```
HorizontalAlignment="Left" />
```

```
<!-- выпадающее всплывающее окно Popup -->
<Popup x:Name="Popup"
Placement="Bottom"
IsOpen="{TemplateBinding IsDropDownOpen}"
AllowsTransparency="True"
Focusable="False"
PopupAnimation="Slide">
<Grid x:Name="DropDown"
SnapsToDevicePixels="True"
MinWidth="{TemplateBinding ActualWidth}"
MaxHeight="{TemplateBinding MaxDropDownHeight}">
<Border x:Name="DropDownBorder"
Background="{DynamicResource TextBoxBackgroundBrush}"
BorderBrush="{DynamicResource BorderBrush}"
BorderThickness="1"
CornerRadius="0,0,3,3"
Margin="0,1,0,0"/>
<ScrollViewer Margin="4" SnapsToDevicePixels="True">
<StackPanel IsItemsHost="True" KeyboardNavigation.DirectionalNavigation="Contained"/>
</ScrollViewer>
</Grid>
</Popup>
</Grid>
<ControlTemplate.Triggers>
<!-- ИСПРАВЛЕНО: Когда список открыт, свернутый контрол остается темным, рамка горит синим! -->
<Trigger Property="IsDropDownOpen" Value="True">
<Setter TargetName="ToggleButton" Property="BorderBrush" Value="{DynamicResource AccentBrush}"/>
</Trigger>
<Trigger Property="HasItems" Value="false">
<Setter TargetName="DropDownBorder" Property="Height" Value="95"/>
</Trigger>
</ControlTemplate.Triggers>
</ControlTemplate>
</Setter.Value>
</Setter>

<!-- Стилизация элементов внутри выпадающего списка (ComboBoxItem) -->
<Setter Property="ItemContainerStyle">
<Setter.Value>
<Style TargetType="ComboBoxItem">
<Setter Property="Background" Value="Transparent"/>
<Setter Property="Foreground" Value="{DynamicResource TextBrush}"/>
<Setter Property="BorderThickness" Value="0"/>
<Setter Property="Padding" Value="6,3"/>
<Style.Triggers>
<Trigger Property="IsMouseOver" Value="True">
<Setter Property="Background" Value="{DynamicResource AccentBrush}"/>
<Setter Property="Foreground" Value="#FFFFFF"/>
</Trigger>
<Trigger Property="IsSelected" Value="True">
<Setter Property="Background" Value="{DynamicResource SplitterBrush}"/>
<Setter Property="Foreground" Value="{DynamicResource TextBrush}"/>
</Trigger>
</Style.Triggers>
</Style>
</Setter.Value>
</Setter>
</Style>
```

```

<BooleanToVisibilityConverter x:Key="BoolToVisConverter"/>
<!-- 2. НЕЯВНЫЙ СТИЛЬ СТРОК МЕНЮ (Применится ко всем MenuItem в окне, убирает
плейсхолдер иконок) -->
<!-- Внутри Window.Resources в MainWindow.xaml -->
<Style TargetType="ContextMenu">
<Setter Property="OverridesDefaultStyle" Value="True"/>
<Setter Property="Background" Value="{DynamicResource ControlBackgroundBrush}"/>
<Setter Property="BorderBrush" Value="{DynamicResource BorderBrush}"/>
<Setter Property="BorderThickness" Value="1"/>
<Setter Property="Template">
<Setter.Value>
<ControlTemplate TargetType="ContextMenu">
<Border Background="{TemplateBinding Background}"
BorderBrush="{TemplateBinding BorderBrush}"
BorderThickness="{TemplateBinding BorderThickness}"
CornerRadius="3" Padding="0,2">
<ItemsPresenter SnapsToDevicePixels="True" Grid.IsSharedSizeScope="False"/>
</Border>
</ControlTemplate>
</Setter.Value>
</Setter>
</Style>

<!-- 2. ИСПРАВЛЕНО: Дали имя стилю строк меню через x:Key во избежание
InvalidCastException -->
<Style TargetType="MenuItem">
<Setter Property="Foreground" Value="{DynamicResource TextBrush}"/>
<Setter Property="Background" Value="Transparent"/>
<Setter Property="Template">
<Setter.Value>
<ControlTemplate TargetType="MenuItem">
<Border x:Name="MenuRowBorder" Background="{TemplateBinding Background}" Padding="15,6"
SnapsToDevicePixels="True">
<ContentPresenter ContentSource="Header" VerticalAlignment="Center"
TextBlock.Foreground="{TemplateBinding Foreground}"/>
</Border>
<ControlTemplate.Triggers>
<Trigger Property="IsMouseOver" Value="True">
<Setter TargetName="MenuRowBorder" Property="Background" Value="{DynamicResource
SplitterBrush}"/>
</Trigger>
<Trigger Property="IsHighlighted" Value="True">
<Setter Property="FontWeight" Value="SemiBold"/>
</Trigger>
<Trigger Property="IsEnabled" Value="False">
<Setter Property="Opacity" Value="0.3"/>
</Trigger>
</ControlTemplate.Triggers>
</ControlTemplate>
</Setter.Value>
</Setter>
</Style>

<!-- 3. ИСПРАВЛЕНО: Дали имя стилю сепаратора -->
<Style TargetType="Separator">
<Setter Property="Background" Value="{DynamicResource SplitterBrush}"/>
<Setter Property="Margin" Value="0,4"/>
<Setter Property="Height" Value="1"/>
<Setter Property="Template">
<Setter.Value>
<ControlTemplate TargetType="Separator">
<Border Height="1" Background="{TemplateBinding Background}"
SnapsToDevicePixels="True"/>
</ControlTemplate>
</Setter.Value>
</Setter>
</Style>
</ResourceDictionary>
</Window.Resources>

```

```

<Grid Background="{DynamicResource WindowBackgroundBrush}">
<Grid.ColumnDefinitions>
<!-- Левая колонка: Вертикальный Экспандер с ListBox проектов -->
<ColumnDefinition Width="Auto"/>
<!-- Правая колонка: Вся основная рабочая область приложения -->
<ColumnDefinition Width="*" />
</Grid.ColumnDefinitions>

<!-- ===== -->
<!-- 1. СВОРАЧИВАЕМЫЙ ВЕРТИКАЛЬНЫЙ SIDEBAR (СПИСОК ПРОЕКТОВ) -->
<!-- ===== -->
<Expander Grid.Column="0" IsExpanded="True" ExpandDirection="Right"
Background="{DynamicResource PanelBackgroundBrush}"
BorderBrush="{DynamicResource BorderBrush}" BorderThickness="0,0,1,0">
<Expander.Header>
<TextBlock Text="ПРОЕКТЫ" Foreground="{DynamicResource TextBrush}" FontWeight="Bold"
FontSize="11">
<TextBlock.LayoutTransform>
<RotateTransform Angle="90"/>
</TextBlock.LayoutTransform>
</TextBlock>
</Expander.Header>

<Grid Width="220" Margin="5">
<Grid.RowDefinitions>
<RowDefinition Height="*" />
<RowDefinition Height="Auto" />
</Grid.RowDefinitions>
<!-- Идеально чистый ListBox в MainWindow.xaml -->
<!-- Идеально чистый ListBox в MainWindow.xaml (проверьте строку 316) -->
<ListBox Grid.Row="0" ItemsSource="{Binding Projects}" SelectedItem="{Binding
SelectedItem}"
PreviewMouseRightButtonDown="ListBox_PreviewRightMouseDown"
Background="{DynamicResource InputBackgroundBrush}"
BorderBrush="{DynamicResource BorderBrush}"
Foreground="{DynamicResource TextBrush}" Margin="0,0,0,5">

<ListBox.ItemContainerStyle>
<Style TargetType="ListBoxItem">
<Setter Property="Background" Value="Transparent" />
<Setter Property="Foreground" Value="{DynamicResource TextBrush}" />

</Style>
</ListBox.ItemContainerStyle>
<ListBox.ContextMenu>
<ContextMenu>
<!-- 1. Пункт: Обновить структуру (Доступен всегда) -->
<MenuItem Header="{DynamicResource Str_Mnu_UpdateProject}"
Command="{Binding PlacementTarget.DataContext.RetryUpdateCommand,
RelativeSource={RelativeSource AncestorType=ContextMenu}}" />

<!-- 2. ИСПРАВЛЕНО: Пункт "Очистить и пересоздать" (Появляется ТОЛЬКО для GoogleDoc) -->
<MenuItem Header="{DynamicResource Str_Mnu_RecreateGoogleDoc}"
Command="{Binding PlacementTarget.DataContext.RecreateGoogleDocCommand,
RelativeSource={RelativeSource AncestorType=ContextMenu}}">
<MenuItem.Style>
<Style TargetType="MenuItem" BasedOn="{StaticResource {x:Type MenuItem}}">
<Setter Property="Visibility" Value="Collapsed" />
<Style.Triggers>
<!-- Если тип выбранного проекта равен GoogleDoc (это тип 1 в enum или имя типа) -->
<DataTrigger Binding="{Binding PlacementTarget.SelectedItem.Type,
RelativeSource={RelativeSource AncestorType=ContextMenu}}" Value="GoogleDoc">
<Setter Property="Visibility" Value="Visible" />
</DataTrigger>
</Style.Triggers>
</Style>
</MenuItem.Style>

```

```

</MenuItem>

<Separator />

<!-- 3. Пункт: Удалить проект (Доступен всегда) -->
<MenuItem Header="{DynamicResource Str_Mnu_DeleteProject}"
Command="{Binding PlacementTarget.DataContext.DeleteProjectCommand,
RelativeSource={RelativeSource AncestorType=ContextMenu}}" />
</ContextMenu>
</ListBox.ContextMenu>

<ListBox.ItemTemplate>
<DataTemplate>
<StackPanel Orientation="Horizontal" Margin="2">
<TextBlock Text="{Binding ProjectTypeIcon}" Margin="0,0,6,0"
VerticalAlignment="Center"/>
<TextBlock Text="{Binding ProjectName}" VerticalAlignment="Center" FontWeight="SemiBold"
Foreground="{DynamicResource TextBrush}"/>
</StackPanel>
</DataTemplate>
</ListBox.ItemTemplate>
</ListBox>

<!-- ИСПРАВЛЕННАЯ КНОПКА СОЗДАНИЯ ПРОЕКТА: В стиле кнопок редактирования! -->
<!-- Она прозрачная, с легкой рамкой и ярким синим плюсом, видимым везде -->
<Button Grid.Row="1" Content="+" ToolTip="Создать новый проект"
Command="{Binding CreateNewProjectConfigCommand}"
Height="26" FontWeight="Bold" FontSize="18" Padding="0,0,0,3"
VerticalContentAlignment="Center" HorizontalContentAlignment="Center"
>
<Button.Style>
<Style TargetType="Button" BasedOn="{StaticResource {x:Type Button}}">
<Setter Property="Background" Value="Transparent"/>
<Setter Property="BorderBrush" Value="{DynamicResource BorderBrush}"/>
<Setter Property="Foreground" Value="{DynamicResource AccentBrush}"/>
</Style>
</Button.Style>
</Button>
</Grid>
</Expander>
<!-- ===== -->
<!-- ЧАСТЬ 2.2.1: ОСНОВНАЯ СЕТКА И ДВУХСТРОЧНАЯ ВЕРХНЯЯ ПАНЕЛЬ -->
<!-- ===== -->
<!-- ===== -->
<!-- ПОЛНАЯ И ИСПРАВЛЕННАЯ РАБОЧАЯ ОБЛАСТЬ (ВЕРХ, ДЕРЕВО, ПРАВИЛА, ПОДВАЛ) -->
<!-- ===== -->
<Grid Grid.Column="1" Margin="10,10,10,5">
<Grid.RowDefinitions>
<RowDefinition Height="Auto"/>
<!-- Строка 0: Верхняя панель -->
<RowDefinition Height="*/>
<!-- Строка 1: Дерево и правила -->
<RowDefinition Height="Auto"/>
<!-- Строка 2: Техническая панель -->
<RowDefinition Height="Auto"/>
<!-- Строка 3: Подвал -->
</Grid.RowDefinitions>

<!-- ВЕРХНЯЯ ПАНЕЛЬ КОНТЕКСТА -->
<GroupBox Grid.Row="0" Header="{DynamicResource Str_ModuleManagement}" Margin="0,0,0,10"
Padding="8"
Foreground="{DynamicResource TextBrush}" BorderBrush="{DynamicResource BorderBrush}">
<Grid>
<Grid.RowDefinitions>

```

```

<RowDefinition Height="Auto"/>
<RowDefinition Height="Auto" MinHeight="34"/>
</Grid.RowDefinitions>

<!-- СТРОКА 0: УПРАВЛЕНИЕ ПРОЕКТОМ -->
<Grid Grid.Row="0" Margin="0,0,0,8">
<Grid.ColumnDefinitions>
<ColumnDefinition Width="Auto"/>
<ColumnDefinition Width="Auto"/>
<ColumnDefinition Width="*/>
<ColumnDefinition Width="Auto"/>
</Grid.ColumnDefinitions>

<TextBlock Grid.Column="0" Text="{Binding SelectedProject.ProjectName,
FallbackValue='Проект не выбран'}"
Foreground="{DynamicResource TextBrush}" FontSize="15" FontWeight="Bold"
VerticalAlignment="Center" Margin="5,0,12,0"/>

<StackPanel Grid.Column="1" Orientation="Horizontal" VerticalAlignment="Center">
<Button Content="⌵" Tooltip="{DynamicResource Str_TooltipProjectEdit}"
Command="{Binding ShowProjectSettingsCommand}"
FontFamily="Segoe MDL2 Assets" FontSize="13" Width="28" Height="28" Padding="0"
Background="Transparent" BorderBrush="{DynamicResource BorderBrush}"
Foreground="{DynamicResource TextBrush}"
VerticalContentAlignment="Center" HorizontalContentAlignment="Center"/>
<Button Content="⌵" Tooltip="{DynamicResource Str_TooltipProjectSync}"
Command="{Binding SyncProjectSourceCommand}"
FontFamily="Segoe MDL2 Assets" FontSize="13" Width="28" Height="28" Margin="5,0,0,0"
Padding="0"
Background="Transparent" BorderBrush="{DynamicResource BorderBrush}"
Foreground="{DynamicResource TextBrush}"
VerticalContentAlignment="Center" HorizontalContentAlignment="Center"/>
</StackPanel>

<StackPanel Grid.Column="3" Orientation="Horizontal" VerticalAlignment="Center"
Margin="0,0,5,0">
<CheckBox Content="{DynamicResource Str_IncludeStructure}" IsChecked="{Binding
IncludeDirectoryStructure}"
Foreground="{DynamicResource TextBrush}" FontSize="12" Margin="0,0,16,0"
VerticalContentAlignment="Center"/>
<CheckBox Content="{DynamicResource Str_Autosave}" IsChecked="{Binding
IsAutoSaveEnabled}"
Foreground="{DynamicResource TextBrush}" FontSize="12"
VerticalContentAlignment="Center"/>
</StackPanel>
</Grid>

<!-- СТРОКА 1: УПРАВЛЕНИЕ МОДУЛЯМИ -->
<Grid Grid.Row="1" Margin="0,4,0,0">
<Grid.ColumnDefinitions>
<ColumnDefinition Width="Auto"/>
<ColumnDefinition Width="150"/>
<ColumnDefinition Width="200"/>
<ColumnDefinition Width="Auto"/>
<ColumnDefinition Width="Auto"/>
<ColumnDefinition Width="Auto"/>
<ColumnDefinition Width="*/>
</Grid.ColumnDefinitions>

<TextBlock Grid.Column="0" Text="{DynamicResource Str_SelectModule}"
Foreground="{DynamicResource TextBrush}"
VerticalAlignment="Center" Margin="5,0,10,0" FontWeight="SemiBold" FontSize="13"/>
<!-- Полностью очищенный от локальных стилей ComboBox модулей -->
<ComboBox Grid.Column="1" ItemsSource="{Binding Modules}" SelectedItem="{Binding
SelectedModule}"

```

```

DisplayMemberPath="ModuleName" Height="26" VerticalContentAlignment="Center"
Margin="0,0,10,0"
Style="{DynamicResource ThemeComboBoxStyle}"/>
<TextBox Grid.Column="2" Text="{Binding NewModuleNameInput,
UpdateSourceTrigger=PropertyChanged}"
Height="26" VerticalContentAlignment="Center" Margin="0,0,10,0" Padding="5,0"
Background="{DynamicResource TextBoxBackgroundBrush}" Foreground="{DynamicResource
TextBrush}"
BorderBrush="{DynamicResource BorderBrush}"/>

<Button Grid.Column="3" Content="+" ToolTip="{DynamicResource Str_TooltipModuleAdd}"
Command="{Binding CreateNewModuleCommand}"
Width="28" Height="26" FontWeight="Bold" FontSize="16" Padding="0,0,0,2"
Background="Transparent" BorderBrush="{DynamicResource BorderBrush}"
Foreground="{DynamicResource AccentBrush}"
VerticalContentAlignment="Center" HorizontalContentAlignment="Center"/>

<Button Grid.Column="4" Content="ⓧ" ToolTip="{DynamicResource
Str_TooltipModuleEdit}" Command="{Binding ShowEditOverlayCommand}"
FontFamily="Segoe MDL2 Assets" FontSize="11" Width="28" Height="26" Margin="5,0,0,0"
Padding="0"
Background="Transparent" BorderBrush="{DynamicResource BorderBrush}"
Foreground="{DynamicResource TextBrush}"
VerticalContentAlignment="Center" HorizontalContentAlignment="Center"
IsEnabled="{Binding IsModuleSelectorEnabled}"/>

<Button Grid.Column="5" Content="ⓧ" ToolTip="{DynamicResource
Str_TooltipModuleDelete}" Command="{Binding DeleteCurrentModuleCommand}"
FontFamily="Segoe MDL2 Assets" FontSize="12" Width="28" Height="26" Margin="5,0,5,0"
Padding="0"
Background="Transparent" BorderBrush="{DynamicResource BorderBrush}"
Foreground="{DynamicResource TextBrush}"
VerticalContentAlignment="Center" HorizontalContentAlignment="Center"
IsEnabled="{Binding IsModuleSelectorEnabled}"/>
</Grid>
</Grid>
</GroupBox>
<!-- ===== -->
<!-- ЧАСТЬ 2.2.2: ЦЕНТРАЛЬНАЯ РАБОЧАЯ ЗОНА (ВОЗВРАТ ДЕРЕВА И ПРАВИЛ) -->
<!-- ===== -->
<Grid Grid.Row="1" Margin="0,0,0,10">
<Grid.ColumnDefinitions>
<!-- Левая часть: Универсальное дерево TreeView -->
<ColumnDefinition Width="280"/>
<!-- Разделитель колонок (GridSplitter) -->
<ColumnDefinition Width="Auto"/>
<!-- Правая часть: Два больших поля правил -->
<ColumnDefinition Width="*" />
</Grid.ColumnDefinitions>

<!-- Наше синтаксическое/литературное дерево файлов -->
<TreeView Grid.Column="0" ItemsSource="{Binding RootNode.Children}"
TreeViewItem.Expanded="TreeView_Expanded"
Background="{DynamicResource ControlBackgroundBrush}" BorderBrush="{DynamicResource
BorderBrush}">
<!-- Шаблон иерархии (наш рабочий плоский вариант из прошлой задачи) -->
<TreeView.Resources>
<HierarchicalDataTemplate DataType="{x:Type local:FileSystemNode}" ItemsSource="{Binding
Children}">
<StackPanel Orientation="Horizontal" Margin="2">
<CheckBox IsChecked="{Binding IsChecked}" VerticalAlignment="Center" Margin="0,0,6,0"/>
<TextBlock Text="{Binding TypePrefixText}" Foreground="{DynamicResource AccentBrush}"
FontWeight="SemiBold" Margin="0,0,5,0">
<TextBlock.Style>
<Style TargetType="TextBlock">
<Style.Triggers>
<Trigger Property="Text" Value="">
<Setter Property="Visibility" Value="Collapsed"/>

```

```

</Trigger>
</Style.Triggers>
</Style>
</TextBlock.Style>
</TextBlock>
<TextBlock Text="{Binding Name}" Foreground="{DynamicResource TextBrush}"
VerticalAlignment="Center"/>
</StackPanel>
</HierarchicalDataTemplate>
</TreeView.Resources>
<TreeView.ItemContainerStyle>
<Style TargetType="TreeViewItem">
<!-- Двусторонняя привязка свойства авторазворачивания к модели -->
<Setter Property="IsExpanded" Value="{Binding IsExpanded, Mode=TwoWay}" />

<!-- Принудительный сброс стандартных серых фонов WPF (опционально для темной темы) -->
<Setter Property="Background" Value="Transparent"/>
</Style>
</TreeView.ItemContainerStyle>
</TreeView>

<!-- Вертикальный разделитель колонок -->
<GridSplitter Grid.Column="1" Width="4" HorizontalAlignment="Center"
VerticalAlignment="Stretch" Background="Transparent"/>

<!-- Зона правил (Два больших текстовых поля) -->
<Grid Grid.Column="2" Margin="5,0,0,0">
<Grid.RowDefinitions>
<RowDefinition Height="*/>
<!-- Глобальные правила проекта -->
<RowDefinition Height="Auto"/>
<!-- Сплиттер строк -->
<RowDefinition Height="*/>
<!-- Специфичные правила модуля -->
</Grid.RowDefinitions>

<!-- Глобальные правила -->
<GroupBox Grid.Row="0" Header="{DynamicResource Str_PromptHeader}" Padding="2"
Foreground="{DynamicResource TextBrush}" BorderBrush="{DynamicResource BorderBrush}">
<TextBox Text="{Binding PromptRules, UpdateSourceTrigger=PropertyChanged}"
TextWrapping="Wrap" AcceptsReturn="True"
Background="{DynamicResource TextBoxBackgroundBrush}" Foreground="{DynamicResource
TextBrush}"
BorderThickness="0" VerticalScrollBarVisibility="Auto"/>
</GroupBox>

<!-- Горизонтальный разделитель строк -->
<GridSplitter Grid.Row="1" Height="4" HorizontalAlignment="Stretch"
VerticalAlignment="Center" Background="Transparent"/>

<!-- Инструкции модуля -->
<GroupBox Grid.Row="2" Header="{DynamicResource Str_ModulePromptHeader}" Padding="2"
Foreground="{DynamicResource TextBrush}" BorderBrush="{DynamicResource BorderBrush}">
<TextBox Text="{Binding ModuleRules, UpdateSourceTrigger=PropertyChanged}"
TextWrapping="Wrap" AcceptsReturn="True"
Background="{DynamicResource TextBoxBackgroundBrush}" Foreground="{DynamicResource
TextBrush}"
BorderThickness="0" VerticalScrollBarVisibility="Auto"/>
</GroupBox>
</Grid>
</Grid>
<!-- ===== -->
<!-- ЧАСТЬ 2.2.3: ТЕХНИЧЕСКАЯ ПАНЕЛЬ И ПОДВАЛ (ФИНАЛ СЕТКИ ОКНА) -->
<!-- ===== -->
<Grid Grid.Row="2" Margin="0,0,0,10">
<Grid.ColumnDefinitions>

```

```

<ColumnDefinition Width="Auto"/>
<!-- Радиокнопки форматов -->
<ColumnDefinition Width="Auto"/>
<!-- Кнопка "Фильтры" -->
<ColumnDefinition Width="*/>
<!-- Разделитель и статистика -->
</Grid.ColumnDefinitions>

<!-- Переключатели формата итогового файла -->
<StackPanel Grid.Column="0" Orientation="Horizontal" VerticalAlignment="Center"
Margin="0,0,20,0">
<RadioButton Content="TXT" IsChecked="{Binding IsTxtFormat}"
Foreground="{DynamicResource TextBrush}" Margin="0,0,12,0"/>
<RadioButton Content="PDF" IsChecked="{Binding IsPdfFormat}"
Foreground="{DynamicResource TextBrush}" />
</StackPanel>

<!-- Кнопка вызова фильтров масок-исключений (Переведена на ресурсы) -->
<Button Grid.Column="1" Content="{DynamicResource Str_GlobalFilters}" Command="{Binding
ShowFiltersCommand}"
Padding="10,2" Background="Transparent" Foreground="{DynamicResource TextBrush}"
BorderBrush="{DynamicResource BorderBrush}" />

<!-- Панель статистики (Крупные контрастные цифры, меняющие цвет с темой) -->
<StackPanel Grid.Column="2" Orientation="Horizontal" HorizontalAlignment="Right"
VerticalAlignment="Center">
<Separator Style="{StaticResource {x:Static ToolBar.SeparatorStyleKey}}" Margin="15,0"/>

<TextBlock Text="{DynamicResource Str_StatsChars}" Foreground="{DynamicResource
TextBrush}" FontSize="13" FontWeight="SemiBold"/>
<TextBlock Text="{Binding TotalCharacters, FallbackValue=0}"
Foreground="{DynamicResource TextBrush}" FontSize="14" FontWeight="Bold"
Margin="0,0,20,0"/>

<TextBlock Text="{DynamicResource Str_StatsTokens}" Foreground="{DynamicResource
TextBrush}" FontSize="13" FontWeight="SemiBold"/>
<TextBlock Text="{Binding TotalTokens, FallbackValue=0}" Foreground="#4CAF50"
FontSize="14" FontWeight="Bold"/>
</StackPanel>
</Grid>

<!-- САМЫЙ ПОДВАЛ ОКНА -->
<Grid Grid.Row="3" Margin="0,5,0,0">
<Grid.ColumnDefinitions>
<ColumnDefinition Width="Auto"/>
<!-- Кнопка Сохранить -->
<ColumnDefinition Width="*/>
<!-- Кнопка Генерировать -->
<ColumnDefinition Width="Auto"/>
<!-- Чекбокс Темы -->
</Grid.ColumnDefinitions>

<!-- Кнопка ручного сохранения проекта -->
<Button Grid.Column="0" Content="{DynamicResource Str_SaveProject}" Command="{Binding
SaveProjectCommand}"
Width="170" Height="30" Background="Transparent" Foreground="{DynamicResource
TextBrush}" BorderBrush="{DynamicResource BorderBrush}" />

<!-- Кнопка генерации контекста в легком прозрачном стиле с рамкой -->
<Button Grid.Column="1" Content="{DynamicResource Str_Generate}" Command="{Binding
GenerateContextCommand}"
Height="30" Margin="10,0" FontWeight="Bold" FontSize="13"
VerticalContentAlignment="Center" HorizontalContentAlignment="Center">
<Button.Style>
<Style TargetType="Button" BasedOn="{StaticResource {x:Type Button}}">

```

```

<Setter Property="Background" Value="Transparent"/>
<Setter Property="BorderBrush" Value="{DynamicResource BorderBrush}"/>
<Setter Property="Foreground" Value="{DynamicResource AccentBrush}"/>
</Style>
</Button.Style>
</Button>

<!-- Переключатель темного/светлого оформления -->
<CheckBox Grid.Column="2" Content="{DynamicResource Str_DarkTheme}" IsChecked="{Binding
IsDarkTheme}"
Foreground="{DynamicResource TextBrush}" VerticalAlignment="Center"/>
</Grid>
</Grid>
<!-- ===== -->
<!-- ЧАСТЬ 1: СЕТКА ИНТЕРФЕЙСА ВВОДА ОВЕРЛЕЯ ПРОЕКТОВ (НАЧАЛО) -->
<!-- ===== -->
<Grid Panel.ZIndex="99" Grid.Column="0" Grid.ColumnSpan="2" Grid.Row="0"
Grid.RowSpan="4"
Visibility="{Binding IsProjectOverlayVisible, Converter={StaticResource
BooleanToVisibilityConverter}, FallbackValue=Collapsed}">

<!-- Полное полупрозрачное затемнение основного окна приложения -->
<Border Background="#A0000000"/>

<!-- Карточка модального окна: принудительно центрирована -->
<Border Width="460" Height="320"
Background="{DynamicResource ControlBackgroundBrush}"
BorderBrush="{DynamicResource BorderBrush}" BorderThickness="1.5"
CornerRadius="6" Padding="15"
HorizontalAlignment="Center" VerticalAlignment="Center">
<Grid>
<!-- СЛОЙ ИНТЕРФЕЙСА ВВОДА (Отрисовывается первым, на заднем плане) -->
<Grid x:Name="OverlayInputGrid">
<Grid.Style>
<Style TargetType="Grid">
<Setter Property="IsEnabled" Value="True"/>
<Style.Triggers>
<DataTrigger Binding="{Binding IsProjectLoading}" Value="True">
<Setter Property="IsEnabled" Value="False"/>
</DataTrigger>
</Style.Triggers>
</Style>
</Grid.Style>
<Grid.RowDefinitions>
<RowDefinition Height="Auto"/>
<RowDefinition Height="*/>
<RowDefinition Height="Auto"/>
</Grid.RowDefinitions>

<TextBlock Grid.Row="0" Text="{DynamicResource Str_ProjManagement}"
Foreground="{DynamicResource TextBrush}" FontWeight="Bold" FontSize="14"
Margin="5,0,0,15"/>

<Grid Grid.Row="1" Margin="0,0,0,5">
<Grid.RowDefinitions>
<RowDefinition Height="34"/>
<RowDefinition Height="34"/>
<RowDefinition Height="34"/>
<RowDefinition Height="Auto"/>
<RowDefinition Height="34"/>
</Grid.RowDefinitions>
<Grid.ColumnDefinitions>
<ColumnDefinition Width="135"/>
<ColumnDefinition Width="*/>
</Grid.ColumnDefinitions>

```

```

<!-- Название проекта -->
<TextBlock Grid.Row="0" Grid.Column="0" Text="{DynamicResource Str_ProjName}"
Foreground="{DynamicResource TextBrush}" VerticalAlignment="Center" Margin="5,0,0,0"/>
<TextBox Grid.Row="0" Grid.Column="1" Text="{Binding EditProjectNameInput,
UpdateSourceTrigger=PropertyChanged}" Height="24" VerticalContentAlignment="Center"/>

<!-- Переключатели типов проекта -->
<StackPanel Grid.Row="1" Grid.Column="1" Orientation="Horizontal"
VerticalAlignment="Center">
<RadioButton Content="Папка" IsChecked="{Binding IsLocalFolderSelected}"
Foreground="{DynamicResource TextBrush}"/>
<RadioButton Content="Google Doc" IsChecked="{Binding IsGoogleDocSelected}"
Foreground="{DynamicResource TextBrush}" Margin="12,0,0,0"/>
<RadioButton Content="MS Word" IsChecked="{Binding IsWordDocSelected}"
Foreground="{DynamicResource TextBrush}" Margin="12,0,0,0"/>
</StackPanel>

<!-- Путь к источнику данных -->
<TextBlock Grid.Row="2" Grid.Column="0" Text="{DynamicResource Str_SourceDataLabel}"
Foreground="{DynamicResource TextBrush}" VerticalAlignment="Center" Margin="5,0,0,0"/>
<Grid Grid.Row="2" Grid.Column="1">
<Grid.ColumnDefinitions>
<ColumnDefinition Width="*" />
<ColumnDefinition Width="Auto" />
</Grid.ColumnDefinitions>
<Grid Grid.Column="0">
<TextBox Text="{Binding EditProjectRootPathInput, UpdateSourceTrigger=PropertyChanged}"
Height="24" VerticalContentAlignment="Center" Padding="4,2,24,2"/>
<Button Content="□" FontSize="10" Width="22" Height="22" Padding="0" Command="{Binding
PasteUrlFromClipboardCommand}" ToolTip="Вставить ссылку" HorizontalAlignment="Right"
VerticalAlignment="Center" Margin="0,0,1,0" Background="Transparent"
BorderBrush="Transparent" Foreground="{DynamicResource TextBrush}">
<Button.Style>
<Style TargetType="Button" BasedOn="{StaticResource {x:Type Button}}">
<Setter Property="Visibility" Value="Collapsed"/>
<Style.Triggers>
<DataTrigger Binding="{Binding IsGoogleDocSelected}" Value="True">
<Setter Property="Visibility" Value="Visible"/>
</DataTrigger>
</Style.Triggers>
</Style>
</Button.Style>
</Button>
</Grid>
<Button Grid.Column="1" Content="{DynamicResource Str_Browse}" Command="{Binding
BrowseProjectSourceCommand}" Width="75" Margin="5,0,0,0" Height="24">
<Button.Style>
<Style TargetType="Button" BasedOn="{StaticResource {x:Type Button}}">
<Setter Property="IsEnabled" Value="True"/>
<Style.Triggers>
<DataTrigger Binding="{Binding IsGoogleDocSelected}" Value="True">
<Setter Property="IsEnabled" Value="False"/>
</DataTrigger>
</Style.Triggers>
</Style>
</Button.Style>
</Button>
</Grid>
<!-- ===== -->
<!-- ЧАСТЬ 2: СТРОКА GOOGLE KEY, ВЫГРУЗКА, КНОПКИ И СЛОЙ ЗАГРУЗКИ (ФИНАЛ) -->
<!-- ===== -->
<!-- Метка строки ключа (Показывается только если выбран Google Doc) -->
<TextBlock Grid.Row="3" Grid.Column="0" Text="Google API Key:"
Foreground="{DynamicResource TextBrush}" VerticalAlignment="Center" Margin="5,0,0,0">
<TextBlock.Style>
<Style TargetType="TextBlock">
<Setter Property="Visibility" Value="Collapsed"/>
<Style.Triggers>
<DataTrigger Binding="{Binding IsGoogleDocSelected}" Value="True">

```

```

<Setter Property="Visibility" Value="Visible"/>
</DataTrigger>
</Style.Triggers>
</Style>
</TextBlock.Style>
</TextBlock>

<!-- Контейнер ввода ключа (Показывается только если выбран Google Doc) -->
<Grid Grid.Row="3" Grid.Column="1" Height="34">
<Grid.Style>
<Style TargetType="Grid">
<Setter Property="Visibility" Value="Collapsed"/>
<Style.Triggers>
<DataTrigger Binding="{Binding IsGoogleDocSelected}" Value="True">
<Setter Property="Visibility" Value="Visible"/>
</DataTrigger>
</Style.Triggers>
</Style>
</Grid.Style>
<Grid.ColumnDefinitions>
<ColumnDefinition Width="*" />
<ColumnDefinition Width="Auto" />
<ColumnDefinition Width="Auto" />
</Grid.ColumnDefinitions>

<ComboBox Grid.Column="0" Height="24" VerticalContentAlignment="Center"
Style="{StaticResource ThemeComboBoxStyle}" ItemsSource="{Binding
AvailableServiceAccounts}" SelectedItem="{Binding SelectedServiceAccount,
UpdateSourceTrigger=PropertyChanged}" />
<Button Grid.Column="1" Content="□" FontSize="11" Width="32" Margin="5,0,0,0"
Height="24" Command="{Binding CopyServiceAccountEmailToClipboardCommand}"
ToolTip="Скопировать Email">
<Button.Style>
<Style TargetType="Button" BasedOn="{StaticResource {x:Type Button}}">
<Setter Property="IsEnabled" Value="False"/>
<Style.Triggers>
<DataTrigger Binding="{Binding SelectedServiceAccount, Converter={StaticResource
NullToBoolConverter}}" Value="True">
<Setter Property="IsEnabled" Value="True"/>
</DataTrigger>
</Style.Triggers>
</Style>
</Button.Style>
</Button>
<Button Grid.Column="2" Content="{DynamicResource Str_Browse}" Width="65"
Margin="5,0,0,0" Height="24" Command="{Binding BrowseGoogleAuthJsonCommand}"
ToolTip="Импортировать новый JSON-ключ" />
</Grid>

<!-- 4. Папка загрузки итогового контекста -->
<TextBlock Grid.Row="4" Grid.Column="0" Text="{DynamicResource Str_OutputPath}"
Foreground="{DynamicResource TextBrush}" VerticalAlignment="Center" Margin="5,0,0,0"/>
<Grid Grid.Row="4" Grid.Column="1">
<Grid.ColumnDefinitions>
<ColumnDefinition Width="*" />
<ColumnDefinition Width="Auto" />
</Grid.ColumnDefinitions>
<TextBox Grid.Column="0" Text="{Binding EditProjectOutputPathInput,
UpdateSourceTrigger=PropertyChanged}" Height="24" VerticalContentAlignment="Center"/>
<Button Grid.Column="1" Content="{DynamicResource Str_Browse}" Command="{Binding
BrowseProjectOutputFolderCommand}" Width="75" Margin="5,0,0,0" Height="24"/>
</Grid>
</Grid>

<!-- НИЖНИЕ КНОПКИ ДЕЙСТВИЙ -->
<StackPanel Grid.Row="2" Orientation="Horizontal" HorizontalAlignment="Right"
Margin="0,15,5,0">

```

```

<Button Content="{DynamicResource Str_BtnCancel}" Command="{Binding
CancelProjectOverlayCommand}" IsCancel="True" Width="85" Height="26" Margin="0,0,8,0"/>
<Button Content="{DynamicResource Str_BtnAccept}" Command="{Binding
ConfirmSaveProjectConfigCommand}" IsDefault="True" Width="85" Height="26">
<Button.Style>
<Style TargetType="Button" BasedOn="{StaticResource {x:Type Button}}">
<Setter Property="Background" Value="Transparent"/>
<Setter Property="BorderBrush" Value="{DynamicResource BorderBrush}"/>
<Setter Property="Foreground" Value="{DynamicResource AccentBrush}"/>
<Style.Triggers>
<Trigger Property="IsEnabled" Value="False">
<Setter Property="Opacity" Value="0.4"/>
</Trigger>
</Style.Triggers>
</Style>
</Button.Style>
</Button>
</StackPanel>
</Grid>

```

```

<!-- ИСПРАВЛЕНО: СЛОЙ ЗАГРУЗКИ НА ПЕРЕДНЕМ ПЛАНЕ (Panel.ZIndex="100" и Grid.RowSpan="3")
-->
<!-- ИСПРАВЛЕНО: Привязали Value к вашему родному свойству ProgressValue -->
<Grid Grid.Row="0" Grid.RowSpan="3" Panel.ZIndex="100" Margin="-15"
Visibility="{Binding IsProjectLoading, Converter={StaticResource
BooleanToVisibilityConverter}, FallbackValue=Collapsed}">
<Border Background="{DynamicResource ControlBackgroundBrush}" CornerRadius="6"/>
<StackPanel VerticalAlignment="Center" HorizontalAlignment="Center">
<!-- ИСПРАВЛЕНО: Вернули стандартный детерминированный ProgressBar без лишних
конвертеров -->
<ProgressBar Width="180" Height="6" Minimum="0" Maximum="100"
Value="{Binding ProgressValue, FallbackValue=0}"
Background="{DynamicResource WindowBackgroundBrush}"
BorderThickness="0" Margin="0,0,0,12" IsIndeterminate="True"/>

```

```

<!-- ИСПРАВЛЕНО: Привязали Text к вашему родному свойству ProgressText -->
<TextBlock Text="{Binding ProgressText, FallbackValue='Синхронизация структуры...'}"
Foreground="{DynamicResource TextBrush}"
FontWeight="SemiBold" FontSize="12" HorizontalAlignment="Center"/>
</StackPanel>
</Grid>

```

```

</Grid>
</Border>
</Grid>
<!-- ===== -->
<!-- ГЛОБАЛЬНЫЙ ОВЕРЛЕЙ: ОБНОВЛЕНИЕ ПРОЕКТА И СЕТЕВЫЕ СВОИ -->
<!-- ===== -->
<!-- Размещать строго в самом конце корневого Grid внутри MainWindow.xaml -->
<Grid Grid.Row="0" Grid.RowSpan="5" Grid.Column="0" Grid.ColumnSpan="5"
Visibility="{Binding IsUpdateOverlayVisible, Converter={StaticResource
BoolToVisConverter}, FallbackValue=Collapsed}"
Background="#AA111111" Panel.ZIndex="9999"
HorizontalAlignment="Stretch" VerticalAlignment="Stretch">

```

```

<Grid.InputBindings>
<MouseBinding MouseAction="LeftClick" Command="{Binding CancelUpdateCommand}"/>
</Grid.InputBindings>

```

```

<Border Width="420" Height="220"
Background="{DynamicResource PanelBackgroundBrush}"
BorderBrush="{DynamicResource BorderBrush}"
BorderThickness="1" CornerRadius="5"
HorizontalAlignment="Center" VerticalAlignment="Center" Padding="20">

```

```

<Border.InputBindings>
<MouseBinding MouseAction="LeftClick" Command="{Binding DummyCommand}"/>
</Border.InputBindings>

<Grid>
<Grid.RowDefinitions>
<RowDefinition Height="Auto"/>
<RowDefinition Height="*/>
<RowDefinition Height="Auto"/>
</Grid.RowDefinitions>

<TextBlock Grid.Row="0" Text="{DynamicResource Str_Update_Title}"
FontWeight="Bold" FontSize="14" Foreground="{DynamicResource TextBrush}"
Margin="0,0,0,10"/>

<StackPanel Grid.Row="1" VerticalAlignment="Center" HorizontalAlignment="Center">
<TextBlock Text="{Binding UpdateOverlayMessage}" Foreground="{DynamicResource
TextBrush}"
FontSize="12" TextAlignment="Center" TextWrapping="Wrap" Margin="0,0,0,20"/>

<!-- ИСПРАВЛЕНО: Задан благородный темно-зеленый цвет (Foreground="#006400") для бегунка
-->
<ProgressBar IsIndeterminate="True" Width="220" Height="5"
Visibility="{Binding IsUpdateProcessing, Converter={StaticResource BoolToVisConverter}}"
Background="{DynamicResource InputBackgroundBrush}"
Foreground="#006400" BorderThickness="0"/>
</StackPanel>

<StackPanel Grid.Row="2" Orientation="Horizontal" HorizontalAlignment="Right"
Margin="0,10,0,0">
<!-- ИСПРАВЛЕНО: Кнопка "Повторить" получила вашу родную стилизацию -->
<Button Content="{DynamicResource Str_Update_BtnRetry}"
Command="{Binding RetryUpdateCommand}"
Visibility="{Binding IsRetryButtonVisible, Converter={StaticResource
BoolToVisConverter}}"
Style="{DynamicResource {x:Type Button}}"
Width="90" Height="26" Margin="0,0,8,0" FontWeight="SemiBold"/>

<!-- ИСПРАВЛЕНО: Кнопка "Отмена" приведена к общему кнопочному стилю темы -->
<Button Content="{DynamicResource Str_Update_BtnCancel}"
Command="{Binding CancelUpdateCommand}"
IsEnabled="{Binding IsCancelButtonEnabled}"
Style="{DynamicResource {x:Type Button}}"
Width="90" Height="26" IsCancel="True"/>
</StackPanel>
</Grid>
</Border>
</Grid>

</Grid>
<!-- Заккрытие самого корневого Grid окна из страницы 4 PDF -->
</Window>
<!-- Заккрытие Window -->

```

=== ПОЛНЫЙ ТЕКСТ ДОКУМЕНТА: MainWindow.xaml.cs ===

```
using ContextSlicer.Filesystem;
using System.Collections.ObjectModel;
using System.ComponentModel;
using System.Globalization;
using System.IO;
using System.Text;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Navigation;
using System.Windows.Shapes;

namespace ContextSlicer;

/// <summary>
/// Interaction logic for MainWindow.xaml
/// </summary>
public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeLanguage();
        InitializeComponent();
        DataContext = new MainViewModel();

        // Красим заголовок главного окна при старте (считываем текущее состояние темы из App)
        this.SourceInitialized += (s, e) => {
            bool isDark = (DataContext as MainViewModel)?.IsDarkTheme ?? false;
            InverseBooleanConverter.ApplyTitleBarColor(this, isDark);
        };

        // Импортируем функцию из системной библиотеки Windows для управления внешним видом окон
        [System.Runtime.InteropServices.DllImport("dwmapi.dll")]
        private static extern int DwmSetWindowAttribute(IntPtr hwnd, int attr, ref int
            attrValue, int attrSize);

        // Публичный метод, который сможет вызывать наша ViewModel при смене темы
        public void UpdateTitleBarColor(bool isDark)
        {
            try
            {
                var interopHelper = new System.Windows.Interop.WindowInteropHelper(this);
                IntPtr hwnd = interopHelper.Handle;

                if (hwnd == IntPtr.Zero) return;

                // В Windows 11 константа 35 отвечает за цвет фона заголовка (DWMWA_CAPTION_COLOR)
                int attribute = 35;

                // Переводим HEX-цвета в формат BGR, который понимает Windows:
                // #1E1E1E (темный) превращается в 0x1E1E1E
                // #F5F5F5 (светлый) превращается в 0xF5F5F5
                int colorBGR = isDark ? 0x1E1E1E : 0xF5F5F5;

                DwmSetWindowAttribute(hwnd, attribute, ref colorBGR, sizeof(int));
            }
            catch { }
        }
    }
}
```

```

// Дополнительно меняем цвет текста заголовка (DWMWA_TEXT_COLOR = 36), чтобы он
оставался контрастным
int textAttribute = 36;
int textColorBGR = isDark ? 0xF1F1F1 : 0x000000;
DwmSetWindowAttribute(hwnd, textAttribute, ref textColorBGR, sizeof(int));
}
catch { /* Игнорируем на старых версиях Windows 10, где этот атрибут не поддерживается
*/ }
}

private void InitializeLanguage()
{
// Определяем язык операционной системы Windows
string currentCulture = CultureInfo.CurrentUICulture.TwoLetterISOLanguageName;

// Выбираем нужный файл словаря (по умолчанию для всех – английский, для ru – русский)
string languageFile = currentCulture.Equals("ru", StringComparison.OrdinalIgnoreCase)
? "Strings.ru.xaml"
: "Strings.en.xaml";

try
{
var dict = new ResourceDictionary
{
Source = new Uri($"pack://application:,,,/Localization/{languageFile}",
UriKind.Absolute)
};

// Добавляем строковые ресурсы напрямую в контекст этого окна
this.Resources.MergedDictionaries.Add(dict);

// Синхронизируем заголовок окна из загруженного словаря
if (this.Resources.Contains("Str_AppTitle"))
{
this.Title = this.Resources["Str_AppTitle"] as string;
}
}
catch (Exception ex)
{
MessageBox.Show($"Failed to load language file: {ex.Message}", "Language Error",
MessageBoxButton.OK, MessageBoxImage.Warning);
}
}

private void Window_Closing(object sender, System.ComponentModel.CancelEventArgs e)
{
if (DataContext is MainViewModel vm)
{
vm.OnWindowClosing();
}
}

private void TxtProjectName_LostFocus(object sender, RoutedEventArgs e)
{
if (DataContext is MainViewModel vm &&
vm.UpdateCurrentProjectNameCommand.CanExecute(null))
{
vm.UpdateCurrentProjectNameCommand.Execute(null);
}
}

// Обработчик ручного раскрытия узла дерева (Ленивая загрузка)

```

```

private async void TreeView_Expanded(object sender, RoutedEventArgs e)
{
    if (e.OriginalSource is TreeViewItem tvi && tvi.DataContext is FileSystemNode node)
    {
        // Строгое условие: парсим только если внутри РЕАЛЬНО лежит наша текстовая заглушка
        if (node.Children.Any(c => c.Name == "LoadingStub..."))
        {
            if (DataContext is MainViewModel vm)
            {
                try
                {
                    // Метод удалит LoadingStub, вызовет ParseFileAsync и подселит главы-листья (IsFile=true)
                    await vm.PopulateSyntaxNodesAsync(node);
                    node.VerifyCheckState();
                }
                catch (Exception ex)
                {
                    System.Diagnostics.Debug.WriteLine($"[TreeView Expand Error] {ex.Message}");
                }
            }
        }
    }
}

private void TxtModuleName_TextChanged(object sender, TextChangedEventArgs e)
{
}

private void ListBox_PreviewRightMouseDown(object sender, MouseButtonEventArgs e)
{
    var listBox = sender as ListBox;
    if (listBox == null) return;

    // Шаг 1. Находим контейнер строки (ListBoxItem), над которым находится курсор
    DependencyObject dep = (DependencyObject)e.OriginalSource;
    while (dep != null && dep != listBox && !(dep is ListBoxItem))
    {
        dep = VisualTreeHelper.GetParent(dep);
    }

    // Отсечка: клик по пустому месту — игнорируем
    if (!(dep is ListBoxItem item))
    {
        e.Handled = true;
        return;
    }

    // Шаг 2. Принудительно выделяем строку проекта в UI
    item.IsSelected = true;

    // Шаг 3. ИСПРАВЛЕНО: Безопасный поиск конкретного пункта меню по имени через контекст
    if (item.ContextMenu != null && item.DataContext is ProjectConfig project)
    {
        // Перебираем элементы коллекции Items и ищем нужный нам MenuItem
        foreach (var menuObject in item.ContextMenu.Items)
        {
            if (menuObject is MenuItem mnuItem && mnuItem.Name == "MnuRecreateGoogle")
            {
                // Пункт отображается только для проектов типа GoogleDoc
                mnuItem.Visibility = (project.Type == ProjectType.GoogleDoc)
                    ? Visibility.Visible
                    : Visibility.Collapsed;
                break; // Элемент найден, прерываем цикл
            }
        }
    }
}

```

```

}
}
}

/// <summary>
/// Обработчик пункта меню "Обновить структуру"
/// </summary>
/// Внутри MainWindow.xaml.cs

/// <summary>
/// ИСПРАВЛЕНО: Сигнатура приведена к универсальному RoutedEventArgs для полной
/// совместимости с автогенератором WPF
/// </summary>
private async void MenuUpdate_Click(object sender, RoutedEventArgs e)
{
    // Получаем нашу ViewModel из DataContext окна
    if (DataContext is MainViewModel viewModel)
    {
        // ИСПРАВЛЕНО: Безопасно вытаскиваем путь к текущему проекту.
        // Так как тип обновления зависит от источника данных, мы проверяем расширение или тип.
        // Для сценария ReadDirectory (вычитка каталогов кода/файлов):

        // В вашей конфигурации путь обычно лежит в настройках или выбранном проекте.
        // Передаем путь к корневой папке проекта:
        string projectPath = viewModel.SelectedProject?.RootPath ?? string.Empty;

        // Если у вас свойство называется по-другому (например, просто RootPath),
        // парсер try-catch оверлея всё равно отработает сбой, если путь окажется пустым.

        await viewModel.StartProjectUpdateAsync(
            ContextSlicer.Filesystem.UpdateType.ReadDirectory,
            projectPath
        );
    }
}

/// <summary>
/// ИСПРАВЛЕНО: Универсальная сигнатура клика для пункта "Удалить"
/// </summary>
private void MenuDelete_Click(object sender, RoutedEventArgs e)
{
    var vm = DataContext as MainViewModel;
    if (vm != null && vm.DeleteCurrentProjectCommand.CanExecute(null))
    {
        vm.DeleteCurrentProjectCommand.Execute(null);
    }
}

}

```

=== ПОЛНЫЙ ТЕКСТ ДОКУМЕНТА: ProjectConfig.cs ===

```
using ContextSlicer.Filesystem;
using System;
using System.Collections.Generic;

namespace ContextSlicer
{
    // Строгое перечисление типов поддерживаемых источников проекта
    public enum ProjectType
    {
        Folder = 0,
        GoogleDoc = 1,
        WordDoc = 2
    }

    public class ContextModule
    {
        public string ModuleName { get; set; } = string.Empty;
        public string ContextFileName { get; set; } = string.Empty;
        public string ModuleRules { get; set; } = string.Empty;
        public List<string> CheckedFiles { get; set; } = new List<string>();
        public List<SyntaxEntry> CheckedEntries { get; set; } = new List<SyntaxEntry>();
    }

    public class ProjectConfig
    {
        public string ProjectName { get; set; } = string.Empty;
        public string RootPath { get; set; } = string.Empty; // Папка, URL или путь к .docx
        public string OutputPath { get; set; } = string.Empty;
        public string PromptRules { get; set; } = string.Empty;
        public bool IncludeDirectoryStructure { get; set; } = true;
        public string GoogleApiKey { get; set; } = string.Empty;

        // ОБНОВЛЕНО: Вместо bool IsUrl теперь используем перечисление типов
        public ProjectType Type { get; set; } = ProjectType.Folder;

        public List<ContextModule> Modules { get; set; } = new List<ContextModule>();

        // ОБНОВЛЕНО: Интеллектуальный вывод иконок для ListBox на основе типа проекта
        [Newtonsoft.Json.JsonIgnore]
        public string ProjectTypeIcon => Type switch
        {
            ProjectType.GoogleDoc => "📄 ",
            ProjectType.WordDoc => "📄 ",
            _ => "📁 "
        };
    }
}
```